ZF-Microglia-AI — Complete User Guide

For zebrafish confocal microscopy — step by step, from zero to microglia labels.

See also: STATISTICS_GUIDE.md (the algorithm/formula behind every Statistics CSV column) · SOFTWARE_GUIDE.md (developer reference — every module and function explained in pseudocode).

Table of Contents

1. What this plugin does
2. Installation
3. Getting the model files
4. Opening the plugin in napari
5. Tab 1 — Skin Remover
6. Tab 2 — Create MG Labels
    - 6a. Which tool is active — Pixel Classifier or Cellpose-SAM?
    - 6b. Pixel Classifier — 3D Connected-Component Labels
    - 6c. Cellpose-SAM Segmentation
7. Tab 3 — Edit MG Labels
    - 7a. Layers and label(s) being edited
    - 7b. Correction Settings (Bbox / Auto-grow / Stability)
    - 7c. Sanding
    - 7d. Contrast low (best_lo) calibration
8. Tab 4 — Statistics
    - 8a. Analysing cells by brain region (optic tectum / hindbrain)
    - 8b. Intensity statistics per label
9. Tab 5 — AI Tools
    - 9a. GT Annotation
    - 9b. MONAI Training
    - 9c. Cellpose-SAM Training
10. Tab 6 — Sweeps & Utilities
    - 10a. Verify MONAI Threshold / Erosion (GT Sweep)
    - 10b. Verify BG Threshold / Signal Erosion (GT Sweep)

    - 10c. Calibrate Cellprob + Krendl Merge Parameters + Min size + Large-contact (GT Sweep)
    - 10d. Verify Best Epoch (GT Sweep)
    - 10e. Score Against GT
    - 10f. Build GT-Correction Package
    - 10g. Verify Smooth σ XY / σ Z (GT Sweep)
    - 10h. Email notification (optional)
    - 10i. Calibrate Correct-Label Contrast
    - 10j. Drift View in 3D
    - 10k. Autosave Labels Layer
    - 10l. GT Toolkit Tuning Tool
11. Output files and folder structure
12. Statistics CSV — all columns explained — for the algorithm/formula behind each column instead, see the separate STATISTICS_GUIDE.md
13. Setting up description backends
    - 13a. Setting up email notification (Gmail App Password)
14. Full workflow: from raw stack to labelled cells
    - Step 8a. Assign cells to optic tectum / hindbrain
15. Reinstalling after an update
16. Troubleshooting

---

1. What this plugin does

You have a confocal microscopy stack of a zebrafish brain. The image contains the brain you care about plus skin, tissue, and background surrounding it.

This plugin does two things, in order:

Step A — Skin Removal (Tab 1): Uses a trained AI model (MONAI 3D U-Net) to automatically detect and remove everything outside the brain, producing a clean `brain_only` image where only the cells of interest remain visible.

Step B — Label (Tab 2 — Create MG Labels): From the cleaned image, automatically finds and labels each individual cell as a separately numbered 3D region, using one of two methods — **Cellpose-SAM Segmentation**, a fine-tuned AI foundation model and the recommended choice, or the **Pixel Classifier**, an older-technology, threshold-based fallback for machines with no GPU. Each appears automatically once your Tab 1 output for it is open in the viewer; see

Section 6a. Editing tools (sort, split, join, correct, protect skin, auto-correct) live in **Tab 3 — Edit MG Labels** (Section 7) instead, behind one shared layer/label selector.

Step C — Analyse (Tab 4): Computes a comprehensive set of shape statistics for each labelled cell and exports them to a CSV file, with an optional AI-generated plain-language description per cell.

Beyond these three core steps, **Tab 3 — Edit MG Labels** (Section 7) gathers every edit/correct/modify tool for an existing Labels layer behind one shared layer/label selector, **Tab 5** launches GT annotation and MONAI/Cellpose-SAM training, and **Tab 6 — Sweeps & Utilities** (Section 10) collects every GT-verification sweep tool, the one-button GT Toolkit Tuning Tool and the related utilities in one place, so Tabs 1-4 stay focused on running the pipeline rather than tuning it.

> **Every numeric field with a slider next to it is directly editable** — click the number box and type an exact value instead of dragging the slider. Both stay in sync.

> **Most group boxes (the titled, bordered sections with a checkbox in their title) are collapsible** — the exceptions are Tab 2's Common Settings and Tab 3's selector, Correction Settings, Sanding and Contrast calibration boxes, which are always open. Click a collapsible group's title checkbox to hide its contents, freeing up vertical space so groups further down become reachable. Useful on smaller screens where a tab has more sections than fit on screen at once. Collapsing doesn't discard anything — every field keeps its value, and re-expanding restores it exactly as it was.

> **Most group boxes also open with a short description of what they actually do**, not just how to operate them — e.g. the Pixel Classifier and Cellpose-SAM Segmentation sections each explain their underlying pipeline before the controls. Each tab itself opens with the same kind of short overview too, above its first group.

> **Each tab also scrolls independently** — if a tab is taller than your napari window even with some groups collapsed, a vertical scrollbar appears on the right edge of the panel so you can reach everything below the fold. Only vertical scrolling is enabled; the panel's width work means nothing should ever need to scroll sideways.

---

2. Installation

You need Python with napari already installed. Open a terminal and run:

```
pip install git+https://github.com/CTichy/ZF-Microglia-AI.git
```

All dependencies (PyTorch, MONAI, scikit-image, etc.) are installed automatically.

> **Mac with Apple Silicon (M1/M2/M3):** The plugin automatically uses your GPU via Metal (MPS). No extra steps needed.

> **Windows / Linux with NVIDIA GPU:** CUDA is detected and used automatically for Tab 1 inference and Tab 2 GPU-accelerated labelling (`cupy-cuda12x`, which has wheels for both platforms). Tab 4's *fastest* statistics path additionally uses `cucim` for GPU batch regionprops — this only has Linux wheels (RAPIDS/cuCIM has no native Windows build), so on Windows Tab 4 automatically falls back to a CPU-threaded path instead; nothing breaks, statistics just run somewhat slower. See Section 16 for details.

> **No GPU:** Works on CPU too, just slower (~30–60 minutes per stack for inference).

---

3. Getting the model files

The plugin needs up to **two** trained checkpoints, depending on which labelling method you plan to use. Neither is bundled in the plugin.

Suggested layout — not required (the plugin remembers whatever path you browse to), but a tidy default if you'd rather not decide where to put things:

```
Documents/
└── zf-microglia-ai-models/
    ├── MONAI/
    │   └── best_model_fullstack.pth
    └── Cellpose/
        └── <your checkpoint>
```

MONAI skin-removal model (required — Tab 1)

The AI model (~220 MB) that powers skin removal.

1. Download it:

   `https://cloud.technikum-wien.at/s/kYQ4qq3Jsn4xEyY`

2. Save the file `best_model_fullstack.pth` into `MONAI/` (or anywhere else easy to find).

3. Open the plugin, go to **Tab 1**, click the model `[...]` Browse button, and select the file — see "Model (.pth) — Browse button" under Section 5 below for the exact steps.

The plugin remembers the path — you only need to do this once per installation.

Cellpose-SAM checkpoint (optional — Tab 2, only if using Cellpose-SAM Segmentation)

Only needed if you plan to label cells with **Cellpose-SAM Segmentation** rather than the **Pixel Classifier** (see Section 6a). This is a project-specific fine-tuned Cellpose-SAM model (~580 MB), branch-weighted 3-fish checkpoint (`multi3_bw`, epoch 150).

1. Download it:

   `https://cloud.technikum-wien.at/s/eFBJepk9DakDxyb`

2. Save the file `cpsam_microglia_512_multi3_bw_epoch_0150` into `Cellpose/` (or anywhere else easy to find).

3. Open the plugin, go to **Tab 2**'s Cellpose-SAM Segmentation section, click the model **Browse `[...]`** button, and select the file. The path is remembered the same way as the MONAI model path.

If you don't have a checkpoint yet, use the **Pixel Classifier** instead — it needs no additional model file.

---

4. Opening the plugin in napari

1. Open a terminal and type `napari` to launch it.
2. In the napari menu bar, click **Plugins**.
3. Click **ZF-Microglia-ToolKit (ZF-Microglia-AI)**.

4. A panel appears on the right side with tabs: **Skin Remover**, **Create MG Labels**, **Edit MG Labels** (every edit/correct/modify tool for an existing Labels layer, behind one shared layer/label selector — see Section 7), **Statistics** (always visible — shows an explanatory hint in place of its controls until at least one Labels layer exists), **AI Tools** (always available — shows a disclaimer instead of hiding the tab if your GPU is missing or under the recommended 8GB, see Section 9), and **Sweeps & Utilities** (Section 10).

---

5. Tab 1 — Skin Remover

Open TIF / IMS file

Click this button to open your confocal stack (`.tif`, `.tiff`, or `.ims` format).

- All channels in the file are loaded as separate napari layers, each coloured differently:
  - Channel 0 → gray
  - Channel 1 → green
  - Channel 2 → magenta
  - Channel 3 → cyan
- Voxel size (physical scale in µm) is read automatically from the file metadata and applied to all layers.
- The folder and filename are remembered for automatic output file naming (see Section 11).

> **Important:** After loading, **click on the channel you want to process** in the Layers panel on the left. **Run Skin-Remover** (and the Pixel Classifier's **Create Labels**) run on the selected Image layer — or the topmost Image layer if none is selected. For microglia, this is usually the green channel (ch1). The output folder and file names come from the most recently opened file (see Section 11).

Load Labels layer (.tif)

Loads a previously saved labels `.tif` (e.g. from Save Labels or a GT correction) directly as a Labels layer, without going through Create Labels again. Useful for reopening a labels file for further correction, or bringing in a hand-corrected ground-truth file to compare against.

- Picks any `.tif`/`.tiff` file — the resulting layer is named `<active Image layer's own name>_labels`, the same convention Create Labels itself uses (e.g. `..._brain_only_ExtRm_labels`), not the source TIFF's own filename. Loading the same file again replaces that layer rather than creating a duplicate. If no Image layer is open at all, falls back to the file's own name.
- The new layer's **scale** is set to match the currently open stack's own physical (µm) voxel scale — the same one **Open TIF / IMS file** (above) applied to its own Image layers — so the loaded labels line up correctly in 3D space with whatever stack is already in the viewer. **Open the full stack first**, or the labels layer loads at napari's own default (1, 1, 1) scale instead, with a warning in the status line.

Model (.pth) — Browse button `[...]`

Shows the path to the AI model file. If it says "— no model selected —":

1. Click the `[...]` button.
2. Navigate to where you saved the model file.
3. Select `best_model_fullstack.pth` and click Open.

The path is saved automatically to `~/.config/napari-zf-microglia-ai/config.json`. Next time you open the plugin, the model is already loaded.

Input info display

Below the model path, the plugin shows:

- The name and shape of the currently selected layer (e.g. `"NT54_ch1"  (300, 1024, 1024)  uint16`)
- The voxel dimensions: `Z=1.0000  Y=0.1740  X=0.1740 µm`
- The anisotropy ratio and the source of the scale information (from file metadata, from the layer scale, or default 1,1,1)

This is read-only — it updates automatically when you click a different layer.

MONAI Threshold

Range: 0.01 to 0.99 — **Default: 0.38**

The AI model outputs a probability map (0 = definitely not brain, 1 = definitely brain). This slider sets the cutoff: voxels above the threshold are classified as brain.

> Post-processing (largest connected component + hole filling) cleans up most artefacts regardless of threshold.

A read-only **Recommended MONAI Threshold** line sits underneath the slider — distinct from the slider's own live value, which stays freely editable. It reads "not yet measured" until **Verify MONAI Threshold / Erosion (GT Sweep)** (10a) is run through the **GT Toolkit Tuning Tool** (10l) with GT verified; that run sets both the line and the slider. Moving the slider afterward to try something else never touches this line, so the sweep's own finding is never silently lost.

MONAI Erosion (vox)

Range: 0 to 15 voxels — **Default: 0**

After the brain mask is computed, this many voxels are stripped inward from the mask edge before applying it to `brain_only`. This removes a thin skin rim.

- **0:** No erosion — use the mask exactly as computed.
- **2–3:** Typical for zebrafish — removes a ~0.3–0.5 µm rim in XY or 2–3 µm in Z.

> The `brain_mask.tif` saved to disk is **always the un-eroded mask**. Erosion only affects `brain_only.tif`.

A read-only **Recommended MONAI Erosion** line sits underneath, same idea as MONAI Threshold's above — updated only by **Verify MONAI Threshold / Erosion** (10a) run through the GT Toolkit Tuning Tool with GT verified. (The BG Threshold sweep tunes the separate **Signal Erosion** slider below, not this one.)

Background (brain mode)

Four radio buttons controlling how background signal is handled after inference.

The background level is estimated automatically using the **mode** (most common intensity) of pixels inside the brain, computed from the result of inference. The mode represents the baseline scanner noise because background pixels vastly outnumber bright cell pixels.

Off

No background processing. `brain_only` = original volume × the (eroded) brain mask. Everything outside the brain is zeroed; everything inside is the original signal unchanged.

1 — Remove background outside brain (inference)

Zeroes everything outside the (eroded) brain mask boundary and leaves the interior completely untouched — the same image as **Off** above (both use the eroded mask, so **MONAI Erosion** takes effect either way). The difference is the file suffix and that, like every mode other than Off, it also saves the raw input as `<layer>_original.tif`. **BG Threshold has no effect in this mode.**

The saved filename gets the suffix `_ExtRm` (e.g. `NT54_ch1_brain_only_ExtRm.tif`) — this is the input **Cellpose-SAM Segmentation** (6c) expects, since it needs the brain interior left intact.

2 — Remove background globally (full stack) ⭐ Recommended before labelling

Removes all pixels across the **entire stack** (including inside the brain) whose intensity falls at or below the background threshold, then erodes the surviving signal inward by **Signal Erosion** (see below), then zeroes everything outside the (eroded) brain mask — the only mode that uses BG Threshold, and the only mode that shows the Signal Erosion control.

Result: Only the actual signal (bright microglia, stained cells) survives. Background becomes zero everywhere, leaving clean isolated blobs with empty space between them — exactly what the Pixel Classifier's Create Labels algorithm needs.

Use this option before creating labels with the Pixel Classifier.

The saved filename gets the suffix `_NoBG`
(e.g. `NT54_ch1_brain_only_NoBG.tif`).

3 — Fill removed with random background

After skin removal, every voxel outside the (eroded) brain mask is filled with **uniform random noise** in a narrow window around the background level — the mode of the in-brain intensity histogram ± 0.1% of the stack's data range. The result looks like the original stack but with skin replaced by flat noise — no hard black boundary at the brain edge. Inside the brain the original signal is unchanged.

- BG Threshold is not used in this mode.

The saved filename gets the suffix `_RndFill`
(e.g. `NT54_ch1_brain_only_RndFill.tif`).

BG Threshold

Range: 0.00 to 2.00 — **Default: 1.025**

(Only active for background mode 2 — no effect in Off, mode 1, or mode 3)

Fine-tunes the background removal threshold. The value is a **percentage of the stack's data range** (max − min) added on top of the background level (the mode of the in-brain intensity histogram):

```
threshold = background_mode + (max − min) × BG_Threshold / 100
pixels ≤ threshold → removed (treated as background)
pixels  > threshold → kept (treated as signal)
```

Value	Effect
0.00	Threshold = exactly the mode — removes only confirmed background
1.025	**Default**
2.00 (max)	Aggressive — may remove dim signal from thin cell protrusions

> If microglia are losing thin protrusions, lower the value.

A read-only **Recommended BG Threshold** line sits underneath — updated only by **Verify BG Threshold / Signal Erosion** (10b) run through the GT Toolkit Tuning Tool with GT verified.

Signal Erosion (vox)

Range: 0 to 15 voxels — **Default: 0**

(Only shown for background mode 2 — hidden entirely in Off, mode 1, and mode 3)

Erodes the signal that survives BG Threshold's cutoff inward by this many voxels, per 2D Z-slice — a separate operation from the **MONAI Erosion (vox)** slider above, which erodes the brain-segmentation boundary itself, not the retained signal. Use this to strip a thin halo of background immediately around each cell before creating labels with the Pixel Classifier.

A read-only **Recommended Signal Erosion** line sits underneath, updated only by **Verify BG Threshold / Signal Erosion** (10b) run through the GT Toolkit Tuning Tool with GT verified.

Save checkboxes

- **Save brain_only.tif** (checked by default) — saves the brain-only volume with background removed
- **Save brain_mask.tif** (checked by default) — saves the binary mask as 0/255 uint8

Both files are saved zlib-compressed in the output folder (see Section 11). Whenever the Background mode is not Off, the raw, unmasked input volume is also always saved as `<layer>_original.tif` (no checkbox) — the raw signal that skin protection and auto-correct read. The `brain_only` filename includes a background-mode suffix:

Mode	Suffix	Example filename
Off	(none)	`NT54_ch1_brain_only.tif`
1 — Exterior Removed	`_ExtRm`	`NT54_ch1_brain_only_ExtRm.tif`
2 — No Background	`_NoBG`	`NT54_ch1_brain_only_NoBG.tif`
3 — Random Fill	`_RndFill`	`NT54_ch1_brain_only_RndFill.tif`

Run Skin-Remover

Click to start processing. The button is greyed out while running; the status bar shows one summary line, and the small live-output box underneath streams progress from **both** stages the run actually goes through — MONAI's own sliding-window progress (one line per processed window) during brain segmentation, followed by the background/skin-removal step's own messages (background level detected, threshold applied, voxel counts removed or filled) once that starts — the same progress you'd see running each step from a terminal. If the run takes longer than the email threshold, you get an email when it finishes or fails — see Section 10h.

When complete, two new layers appear in napari:

- `*_brain_mask` — binary mask in cyan, semi-transparent
- `*_brain_only[suffix]` — the cleaned volume

Processing time: - NVIDIA GPU: ~30 seconds - Apple Silicon (MPS): ~5–10 minutes - CPU only: ~30–60 minutes

---

Verify MONAI Threshold / Erosion (GT Sweep) — in Section 10a, Tab 6 — Sweeps & Utilities. Scores the Threshold/Erosion sliders above against a hand-corrected GT brain mask.

---

6. Tab 2 — Create MG Labels

> Before using this tab, run Tab 1 first. Which section of Tab 2 appears depends on which background mode you used — see 6a below.

6a. Which tool is active — Pixel Classifier or Cellpose-SAM?

Use Cellpose-SAM Segmentation (6c) if you have a GPU. It's a fine-tuned AI foundation model and handles branching, overlapping, and faint microglia far better than classical thresholding — it's the labelling method every real result in this project has actually used. The **Pixel Classifier (6b)** is an older, simpler threshold-and-stitch tool kept around as an initial aid for machines with no GPU at all; treat it as a fallback, not a first choice, when a GPU is available.

Each labelling section is shown whenever an Image layer with its matching suffix **exists anywhere in the viewer** — not only when it's the selected layer — so both can be visible at once (e.g. to compare routes on the same fish). A hint line at the top of the tab says which suffix it found.

An Image layer ending in … is open	Section shown	Produced by Tab 1 option
`_ExtRm`	**Cellpose-SAM Segmentation** (6c)	Option 1 — Remove background outside brain
`_NoBG`	**Pixel Classifier** (6b)	Option 2 — Remove background globally
both	both sections	—
neither (e.g. only `_RndFill`, or only the raw channel)	*Neither*, with a hint on what to select	—

So the choice is really made back in **Tab 1, Step 5**: pick **Option 1** if you plan to segment with Cellpose-SAM, or **Option 2** if you plan to use the Pixel Classifier.

The editing tools (Resort, Remove Debris, Split, Join, Correct Label, …) are in **Tab 3 — Edit MG Labels** (Section 7); they appear there once at least one Labels layer exists. **Tab 4 — Statistics** stays visible regardless, showing an explanatory hint in place of its controls until at least one Labels layer exists in the viewer, so a first-time user can still discover the tab is there.

Common Settings — shared by both routes

Sits in its own box, above the Pixel Classifier/Cellpose-SAM sections and **always visible regardless of which one is currently shown** — unlike everything documented under 6b/6c below, which only shows up when its own route's input layer is open. These fields were deliberately pulled out of the route-specific sections: a value that's actually shared, or that looks shared but genuinely is not, is easy to lose track of if it only appears half the time. Cellpose-SAM's own Min size field is a deliberate exception — see 6c for why it lives in that route-specific section instead, not here.

Min volume (vox) — informative, not editable

Shown as plain text, not a slider — this value can't be typed or dragged directly. It's the smallest real cell volume ever confirmed by GT, shared by the Pixel Classifier's own volume filter and Cellpose-SAM's Safe-merge "already a whole cell" floor (`gt_min_from_labels()`, Krendl's own name for this quantity, and `min_volume_from_gt()` are literally the same computation) — an empirical fact measured from ground truth, not a knob to hand-tune. It only ever moves *down* (a new fish can only prove an even smaller real cell exists, never invalidate one already confirmed), and only when the **GT Toolkit Tuning Tool** (10l) runs with GT verified — measured directly from `<stem>_GROUND_TRUTH.tif` in its first step, and again inside its BG Threshold and Smooth σ steps — never guessed, and never edited by hand. Starts at 7069 (the shipped default) until such a run proves an even smaller real cell exists.

The actual tunable control for how strictly this floor is enforced is **Final min-size fraction**, below.

> Zebrafish microglia at 4dpf typically occupy 15,000–50,000 voxels at standard resolution.

Max volume (vox) — informative, not editable

Shown the same way as Min volume above, right below it — plain text, not a slider. The largest real cell volume ever confirmed by GT, tracked as its never-falling mirror: it only ever moves *up* (a new fish can only prove an even bigger real cell exists, never invalidate one already confirmed), and only from the first step of a GT-verified **GT Toolkit Tuning Tool** run (10l), which reads the largest cell in `<stem>_GROUND_TRUTH.tif`. Reads "not yet measured" until at least one such run has happened.

Unlike Min volume, this doesn't drive any pipeline stage — nothing in either route deletes an oversized cell. It exists purely so `is_volume_outlier` in the Tab 4 Statistics CSV has something to flag against on the large side, the same way Min volume gives it something to flag against on the small side.

Min hole size (vox) — shared

Range: 0 to 500 (wider if a recommendation is larger) — **Default: 5** (0 fills every enclosed gap with no minimum)

Unlike Min/Max volume above, this one is a genuinely editable, **shared slider** used by both routes. A background region fully enclosed by signal — a "hole" — survives as real background only if its area is **at or above** this value; anything smaller is filled in as noise. Same idea as Min volume, just applied to gaps instead of whole objects, and named the same way on purpose: both name the size something must clear to be trusted as real, not the size at which it gets discarded.

The old behavior, unconditional hole-filling regardless of size, could silently erase real internal structure: a genuine gap inside a cell (debris exclusion, a real internal void) was treated identically to a single stray pixel of imaging noise, since neither route had any way to tell the two apart. Setting this above 0 draws that line explicitly, in both places it was missing:

- **Pixel Classifier** (Create Labels): holes in the smoothed mask are filled **per 2D Z-slice**, before the 3D connected-component step.
- **Cellpose-SAM Segmentation**: Cellpose's own installed library fills holes in each predicted mask's full **3D** volume in one step (`cellpose/utils.py`'s `fill_holes_and_remove_small_masks()`, via `fill_voids.fill()`), completely unconditionally. The plugin swaps in its own version only for the duration of each mask-formation call — see `_make_capped_fill_holes()` in `_cellpose_seg.py` — which keeps Cellpose's unconditional fill at 0, and above 0 fills holes per Z-slice by the same area rule, rather than editing the installed library, which would be lost on every reinstall.

Value	When to use
0	Fills every enclosed gap regardless of size
5	**Default** — only tiny gaps get filled
Measured from GT	The smallest confirmed-real hole size seen in hand-corrected ground truth — see below

> There is no universal correct number — real GT checked during development showed a sharp split between 1–2 voxel gaps (near-certainly annotation noise) and 400+ voxel gaps (clearly real structure), with nothing in between, so a value anywhere in that gap works for that fish; a different fish may look different.
>
> Measured from GT by the GT Toolkit Tuning Tool's first step and by both Pixel Classifier sweeps (10b/10g), and folded into the same never-rising floor (slider + read-only "Recommended floor" line) — but only when they run through the GT Toolkit Tuning

> Tool with GT verified. The Cellprob + Krendl calibration (10c) measures this fish's own value for its own mask-formation calls but never writes it to this shared floor.

Final min-size fraction — both routes

Range: 0.0 to 1.0 — **Default: 0.618 (the golden ratio, 1/φ)**

Unlike Min volume above, this one is a real, editable slider — it's the actual tunable control over how strictly the (non-editable) Min volume floor gets enforced. The actual small-object deletion cutoff **both** routes use is `this fraction × Min volume`, not Min volume itself — applied differently in each, since they have different pipeline shapes:

- **Cellpose-SAM (6c):** the very last stage, run after large-contact merge — any surviving cell below the cutoff is removed as a final safety net, regardless of how it survived every earlier stage. Nothing upstream is guaranteed to catch every debris object: GMM cleanup separates populations by the raw size distribution and can still leave a gray-zone object standing, and safe-merge/large-contact only act when a nearby neighbor exists to merge into.
- **Pixel Classifier (6b):** applied directly as the volume filter's own cutoff, right after 3D objects are formed by connected-component labelling — this route has no merge/reattach stage of its own, so there's no later stage to hand a gray-zone object off to; the relaxed cutoff has to be the filter itself.

The golden ratio is the default for a specific reason, not decoration: it needs to be strict enough that a fragment genuinely that much smaller than the smallest real GT cell ever measured is a defensible "almost certainly not a real cell" cutoff, while staying lenient enough not to reject a legitimately smaller-than-average real cell the way using Min volume itself (fraction = 1.0) would. Set to `1.0` to recover the old, unrelaxed behavior (cutoff == Min volume exactly) on either route; `0.0` disables the Cellpose-SAM safety-net stage entirely and, on the Pixel Classifier, removes its volume filter's floor altogether (every non-zero object survives).

Soften label contours (sanding) after any label correction

Default: on. The **Sigma XY**/**Sigma Z** sliders that control how strongly sanding blurs a label's edges, and the companion **Fill interior cavities (vesicles) before sanding** checkbox, live in **Tab 3 — Edit MG Labels** (7c), not here — this checkbox is the only sanding on/off control in Create MG Labels.

A shared setting used by every tool that regenerates a label's shape from scratch: **Correct Label** (2D or 3D), **Correct Adjacent Labels**, **Auto-correct Labels**, and Cellpose-SAM Segmentation's own **Auto-correct** stage (6c). All of these rebuild a label voxel-by-voxel from an intensity threshold, which can leave contours blocky or spiky at the pixel scale — and can leave a real intracellular vesicle/vacuole excluded as a literal hole, since it's genuinely dimmer than the threshold even though it's real cell material (see 7c for the fill step that fixes this). Sanding itself runs immediately after that fill step, on just the label(s) that tool touched: each one's own binary mask is 3D Gaussian-blurred (anisotropic — separate XY/Z sigma, from Tab 3's own sliders) and re-thresholded at 0.5, rounding off small jagged edges without meaningfully changing the cell's real shape or volume. Purely geometric — no image/intensity involved, unlike the correction itself.

Foreign-protected, same as every Correct Label tool: a neighboring label's already-claimed voxels can never be grown into, even where the blur would otherwise cross into them — so sanding one cell can never merge it into, or eat into, another. If a label's own shape shrinks to nothing under the blur (rare — usually means it was already tightly boxed in by neighbors) or loses all contact with its pre-sanding footprint, sanding is skipped for that label and the tool's status message says so; the correction itself is unaffected either way.

This is a **separate, independent setting from the Pixel Classifier's own Smooth σ XY/Z** (6b, default 2.60/1.00) — that pair decides whether raw blobs merge into one 3D object in the first place, before any labels exist. Sanding runs after labels already exist, only ever polishes one already-correct label's own edges, and is foreign-protected so it structurally cannot merge cells — which is why its default sigmas (0.7/0.7, Tab 3) are much smaller: this is meant to be a light "sand the edges and spikes down" pass, not a reshape.

Uncheck this box to skip sanding entirely and get the older, unsoftened behavior back on all four tools at once.

6b. Pixel Classifier — 3D Connected-Component Labels

On screen this group is titled **Pixel Classifier — 3D Connected-Component Labels**. It runs: binary mask (`volume > 0`) → 3D Gaussian smooth → re-threshold at 0.5 → per-slice hole-fill (floored at Min hole size, Common Settings) → true 3D connected components (26-connectivity) → volume filter → renumber 1…N, largest = 1. Dispatches to whichever backend is fastest on your machine (CUDA, Apple MPS via threaded CPU, or plain threaded CPU) automatically, and falls back to CPU if the CUDA run fails (e.g. out of memory). Shown when a `_NoBG` layer (background removed everywhere, not just outside the brain) is open — needs no additional model file.

Smooth σ XY

Range: 0.0 to 5.0 (one decimal) — **Default: 2.6**

Controls the softness of blob contours in the XY plane, as part of one 3D Gaussian blur applied to the whole volume before re-thresholding and connected-component labelling.

This rounds jagged pixel edges and fills tiny gaps within a given cross-section.

Value	Effect
0.0	No smoothing — raw pixel edges
2.6	**Default**
3.0+	Heavy — risk of merging nearby cells within the same slice

> Do not confuse with Smooth σ Z. They serve completely different purposes.

A read-only **Recommended Smooth σ XY** line sits underneath — updated only by **Verify Smooth σ XY / σ Z** (10g) run through the GT Toolkit Tuning Tool with GT verified.

Smooth σ Z

Range: 0.0 to 5.0 (one decimal) — **Default: 1.0**

Controls the Z-axis strength of the same 3D Gaussian blur — how much a thin gap between slices gets smoothed over before the volume is re-thresholded and grouped into 3D objects by true 3D connected components.

A microglia that disappears for 1–2 slices (due to low signal or a thin neck) and reappears will be correctly merged into one 3D object when σ Z is high enough to bridge that gap before thresholding.

> Zebrafish confocal stacks are highly anisotropic: each Z slice is ~1 µm thick while each XY pixel is ~0.17 µm, so a given voxel-unit σ spans a very different physical distance depending on axis. Both σ XY and σ Z are freely user-editable and GT-sweepable independently — don't assume one must stay a fixed multiple of the other.

Value	Effect
0.0	No cross-slice smoothing — each slice fully independent
0.5	Minimal — only adjacent slices with strong overlap connected
1.0	**Default**
5.0 (max)	Very aggressive — may link cells at different Z depths

A read-only **Recommended Smooth σ Z** line sits underneath, same as σ XY above.

Create Labels

Click to run the 3D labelling algorithm on the selected Image layer (or the topmost Image layer if none is selected — select your `_NoBG` layer first). Processing runs in a background thread — the button is disabled until complete, and the small live-output box underneath shows the same per-stage messages the console gets (backend used, signal voxel count, blobs found/removed), instead of only landing in a terminal you may not have open. The volume filter's actual cutoff is `Final min-size fraction × Min volume` (Common Settings, 6a), not Min volume alone — see that field for why.

When done, a `<layer>_labels` layer appears in napari with each detected cell shown in a different colour (not saved to disk — use Save Labels in Tab 3). The status line reports how many labels were found.

Verify BG Threshold / Signal Erosion (GT Sweep) — in Section 10b, Tab 6 — Sweeps & Utilities. Also measures the Min volume and Min hole size fields above directly from GT (running floors that only ever decrease) rather than leaving them guessed constants.

6c. Cellpose-SAM Segmentation

Shown when an `_ExtRm` layer (background removed only outside the brain — the interior is left intact for Cellpose-SAM to see) is open. Runs `do_3D` Cellpose-SAM inference, then a 3-component-GMM cleanup pass, a Krendl safe-merge pass (rejoins fragments smaller than Min volume based on gap size and contact area), a large-contact merge pass (catches cells accidentally split through a thick junction), and the final min-size safety net.

Signal layer

The one Image layer this section segments — an explicit dropdown, not "whatever is selected in the layer list". When nothing has been picked yet, it pre-selects the first open layer whose name ends in `_ExtRm`. The output layer, every saved file and the flow cache are named after this layer (`<signal layer>_cellpose_labels`, `<signal layer>_cp.tif`, …).

Min hole size, used by this route too, lives in the always-visible **Common Settings** box above (see 6a) rather than in this section — it stays visible there even while this section is hidden. **Min size** below is different: it's specific to this route, not shared, so it lives here instead.

Requires a Cellpose-SAM checkpoint — see Section 3. This is a project-specific fine-tuned model, not shipped with the plugin.

Model (.pt/checkpoint) — Browse button `[...]`

Browse to your trained Cellpose-SAM checkpoint file (checkpoints usually have no extension, so the dialog shows all files). A file that isn't a PyTorch checkpoint (zip archive) is refused with an error instead of being stored. The path is remembered across sessions, the same way the MONAI model path is remembered in Tab 1.

Min size (vox)

Range: 1 to 5000 — **Default: 15**

Cellpose-SAM's own early noise filter, applied right when raw instance masks are formed from the predicted flow field.

Not shared with Common Settings' Min volume — deliberately a different field, not the same value reused. Min volume is a route-agnostic GT-measured floor; nothing about it is specific to Cellpose-SAM. This field is: raw instance masks go through this small early filter, then **3-component GMM cleanup** and **Krendl safe-merge** — those two stages, not this field, make the real "is this debris, or a real fragment that should be reattached to a neighboring cell" decision, using the full size distribution and gap/contact geometry rather than one blunt cutoff. If Min size were raised anywhere near Min volume's range, real small fragments would be discarded here, before GMM cleanup or safe-merge ever got a chance to evaluate them for reattachment — quietly breaking the mechanism those two stages exist for. Leave this small; it only exists to catch prediction artifacts too small to be a fragment of anything.

Cellprob threshold

Range: -6.0 to 6.0 — **Default: 2.84**

Cellpose-SAM's own confidence cutoff for what counts as foreground (cell) vs. background. Lower (more negative) values are more permissive — they recover more of a cell's thin, dim protrusions but can also let in more noise. This calibration measures it against GT.

A read-only **Recommended Cellprob threshold** line sits underneath — updated only by **Calibrate Cellprob** + **Krendl Merge Parameters** + **Min size** + **Large-contact** (10c) run through the GT Toolkit Tuning Tool with GT verified. **Min size** above, and **Safe-merge max gap/min contact** and **Large-contact merge** below, have the same kind of line, updated by the same run.

Flow threshold

There is no Flow threshold field anywhere in this plugin, deliberately. In Cellpose generally, this parameter rejects predicted objects whose internal flow field doesn't self-consistently point back to a single centre — but Cellpose only applies that flow-error QC filter in 2D/stitch mode. Reading `cellpose/dynamics.py`'s `compute_masks()` shows the filter call sits inside `if not do_3D:`, and this plugin always runs `do_3D=True`, so the

parameter has zero effect on any result this plugin produces. Internally, `do_3D`'s function signature still requires a value, fixed at 0.4 and never exposed to the user.

Flow iterations (niter)

Range: 50 to 2000, step 50 — **Default: 200**

Euler-integration steps each voxel's flow trajectory gets before instances are formed. If a cell comes out looking porous ("pumice stone" — parallel banding per 2D slice) in 3D, try raising **Cellprob threshold** first (e.g. toward -0.3) rather than this field — a faint cell's raw probability map is noisy near a permissive threshold, letting marginal voxels flicker in and out slice by slice before flow-following even starts, which is the more common cause; raising Flow iterations alone rarely fixes it by itself, though the two together on a **Re-run This Cell Only** pass can help.

Safe-merge max gap (µm)

Range: 0.0 to 5.0 — **Default: 3.52**

During the Krendl safe-merge pass, two fragments separated by a real physical gap up to this many microns are considered for merging into one cell (in addition to the contact-area check below). Measured in µm, not voxels, so the same value applies consistently regardless of a fish's own Z/XY anisotropy.

A read-only **Recommended Safe-merge max gap** line sits underneath — see Cellprob threshold above.

Safe-merge min contact (vox)

Range: 0 to 200 — **Default: 1**

Minimum shared-boundary voxel count required between two fragments before the safe-merge pass will join them (an OR against the gap check above — either one qualifies a pair for merging). Higher values require a more substantial touching surface before merging.

A read-only **Recommended Safe-merge min contact** line sits underneath, same as Safe-merge max gap above.

Safe-merge “already a whole cell” floor

Not a field of its own here. The volume below which the safe-merge pass treats a fragment as *not yet* a whole cell and a candidate to merge into something else is exactly the same measurement as **Min volume** in Common Settings (6a) — the smallest true voxel volume ever confirmed in real GT. Lowered only by a GT-verified **GT Toolkit Tuning Tool** run (10l) — its own first step, plus its BG Threshold and Smooth σ steps — so you never set it by hand. The Cellprob + Krendl calibration (10c) measures this fish’s own `gt_min` for its own Large-contact step but never writes it to this shared floor.

Large-contact merge (vox)

Range: 1 to 2000 — **Default: 20**

A second, separate merge pass for large blobs that got split apart through a thick junction (more contact area than the safe-merge pass alone would normally join). Raise this if large cells are still coming out fragmented; lower it if separate cells are being wrongly joined.

A read-only **Recommended Large-contact merge** line sits underneath, same as Cellprob threshold above.

Final min-size safety net

The last stage of this pipeline, run automatically after large-contact merge — see **Final min-size fraction** in Common Settings (6a) for the field itself and the golden-ratio reasoning behind its default. Nothing above it is guaranteed to remove every possible debris object: GMM cleanup separates populations by the raw size distribution and can still leave a gray-zone object standing, and safe-merge/large-contact only act when a nearby neighbor exists to merge into. This stage is the backstop underneath all of them, not a replacement for any one of them.

Run Cellpose-SAM Segmentation (button)

Click to start. Before anything runs, the plugin checks the Signal layer, the checkpoint, and — when auto-correct (below) is on — the brain mask layer and the raw channel layer; a missing one stops the run with an error instead of failing hours later. `do_3D` inference is slow — it can take **hours** for a full-size fish — and runs in a background thread, so napari itself stays responsive while it works. The status line shows which pipeline stage is active (do_3D → GMM cleanup → Krendl safe-

merge → large-contact merge), and the live-output box underneath streams Cellpose's own internal progress during `do_3D` itself — normally invisible even from a terminal, since Cellpose only emits it through Python's `logging` module and doesn't configure a handler by default unless its own CLI is used. You get an email when it finishes or fails (if it runs longer than the email threshold) — see Section 10h. When complete, a `<signal layer>_cellpose_labels` layer appears, and two files are always saved to the output folder: `<signal layer>_cp.tif` (the raw prediction, before GMM/merges) and `<signal layer>_cp_krendl.tif` (the final result).

Flow cache: right after the expensive network pass, its output is cached to `<signal layer>_cp_flow_cache.npz` in the output folder. If mask formation then crashes (e.g. GPU out of memory), simply run again: a cache that matches the same checkpoint, anisotropy and volume is reused, skipping the network pass. The cache is deleted automatically once mask formation succeeds.

> If this button errors with `No module named 'cellpose'`, install it in your environment: `pip install cellpose` (already listed in `environment.yml`/`environment-mac.yml` for fresh installs — see Section 16).

Auto-correct labels via contrast sweep after segmentation (checkbox, on by default)

When a full run finishes, this chains a second, fully automatic stage onto it — no GT, no manual contrast dragging. Requires a **Brain mask layer** (field just below the checkbox — see below) and the **raw channel layer open in the viewer** (the Signal layer's name without its `_brain_only`/`_ExtRm`/`_NoBG`/`_RndFill` suffix, e.g. `<stem>_ch1` for `<stem>_ch1_brain_only_ExtRm`). Cellpose-SAM segments the `_ExtRm` layer, but every step below — the contrast sweep, skin protection and the per-cell correction — reads its signal from the **raw channel**: an `_ExtRm`/`_NoBG` image is zeroed outside the brain, which leaves skin nothing to threshold. The run refuses to start (before segmentation, not after) if the raw channel layer is not open:

1. **Calibrates the contrast threshold** that best reproduces the labels the run just produced, straight from the raw signal — the exact same self-referential engine as Calibrate Correct-Label Contrast (Section 10i), with its Cells / Slices/cell / Edge margin / Sweep steps, just run against this run's own output (including that tool's own **Bbox padding** for the sample crops). With **Auto-sweep** off (7d) the sweep is skipped and the raw channel's current contrast low is

used instead; otherwise **Reduce best_lo by** is subtracted from the swept value. Call this calibrated value `lo`. **Only microglia count** in this step: any skin label already present is treated as plain background for the sweep, so the calibrated `lo` can never depend on whether skin happened to be protected first.

2. **Protects skin as its own label** — or **reuses it, if the labels already carry the skin label (-1)**: e.g. you clicked Protect Skin as Label first, or loaded a file that was protected in an earlier session. Existing skin is kept exactly as it is, never re-seeded or re-trimmed, and the report says so. Otherwise it protects skin **before any real cell is touched** — the same engine as **Protect Skin as Label** (below), using the brain mask layer only to *seed* skin (never to clamp its result — see that section's own note on this), at the same `lo` real cells get in step 5, at the same starting Bbox padding real cells get too (**Correction Settings**, 7b). Skin becomes a real, ordinary label (ID **-1**) at this point, with the same auto-grow/until-stable/joint-adjacent-resolution behavior Protect Skin as Label itself has — it adapts to whatever the cells currently look like (still raw, not yet auto-corrected) without ever modifying them.
3. **Removes debris skin just absorbed**, right here, before any real cell's own turn — same golden-ratio floor as every other debris pass in this plugin, and the mechanism that catches any small stray blob skin picked up along the way, by size alone (skin protection does not clamp its own result against the brain mask — see below). **Gate:** before any cell is corrected the pipeline verifies that skin is really present (voxels carrying -1) and stops with a clear error if it is not — corrections without skin could bleed into skin residue, the exact failure skin protection exists to prevent.
4. **Figures out which real cells touch skin** — measured from skin's final geometry: a cell counts if, on any slice, some pixel of it is within **3 px** (in-plane; a gap of up to 3 empty pixels) of skin (see below for why) — then **resorts every real cell by Centroid Z** (cells only; skin is not counted) and partitions all of them into parallel-safe **waves** — the same partitioning either correction mode in step 5 uses, since both share the identical worst-case reach.
5. **Corrects every cell at the calibrated `lo`**, mode chosen by whether that cell touches skin:
    - **Touching skin** — corrected in **2D, slice by slice, jointly against skin** (the exact same per-slice engine trim_skin_label() itself uses, just now writing back the *cell's* own side instead of skin's) — deliberate, not incidental: skin can only ever be corrected in 2D (see Protect Skin as Label, below), so a cell meeting it has to do so on skin's own per-slice terms, rather

than running its usual 3D cross-slice walk and expecting skin's fixed-per-slice shape to somehow keep up.
    - **Not touching skin** — corrected in **3D**, batched into parallel-safe waves rather than strictly one at a time — the same auto-grow + until-stable engine as **Correct Label**'s **3D (whole cell)** mode (below), reading the shared **Correction Settings** (7b, Edit MG Labels tab): base pad 15px, auto-grow up to 20 attempts/slice growing 5px at a time, until-stable up to 50 passes/slice, same values Correct Label/Correct Adjacent Labels/Protect Skin as Label use. Read live at the moment this stage actually starts (after segmentation finishes), so changing them any time before segmentation completes still takes effect.

  Two cells whose own maximum-possible working areas can never overlap are provably unable to interact, so they're corrected concurrently (up to 75% of CPU cores), regardless of which mode either one uses. The CPU budget (75% of cores) is **shared** between cells and slices, never multiplied — a genuine conflict would otherwise multiply thread count and per-thread memory into a real risk of exhausting the machine's RAM. Cells that *can* conflict are pushed into a later wave, which still sees every earlier wave's own already-corrected state, so a cell always resolves against an already-corrected neighbor regardless of which wave it lands in. This holds even on densely-packed fish where no cell is fully isolated from every other. Each cell's own correction already resolves a genuinely adjacent label (skin, or another already-corrected cell) entirely on its own — no separate joint-watershed pass is needed after this loop.
6. **Removes debris** once more over the whole result, skin included (same golden-ratio floor as every other final-safety-net stage in this plugin — see Remove Debris), then **screens for possible non-microglia blobs** (see below) — that screen only reports, it never changes a label.
7. **Fills interior cavities, then softens every cell's contour (sanding)** — a further stage chained on afterward, gated by its own checkbox (Common Settings, above — see Soften label contours (sanding)), on by default. Runs on the full set of auto-corrected labels (skin excluded, same as every other tool that enumerates real cells only): each cell's own interior cavities (real vesicles/vacuoles the intensity threshold correctly excluded as "not signal") are filled back in first — unless that sub-step's own checkbox is off — then its contour is softened, purely geometric either way.

Why "touching skin" is measured from geometry, within 3 px: a strict 0 px adjacency test would be too strict — every joint split in this plugin deliberately clears a 1-voxel gap at the boundary, and a band of sub-threshold pixels can add more, so a cell genuinely 1–2 px from skin would be missed. The rule is: some pixel of the cell, on **any** slice, within **3 px in-plane** of skin's final territory (parameter `skin_touch_px`, default 3). Skin is corrected slice by slice, so only same-slice contact counts.

Brain mask layer (skin protection) — the Labels layer to protect skin against (same meaning as Protect Skin as Label's own field). Auto-selected from `<stem>_brain_mask` whenever it matches the selected brain_only layer — Tab 1's Open/Run already creates and loads that layer alongside every `_ExtRm`/`_NoBG`/`_RndFill` layer, so this is normally already filled in without picking it by hand. Required whenever the auto-correct checkbox is on; the run refuses to start (before the multi-hour segmentation itself, not after) if it's missing or doesn't match the volume's shape.

Possible non-microglia blobs (report only). Sometimes a segmented blob is not a microglia: a **macrophage** (usually outside the skin, or lying over it), or **skin MONAI did not remove** that Cellpose then segmented as a cell. After the final debris pass, every cell is screened, and it is listed in the report if **either** criterion reaches **40%** (parameter `non_microglia_fraction`): (a) at least 40% of the cell's **volume** lies outside the brain mask, or (b) at least 40% of its own **boundary surface** (its outer shell of voxels) is within 3 px of skin. Each listed cell shows its label ID, which criterion fired and by how much, its size, and its centroid (z, y, x) so you can find it in the viewer. **Nothing is removed or changed** — the list is for you to inspect; delete or keep each cell by hand. When nothing qualifies the report says so.

A cell that genuinely doesn't reach the calibrated threshold anywhere (rare, but possible for a very faint true positive) is skipped and left exactly as it was going into this stage — this never aborts the run or erases a cell.

On completion, this stage also updates two things you'd otherwise have no visible sign of: the **contrast limits of both the raw channel layer and the segmented `_ExtRm` layer** move to `lo` (low) and an adaptive high limit measured from this fish's own real signal — the 99th percentile of intensity among the corrected labels, floored at `lo + 50` — same feedback the standalone Calibrate Correct-Label Contrast sweep already gives, sized to show real signal's own dynamic range rather than whatever unrelated ceiling the layer came in with — and **Protect Skin as Label's own section** (its report of which real labels ended up

touching/nearby skin, its status line) is filled in exactly as if you'd clicked that button yourself, even though this stage calls the same underlying engine directly rather than through that button.

Saves the auto-corrected result to `<stem>_cp_krendl_ac.tif` and the full report as plain text to `<stem>_autocorrect_report.txt`, then — if sanding is enabled — saves the sanded result on top as `<stem>_cp_krendl_ac_snd.tif` and appends the sanding report to that same `.txt`, updating the `*_cellpose_labels` layer in place at each stage; `<stem>_cp.tif`/`<stem>_cp_krendl.tif` are unaffected — those stay the raw Krendl-pipeline output, unchanged. On-disk naming is cumulative: each stage's filename is the previous one with one more suffix appended (see 10f's naming table), so the filename alone tells you which stages actually ran. The status line reports the calibrated `lo`, the contrast limits actually applied to the signal layer (low and the adaptive high), the skin label ID, how many cells were corrected vs. skipped (in Centroid-Z order), and how much debris was removed; the log box gets one **consolidated report covering every cell** — skin protection summary, then each corrected cell's own detail (group/touching labels, which slices needed a bigger pad, which took more than one pass to settle, converged or not, debris removed) in the same format Correct Label's own 3D report uses, plus an overall tally at the end.

While it runs, and if it seems stuck. The status line names each step (calibration, skin, resorting, each wave, final debris, non-microglia check). After the computation, napari still has to apply the result to the layer on the GUI thread; the status then reads *"Computation finished — applying the corrected labels to the viewer…"* and the terminal napari was started from prints how long that took. If a run is still going after **30 minutes** (and again every 30), every thread's stack is printed in that terminal — if it looks stuck, send that output. Typical time on one real fish (101×2048×2048, 33 cells): **≈ 4 min** for this stage (calibration ≈ 35 s, waves ≈ 3 min); a crowded fish with 70–90 cells takes longer because more cells conflict and fewer run at once. Peak RAM was ≈ 35 GB there and does not grow with the number of cells.

Untick the auto-correct checkbox to skip both this stage and sanding entirely and keep the plain segmentation output; untick only the sanding checkbox to keep auto-correct but skip the softening pass.

Re-run This Cell Only

Fixes one specific label without redoing the whole fish. Crops to that label's own bounding box (+15 voxels in Z, +40 in XY), re-runs `do_3D` plus the same GMM cleanup / Krendl safe-merge / large-contact merge / final-min-size safety net used by a full run — using this section's current settings, so nudge Cellprob or Flow iterations first if that's what needs to change for this cell — then removes the old label and writes the crop result back into the full label array. Works on any labels layer of the same shape as the volume (see the tip below for how it finds the pair).

Only crop-result pieces that actually overlap the original label's own footprint are kept — a neighboring cell caught by the padding and independently re-detected is discarded, not duplicated. Every kept piece gets a new label ID after the current maximum (the old ID is not reused), so if the re-run reveals more than one real cell in that crop, all of them are kept as separate labels rather than forced back into one. The kept pieces are written only into background (and the old label's own voxels): a neighbouring label or skin inside the crop is never overwritten, and the status line reports how many voxels were left to a neighbour that way.

Label ID to re-run — type it directly, or click the cell in the viewer and use the **Use selected** button next to it.

Typical use: when a cell comes out porous/skeletonized or otherwise wrongly segmented, re-run just that one label at a stricter Cellprob (e.g. -0.3) instead of raising the setting for the whole fish — a fish-wide stricter threshold can clip real boundary pixels off cells that already segment fine, just to fix the minority that don't.

> **Tip:** the tool works on whichever labels layer you have selected (any name — e.g. one loaded with **Load Labels layer**, such as `<stem>_brain_only_ExtRm_labels`), finding its volume by shape and name. If you select the volume instead, it looks for `<image>_cellpose_labels`, then `<image>_labels`, then any single labels layer of the same shape whose name starts with the volume's. If more than one qualifies it asks you to select the one you mean instead of guessing.

Calibrate Cellprob + Krendl Merge Parameters + Min size + Large-contact (GT Sweep) — in Section 10c, Tab 6 — Sweeps & Utilities. Safe-merge's GT-min floor and the shared Min volume field (Common

Settings, 6a) are the same number — this sweep measures it internally for its own use but never writes it to the shared floor (the GT Toolkit Tuning Tool's own first step does that).

Build GT-Correction Package — in Section 10f, Tab 6 — Sweeps & Utilities.

---

7. Tab 3 — Edit MG Labels

> Before using this tab, create or load a Labels layer first (Create MG Labels, or **Load Labels layer (.tif)** in Tab 1). Until at least one Labels layer exists, the tab shows only its short overview — the selector and every tool below appear once a Labels layer is in the viewer.

Every edit/correct/modify tool for an existing Labels layer, gathered in one tab — separate from **Create MG Labels**, which only creates labels (plus the two on/off checkboxes that chain sanding/auto-correction onto that creation). All of them read their layers and label ID(s) from **one shared selector** at the top of this tab (7a) instead of each tool having its own copy — pick the Signal / Labels / Brain mask layers and Label A/B once, and every tool below uses that same choice.

7a. Layers and label(s) being edited

Sits in its own box at the top of this tab, above every tool below. Pick these once here, not separately for each tool:

- **Signal layer** — the Image layer whose contrast window drives every intensity-threshold correction (Correct Label, Correct Adjacent Labels, the contrast-calibration sweep) — **and** the layer Protect Skin/Auto-correct Labels read as real signal outside the brain. It must be the **raw, un-processed channel** (e.g. `<stem>_ch1`), never an `_ExtRm`/`_NoBG` layer — those have already been zeroed everywhere outside the brain mask by design, which leaves nothing for skin's own trim to find no matter what threshold is used. An in-brain cell's own pixel values are identical either way, so there's no downside to always using the raw channel here.
- **Labels layer** — the Labels layer every tool below reads from and writes back to. Its own **n-D editing** setting is forced to 2D the moment it becomes the one selected here — whether you pick it by hand or it auto-fills when a matching layer appears — and is re-

applied every time the layer list refreshes (e.g. right after a new Labels layer is created or loaded), so it can't drift back to napari's own 3D default in the gap before some tool button is next clicked. (This is napari's own control for whether a manual paint/fill/erase click affects only the current slice or bleeds into neighboring ones.) None of the tools on this tab go through napari's paint machinery themselves — each one replaces the layer's data wholesale instead — so this only ever protects your own manual napari-native clicks on this layer.

- **Brain mask layer** — the Labels layer Protect Skin as Label and Auto-correct Labels use to know what "outside the brain" means.
- **Label A** — the only label a single-label tool (Split Label, Correct Label, Copy Label to Adjacent Slice) acts on.
- **Label B** — used together with Label A by the two tools that join or jointly correct two labels (Join Labels, Correct Adjacent Labels).

Each of Label A/Label B has its own **Use selected** button: click a label in the viewer first, then click the button to copy that label's ID into the field, instead of typing it by hand.

Both fields take a positive cell ID or **-1** (skin). The field cannot go below -1, and every tool that reads Label A/B refuses to run on **0** (background) with an error in its status line; **Use selected** likewise refuses 0 or a negative ID other than -1.

All three layer combos auto-populate together the moment a matching `_brain_only_ExtRm`/`_NoBG`/`_RndFill` (or plain `_brain_only`) layer exists in the viewer (e.g. right after Tab 1's Run finishes) — Signal fills in with the raw channel (the open layer named exactly `<stem>`, if there is one), Brain mask with `<stem>_brain_mask`, and Labels with `<brain_only layer>_cellpose_labels` or `<brain_only layer>_labels` — or, if neither exists, the selected (else most recently added) Labels layer that isn't a `_brain_mask`. This only ever fills in a combo that's still empty — once you've picked something yourself (here, or it auto-filled on an earlier refresh), it's never silently overridden, so you can still repoint any of the three at a different fish or layer at any time.

Resort Labels, **Remove Debris**, **Auto-correct Labels**, and **Save Labels** act on the whole Labels layer — no label ID needed, Label A/B are ignored. **Protect Skin as Label**, **Remove Skin Label**, and **Hide skin label in the viewer** always act on the fixed sentinel ID **-1**, never on Label A/B — see Protect Skin as Label below.

On-screen order, top to bottom: this selector, then **Split Label** / **Join Labels** / **Copy Label to Adjacent Slice** (Label A/B tools with no settings of their own), then **Resort Labels** / **Remove Debris** (whole-layer, no settings), then **Correction Settings** (7b) / **Sanding** (7c) / **Contrast low (best_lo) calibration** (7d), then **Correct Label** / **Correct Adjacent Labels** / **Protect Skin as Label** / **Auto-correct Labels** (all four read from the three settings groups just above them), then **Save Labels**.

7b. Correction Settings (Bbox / Auto-grow / Stability)

One shared set of controls for **Correct Label**, **Correct Adjacent Labels**, **Protect Skin as Label**, and **Auto-correct Labels** — standalone, or chained onto Cellpose-SAM Segmentation's own auto-correct stage (6c). Changing a value here changes it for all four at once, so a manual correction and an unattended pipeline run always agree.

- **Bbox padding (px)** — default **15**. Each correction only looks within the target label's own bounding box (+ this padding), not the whole image. Used as the *starting* pad for every one of the four tools above, including Protect Skin as Label — skin's own bounding box already spans nearly the whole frame on most slices regardless of the padding value, so this shared default costs it nothing.
- **Auto-grow until signal clears the border** — checkbox, **default on**. If a corrected result still touches the edge of its own padded working area, the padding is automatically enlarged and retried, repeating until the result stops touching the edge or **Max growth iterations** is reached. Reaching the true edge of the volume itself is never flagged.
- **Growth step (px)** — default **5**. How much the padding grows on each retry.
- **Max growth iterations** — default **20** (range 1–50). Hard cap on retries per slice (3D modes) or per attempt (2D). If still touching the border after the cap, the tool stops and reports exactly which slice(s)/attempt are still touching, rather than continuing to grow unbounded.
- **Keep re-running until the shape stabilizes** — checkbox, **default on**, independent of Auto-grow (usable with it, without it, or instead of it). Re-runs the correction, feeding each pass's result back in as the next pass's starting shape, until the shape (or, for Correct Adjacent Labels/Auto-correct Labels/Protect Skin as Label, every label in the group) stops changing, or **Max stability passes** is reached. Correct Label's **2D** mode does not use this setting (only its 3D mode does).

- **Max stability passes** — default **50** (range 2–200). Hard per-slice/per-attempt cap so nothing can loop forever; if still changing after the cap, the tool stops and reports which slice(s) never settled.

These are read live, at the moment each tool's own computation actually starts — not frozen at the moment you click a Run button. For Correct Label/Correct Adjacent Labels/Protect Skin as Label this is immediate, since clicking IS the computation starting. For **Auto-correct Labels chained onto Cellpose-SAM Segmentation**, the auto-correct stage only begins after the (potentially hours-long) segmentation pass finishes — so if you start segmentation before deciding on these values, changing them any time before segmentation completes still takes effect, since they're read fresh right when the auto-correct stage itself begins, not when you clicked "Run Cellpose-SAM Segmentation." (Segmentation's own settings — Cellprob, Max gap, Min contact, Large contact, Min hole size, Min size, Flow iterations — are the opposite: those ARE captured at the moment you click Run, and changing them mid-run has no effect on that already-in-flight pass.)

7c. Sanding

The **Sigma XY** / **Sigma Z** sliders that control sanding strength (see Soften label contours (sanding), Create MG Labels' own Common Settings) live here, not there. Enable/disable sanding itself with that checkbox in Create MG Labels; these two sliders are the single shared source every sanding call reads — Correct Label, Correct Adjacent Labels, Auto-correct Labels, and the Cellpose-SAM auto-correct pipeline's own sanding stage all read these same two values, regardless of which tab is visible when they run.

Sanding sigma XY (vox) / **Sanding sigma Z (vox)** — both default **0.7**. Anisotropic 3D Gaussian blur strength (separate XY/Z) applied to a label's own binary mask, re-thresholded at 0.5, to round off blocky/jagged voxel edges without meaningfully changing the cell's real shape or volume. Deliberately much smaller than the Pixel Classifier's own Smooth σ XY/Z (Create MG Labels, default 2.60/1.00) — that pair decides whether raw blobs merge into one 3D object before any labels exist; sanding only polishes an already-correct label's own edges afterward, and is foreign-protected so it structurally cannot merge cells.

Fill interior cavities (vesicles) before sanding — checkbox, Create MG Labels' own Common Settings, **default on**. Runs immediately before the blur above, at every one of the same call sites: a real

intracellular vesicle/vacuole (a lipid-processing organelle) shows up dark in the fluorescence channel, so a plain intensity threshold correctly excludes it from the label as "not signal" even though it's genuine cell material — leaving a literal enclosed cavity in the middle of the cell that would otherwise look like a hole no matter which way you slice the volume (Z, Y, or X). This fills any such cavity back in, fully in 3D — not a convex hull, and not a per-slice fill: a real cell's own concave lobes and branches are left exactly as concave as they are (only background voxels *fully enclosed* by the label are ever touched), and a genuine 3D-enclosed cavity is found regardless of which axis it's later viewed from, unlike filling one axis's slices alone (which can miss a cavity that happens to look like two separate blobs from that particular direction). A real open branch, or a different label genuinely sitting inside what topologically looks like a cavity, is left completely untouched — same foreign-label protection as every other tool here. **Cells only — never applied to the skin sentinel label (-1)**: skin's own apparent "cavities" are real cells sitting inside the brain, not vesicles, and filling them would flood the entire brain interior as skin.

7d. Contrast low (best_lo) calibration

Shared by every tool below that needs a contrast-low threshold to decide what counts as real signal — **Protect Skin as Label** and **Auto-correct Labels** today, and any future tool that needs the same calibration.

Auto-sweep for best contrast low (best_lo) — checkbox, **default on**. When checked, sweeps for the best_lo contrast-low value against the active labels, then subtracts **Reduce best_lo by** from it — e.g. best_lo=107 with reduction=2 corrects at 105, capturing slightly more signal. When unchecked, the reduction slider is ignored and the tool instead uses the Signal layer's **own current contrast LOW limit** exactly as set in napari (no sweep, no reduction).

Reduce best_lo by — slider, **range 0-6, default 2**.

The sweep itself — how many cells, how many slices/cell, the edge margin, the number of sweep steps — is the exact same sweep run by **Calibrate Correct-Label Contrast** (Tab 6) — the canonical, single-source-of-truth sweep every autosweep-driven tool here correlates to. Tune those sample-selection parameters there, not per-tool here — including its **Bbox padding (px)**, the crop padding around each

sample, which every one of these sweeps uses (Correction Settings' Bbox padding, 7b, is the per-cell correction padding, not the sweep's). The sweep runs on the **Signal layer**.

Whenever a sweep actually runs and returns a value, the signal layer's contrast limits are set to `(best_lo, adaptive_high)` — not just the low limit. The correction itself only ever reads the low limit (`candidate = image >= lo`); the high limit is purely a napari display setting with no effect on any correction result. But real cell signal sits in a narrow band just above background, so leaving the high limit at whatever unrelated value the layer happened to come in with (e.g. a file's own full sensor range) squeezes that whole real-signal band into a near-black sliver — the display looks flat/empty even though the correction result underneath it is correct. `adaptive_high` is the 99th percentile of raw intensity among whatever the active labels currently call real signal (any nonzero label — a cell, or skin), floored at `best_lo + 50` — measured fresh from this fish's own data each time, so the displayed window is always sized to what's actually there to see, regardless of how bright or faint this particular fish's signal happens to be.

Sort by / Reverse order / Resort Labels

After creating (or loading) labels, you can renumber them by a criterion of your choice.

Sort by dropdown:

Option	Meaning	Default order
Size	Number of voxels	Largest = label 1
Centroid Z	Z coordinate of centre	Smallest Z = label 1
Centroid Y	Y coordinate of centre	Smallest Y = label 1
Centroid X	X coordinate of centre	Smallest X = label 1
Complexity	Skeleton branch count	Most branched = label 1

Only positive labels are renumbered; skin (-1) is left unchanged.

Reverse order checkbox — inverts the ordering (e.g. smallest = label 1 for Size).

Click **Resort Labels** to apply. The Labels layer selected in the shared selector above (7a) is renumbered 1…N in the chosen order, in place. This is useful for consistent numbering across samples or for matching cells to a reference atlas.

Remove Debris

Manual edits in napari — deleting a whole label because it turned out to be misclassified skin, splitting one, painting part of one away — can leave small disconnected fragments behind that never went through Create Labels' own volume filter (which only ran once, before the edit). Click **Remove Debris** and the Labels layer selected above (7a), exactly as it currently stands, is swept for anything smaller than `Final min-size fraction × Min volume` (Common Settings, 6a) — the same golden-ratio-relaxed cutoff Create Labels' own filter and Cellpose-SAM's final safety-net stage already use — and every one found is zeroed out. **Protect Skin as Label's own skin label is swept too**, whenever one is present in the layer (e.g. a small isolated bright-noise speck outside the brain mask, or a leftover fragment from a manual Split Label on skin) — the same size threshold applies to it as to every real cell.

Evaluated per spatially-connected fragment, not per label ID. Erasing most of a wrongly-segmented label by hand (painting over it) commonly leaves several small, disconnected leftover pieces still carrying that *same* original ID — and their combined voxel count can look large enough to survive even though every individual piece is genuine debris on its own. This tool checks each disconnected piece's own size, not the sum across everything still tagged with that ID, so those leftovers are correctly caught.

Works on labels from either route. Surviving labels' IDs are left exactly as they were — this only removes, it never renumbers (use **Resort Labels** above afterward if you also want a clean 1…N sequence). A label ID with one real surviving piece and one small disconnected leftover keeps its ID on the real piece; only the leftover is zeroed.

Split Label

Splits a single merged label (a blob where two or more cells are stuck together) into separate parts using a watershed algorithm.

In 3D mode the watershed runs on the mask's own distance transform, so it finds the **thinnest neck** connecting two large volumes and cuts there — it does not use a simple distance threshold or Euclidean splitting.

Splits **Label A**, from the shared selector at the top of this tab (7a) — type it directly there, or click the blob in napari first and use that section's own **Use selected** button next to Label A.

Split mode

3D (whole label) (default) — splits the entire 3D blob, exactly as described above.

2D (current slice only) — restricts the whole operation to the single Z-slice you're currently viewing: crops to that label's footprint on just that slice and runs the same watershed pipeline in 2D — but on the **Signal layer's intensity** (shared selector, required in this mode) instead of the mask's shape, so seeds sit on brightness peaks and the cut follows the dimmest ridge between them — and splices the result back into only that slice. Every other slice of the label is left completely untouched, and the new piece exists only on that slice — it doesn't extend into neighboring slices the way a 3D split's parts would.

Use 2D mode when two things only touch on **one** cross-section rather than forming a genuine 3D neck — e.g. real signal happening to graze an unrelated skin-residue fragment right at that particular slice, with no real connection above or below it. In that case a 3D split either can't find a cut there at all (there's no true 3D saddle to watershed on) or ends up cutting somewhere unrelated on a different slice instead. Navigate to the problem slice in napari before clicking Split Label.

Split into N parts

Range: 2 to 10 — **Default: 2**

How many separate pieces the blob should be divided into. The algorithm searches for the N largest sub-volumes (separated at their thinnest necks) and cuts between them.

> If the blob genuinely has only one major volume (no neck), splitting may fail or produce uneven results. Increase Smooth σ or use a lower Min distance if that happens.

Smooth σ (Split)

Range: 0.0 to 3.0 — **Default: 1.0**

Gaussian smoothing applied to the split surface (the distance transform in 3D, the signal intensity in 2D) before searching for peaks. Higher values smooth out that surface, making the algorithm more robust to surface noise but less sensitive to subtle necks.

- **0.5–1.0:** Suitable for most cases.
- **1.5–2.0:** Use if the split point jumps around — smoother distance field = more stable result.
- **0.0:** No smoothing — very sensitive to surface texture.

Min distance

Range: 1 to 30 voxels — **Default: 5**

Minimum voxel distance required between accepted seed peaks. If two candidate peaks are closer than this, only the stronger one is kept.

- **Too high:** The two centres of a closely-packed double-blob may be rejected as "too close" → fewer than N peaks found → error.
- **Too low:** Surface noise peaks may be accepted as separate centres → wrong split point.
- **5 voxels** works well for microglia-sized cells.

Split Label (button)

Click to run. The original blob is replaced in-place:

- The original label number is kept for the **first** part (the part whose seed has the highest peak — the thickest chunk in 3D, the brightest in 2D).
- New label numbers (`max_existing + 1`, `max_existing + 2`, …) are assigned to the remaining parts.

The cut is **interface-only**: exactly the voxels at the boundary between parts are removed, creating a 1-voxel gap. The outer surface of each part is not touched — thin protrusions are preserved.

If the algorithm cannot find N distinct sub-volumes, an error message is shown. Try reducing Smooth σ or Min distance.

Join Labels

The inverse of Split Label: merges **Label B** into **Label A** (both from the shared selector, 7a) when one cell was wrongly cut into two pieces (e.g. a thin process fooled the segmenter into treating it as a neck). A single, fast, whole-volume operation — no bounding-box crop or GPU path needed, since nothing here depends on shape or geometry.

Click each fragment in the viewer and use the shared selector's own **Use selected** button next to Label A/Label B to fill them in, or type the label numbers directly.

Click **Join Labels**. Every voxel currently carrying Label B becomes Label A instead; Label A's ID survives, Label B's ID disappears entirely. Works on labels from either route.

Correct Label

Regenerates a label's shape from the raw signal layer's own live contrast display — useful when a particular contrast window traces a cell's true silhouette more accurately than the existing label does. Works in **2D** (one slice) or **3D** (the whole cell), via the **Correction mode** dropdown.

How it works (both modes): dial the signal Image layer's contrast limits in napari until the cell's outline looks right by eye — a narrow window (e.g. displaying only values from ~100 up) usually works best, since it saturates the display into a clean silhouette rather than showing a soft gradient. **Calibrate Correct-Label Contrast** (Tab 6) can find this value automatically instead of by eye — see below. Then, without changing anything else, run this tool: it reads that same contrast window straight off the shared **Signal layer** (7a) and uses everything **at or above** the lower limit as signal — the upper limit is just napari's own display-saturation ceiling, not a boundary on what still counts as real signal, so it isn't used to exclude anything.

Fills the whole padded box, not just what already connects to the label. Every pixel at or above the threshold within the bounding box (+ padding) becomes part of the corrected label, whether or not it's touching (even diagonally) the label's existing shape — real cell boundaries are often irregular or branchy, and signal that's visibly part of the cell to the eye shouldn't be discarded just because it doesn't already connect at this exact threshold. The only pixels ever excluded are ones already claimed by a *different* label (never touched, absorbed,

or grown into) and anything below the threshold. Any small leftover fragment this produces that isn't really part of the cell is exactly what **Remove Debris** (above) is for — in 3D mode, the tool also runs that cleanup automatically at the end (see below).

Signal layer and **Label A** — both read from the shared selector at the top of this tab (7a), not a field of their own here. Label A takes any positive label ID, or **-1** for the Protect Skin as Label sentinel itself — useful for re-trimming skin's own territory by hand after it's been created. Setting Label A to **-1** automatically disables the 3D option below (see the note there).

Bbox padding, **Auto-grow**, **Growth step**, **Max growth iterations**, **Keep re-running until the shape stabilizes**, and **Max stability passes** are all set in the shared **Correction Settings** group (7b) above — not fields of Correct Label's own, and shared with Correct Adjacent Labels, Protect Skin as Label, and Auto-correct Labels, so all four always agree. Each corrected slice only looks within this label's own bounding box (+ padding) on that slice, not the whole image. See below for how Auto-grow and Keep-until-stable specifically behave in this tool's 2D/3D modes.

Correction mode

> **3D is disabled for the skin label (-1).** A 3D correction walks the label's own current territory slice by slice — for skin, that's the WHOLE outside-brain region of the fish, a genuinely large amount of computation for no real benefit over Protect Skin as Label's own dedicated, far cheaper trim (which already does exactly this job, correctly, in one purpose-built pass). Setting **Label A** to -1 automatically disables the 3D option in the dropdown (and switches back to 2D if it was selected) — 2D correction of skin, one slice at a time, is still fully supported.

- **2D (current slice only)** — corrects only the slice you're currently viewing.
- **3D (whole cell)** — corrects the entire cell. **Not a bounding box at any point.** This loops over 2D areas, one Z slice at a time, each with its OWN local neighborhood — the label's own CURRENT footprint on that one slice, padded — reusing the same already-correct 2D tools directly: no other label present in that local area → plain single-label 2D correction (exactly Correct Label's own 2D mode); another label present → the same joint watershed split Correct Adjacent Labels uses, scoped to this label alone, so the local boundary against a genuinely adjacent cell gets properly re-derived

right there too, not just excluded as an immovable wall. The label's own original Z range is corrected this way outright (it already exists on every one of those slices), **walked mid-outward, not in plain Z order**: the walk starts at the range's own upper-middle slice and settles outward in two halves — middle down to the bottom, then middle+1 up to the top. Before each slice's own correction runs, its watershed seed is first topped up with whichever of its immediate neighbors (one slice up, one slice down) fall within the label's own original range: a neighbor the walk has already reached contributes its own just-corrected shape; one not yet reached still contributes its own original, pre-walk shape — real signal, useful context either way, refined by its own correction once the walk gets there. This union only ever ADDS territory into empty background or skin (never onto a different real label's own pixels), purely as a bigger starting marker for that slice's own watershed — the final shape is still entirely re-derived from the real intensity threshold there, so a neighbor's shape can never force-keep a pixel that isn't also real signal on THIS slice. The practical effect: a good correction on one slice actively helps its neighbor's own correction, and a single spuriously undersized original slice — ordinary Cellpose-SAM prediction noise, not necessarily a real taper — gets topped up by its neighbor's own shape as an integrated part of the walk. A genuinely tapering cell still comes out correctly smaller slice by slice regardless — only a truly undersized *original* slice gets rescued, never a real, deliberately narrower one, since the final shape always comes from real signal, never from the seed alone. Growth beyond the original range works the same way but one-directional: copying the current slice's own just-corrected shape onto the next Z first (foreign-protected, same as Copy Label to Adjacent Slice), then running the same local correction there — the moment that finds nothing to support the copied position, the copy is undone and growth in that direction stops, the true edge found. This is also the tool's own answer to "is there real signal at the very end of the cell the segmentation missed" — it keeps extending outward, one slice at a time, for as long as real signal keeps supporting the new position. A hard cap — as many slices beyond the original range as **Bbox padding** (the starting value; auto-grow never raises it) — still bounds how far this can ever extend regardless, so a real, different, genuinely-touching structure can't pull growth along indefinitely just because it happens to overlap slice-to-slice.

Each slice's working area is always locally-anchored — re-derived fresh from wherever the label actually currently sits, every single slice — rather than one fixed window sized from the label's whole 3D extent: a padded working area's whole-box fill has no connectivity requirement (see the note under Correct Label), so a window any bigger than a given slice's real footprint there risks claiming whatever OTHER real, not-yet-labeled signal — skin residue, a different unlabeled structure — happens to also fall inside it. A per-slice area can only ever reach as far as the padding from the label's own real, already-verified position one slice ago.

Because growth only ever adds a slice after confirming real continuation, and the original range is always corrected in place, there's no separate "trim leftover pixels beyond where growth stopped" step needed either.

A debris-cleanup pass then runs automatically (the same golden-ratio-relaxed floor Cellpose-SAM's own final safety net uses) to remove any small disconnected leftover — scoped to **only this label**, so it can never affect any other cell in the fish.

A report appears below the button listing every slice the correction touched, how many debris pixels were removed, and — for each of those slices — any *other* label whose pixels either directly **touch** the correction or merely sit **nearby** (inside the same padded working region, without necessarily touching). This is a visibility check, not a safety gate: a neighboring label's own pixels are never absorbed or overwritten either way (structurally impossible — the correction can only ever claim pixels that were background or already this label) — the report just flags close calls worth a manual look.

Click **Correct Label**. A neighboring label's own pixels are never touched, absorbed, or grown into — even if they sit inside the same padded box and share the exact same intensity range as the label being corrected. If the chosen contrast window leaves no signal at all anywhere in the padded box, the tool refuses to apply rather than silently emptying the label — adjust the window and try again.

If Soften label contours (sanding) (Common Settings) is checked — the default — the corrected label has its interior cavities filled (if that checkbox is on too) and is then sanded immediately afterward, in either mode: same foreign-label protection, purely geometric, rounds off blocky edges left by the intensity-threshold correction. The status line reports whether it was applied.

Auto-grow until signal clears the border

Default: on (Correction Settings, 7b). A fixed padding can cut off real signal that genuinely extends further than expected — the correction fills the padded area as far as it can, but has no way to tell "the cell truly ends here" from "the area ran out of room." This checkbox catches that: if the corrected result touches the edge of its own padded working area, it automatically retries with a bigger one, repeating until the result stops touching the edge or the iteration limit is hit. Reaching the true edge of the volume is never flagged — there's nothing more to grow into there anyway.

- **Growth step (px)** — how much the padding grows each retry (default 5).
- **Max growth iterations** — hard cap on retries (default 20), so this can never grow unbounded. If it's still touching the border after the cap, the tool stops and tells you rather than continuing to grow — a real signal that the cell may extend even further, worth a manual look. **In 3D mode, the report names exactly which slice(s) are still touching the edge** — go correct those individually (a bigger pad on just that slice via Correct Label's own 2D mode, or by hand) instead of guessing which part of the cell needs more room. The same slice list is shown even without auto-grow, whenever a plain 3D correction's own report says `touched_border`.

In 2D mode, if the growing box starts overlapping a different label, that label is automatically folded into the correction instead of being encroached on — the tool switches to the same joint, foreign-protected correction Correct Adjacent Labels uses (generalized to any number of labels, not just two), so growth can never silently eat into a neighbor's territory. The status line reports if the group grew this way. There's only one shared padding value here, since 2D corrects a single slice — nothing to grow "per slice."

In 3D mode, growth is PER SLICE, not a whole-cell retry. Since Correct Label's own 3D mode already loops over 2D local areas rather than one bounding box, each slice's own padding grows independently, only for the slice(s) that actually need it — a cell with 40 well-behaved slices and one long branch reaching further on a single slice only ever regrows THAT one slice, at whatever bigger pad it needs. Every other slice keeps its own already-correct result and its own (smaller) pad completely untouched. This is both cheaper (no reason to redo 39 already-fine slices just because one needed more room) and converges far more easily, since one slice needing extra room never means the whole cell has to be reprocessed at that bigger pad before it can be

judged converged. The report lists exactly which slice(s) ended up needing a bigger pad and what they grew to, plus every OTHER label the final correction ended up near, for visibility. A final debris-cleanup pass (same golden-ratio floor as everywhere else) runs once at the end.

Sanding (if enabled) runs on every label the final correction reported as nearby, not just the one you originally targeted.

Keep re-running until the shape stabilizes (3D only)

Default: on (Correction Settings, 7b). **Independent of auto-grow above** — usable with it, without it, or instead of it. Every 3D correction doesn't just reshape the label you're correcting: wherever it finds a genuinely adjacent label (another real cell, or a Protect Skin as Label neighbor), the joint watershed split it runs there also updates THAT label's own boundary — seeded from each label's CURRENT shape at the time. Feeding one pass's result back in as the next pass's starting point lets the boundary settle a little closer to a mutually-consistent position each time, since each watershed reseeds from a slightly different (just-updated) marker than the pass before — repeated enough times, this converges toward a genuine fixed point where re-running produces the identical result again, rather than stopping after just the first (possibly still-settling) boundary placement.

This runs PER SLICE, exactly like auto-grow does — not a whole-cell retry. Each slice keeps re-correcting itself, feeding its own result back in as its own next attempt's starting point, until THAT slice's footprint matches what the immediately preceding attempt on that same slice produced — or **Max stability passes** (default 50) is reached, whichever comes first. A slice with no adjacent label at all still needs a minimum of 2 attempts to *confirm* nothing is changing (there's no baseline to compare against on the very first attempt), but stays cheap; only a slice genuinely still settling against a real neighbor keeps going beyond that. Every other slice is left completely alone once it's confirmed stable — a cell with 40 slices and only 2-3 still settling near skin never pays for redoing the other 37. The report lists exactly which slice(s) needed more than one pass and how many, plus whether any slice hit the cap still changing.

Max stability passes — hard per-slice cap so no single slice can loop forever; if it's still changing after the cap, the tool stops and tells you which slice(s), rather than continuing indefinitely.

Copy Label to Adjacent Slice

Copies one label's 2D shape from the currently-viewed slice onto the next or previous slice — useful for patching a slice where a cell's cross-section is missing or broken, by reusing a clean neighboring slice's shape instead of hand-painting it.

Copies **Label A** (shared selector, 7a) — accepts any positive label ID or **-1** for skin.

Copy to — choose **Next slice (Z+1)** or **Previous slice (Z-1)**.

Click **Copy Label to Adjacent Slice**. The label's own old shape on the target slice is cleared first, then the copied shape is painted in — running this again on the same pair of slices replaces the previous copy cleanly rather than leaving a stale double outline. As with Correct Label, a neighboring label's pixels on the target slice are never touched: any part of the copied shape that would land on a different label is silently dropped, and the status message reports how many pixels were skipped that way so a partial-overlap copy is never mistaken for an exact one.

Correct Adjacent Labels

For two labels that end up touching or merged on **one slice** — two real cells, or a cell and a skin-residue fragment — e.g. right after Copy Label to Adjacent Slice pastes a shape that now touches a neighbor there. **2D only, current slice.**

Why not just run Correct Label twice: at a shared contrast threshold, both labels' regenerated regions can fuse into one connected blob exactly where they touch, and each single-label correction's own foreign-label guard would draw the boundary from the *other* label's stale original shape — not the real signal boundary between them, which is usually different once both are corrected.

How the cut is placed: this tool regenerates **both labels together** from the intensity threshold — every pixel in the working rectangle at or above the threshold, not just what already connects to either label (see the same note under Correct Label) — then splits that combined region with a watershed **seeded by each label's own existing footprint** — not by auto-detecting the two strongest intensity peaks anywhere in the combined region the way Split Label's 2D mode does. That distinction matters: a combined region can have more than one

real dip (e.g. genuine internal texture inside one of the two cells, in addition to the real seam between them), and blind peak-finding can lock onto the wrong one entirely. Seeding directly from each label's own current region means the correction floods outward from each cell's own known territory, so the two fronts meet near the *actual* boundary's neighborhood — following the real signal there, not just re-drawing the old shape, but also not wandering off to some unrelated dip elsewhere. Any signal island not directly reachable from either label at all still gets assigned to whichever one is nearer (so nothing in the box is silently dropped), with the same clean 1-pixel gap left at the dividing line either way.

The working rectangle is Label A's own bbox + padding — not the union with Label B. Label A is the focus: this is the correction "for" A, using B only to resolve the local boundary right where the two actually meet. If B is larger, or its own extent reaches far beyond A's own neighborhood, none of that distant territory is ever read or touched — only whatever portion of B happens to fall inside A's own (padded) rectangle takes part, exactly like any other foreign label would if it happened to be there. If B doesn't reach into A's rectangle at all, the tool refuses with a clear message rather than doing nothing useful (increase padding, or correct A alone with plain Correct Label instead).

Signal layer, **Label A**, and **Label B** all read from the shared selector at the top of this tab (7a) — type them there, or click each label in the viewer and use that section's own **Use selected** buttons. Order matters: A is the label the working rectangle is sized from. Either field takes any positive label ID or **-1** for the Protect Skin as Label sentinel — e.g. resolving a real cell's boundary against skin the same joint way as against another cell. Label A and Label B must differ.

Bbox padding, **Auto-grow**, **Growth step**, **Max growth iterations**, **Keep re-running until the boundary stabilizes**, and **Max stability passes** are all set in the shared **Correction Settings** group (7b) above, around Label A's own existing footprint on the current slice — not fields of Correct Adjacent Labels' own, shared with Correct Label, Protect Skin as Label, and Auto-correct Labels.

Click **Correct Adjacent Labels**. A third, unrelated label is never touched, absorbed, or grown into — same protection as every other Correct Label tool. If the threshold leaves one label's own existing footprint with nothing to anchor to at all, the tool refuses rather than silently erasing that label — adjust the contrast window and try again. The status message reports each label's final pixel count (within the

working rectangle — Label B's own territory outside it, if any, is unaffected) and how many pixels (if any) were lost right at the cut boundary.

If Soften label contours (sanding) (Common Settings) is checked — the default — **both** Label A and Label B (plus any label auto-grow folded in) have their cavities filled (if that checkbox is on) and are sanded immediately afterward, each independently, same foreign-label protection. The status message reports how many of the two were actually softened.

Auto-grow until signal clears the border — default **on** (Correction Settings, 7b), same behavior as Correct Label's own auto-grow, just seeded with both Label A and Label B from the start instead of one label — the working rectangle stays scoped to Label A throughout, exactly as above, even as growth retries with a bigger pad or folds in a third label. If growth reveals a third label, it's folded into the joint correction too (only via whatever portion of it falls inside A's own rectangle). Same **Growth step (px)** / **Max growth iterations** fields, same non-convergence reporting.

Convergence is judged on Label A alone. Label B is the adjacent label, usually larger or reaching further than A's own rectangle — it will typically keep touching the edge of that rectangle no matter how much the pad grows, since it was never meant to be grown to its own true extent here. Only Label A's own border status decides whether auto-grow stops; Label B touching the edge is expected and doesn't trigger another retry.

Keep re-running until the boundary stabilizes — default **on** (Correction Settings, 7b), **independent of auto-grow above** (usable with it, without it, or instead of it), the same mechanism Correct Label's own 3D-mode stability switch uses, applied here to the single-slice two-label case. The joint watershed split above doesn't just shape Label A — it also redraws Label B's (and any folded-in third label's) boundary at the same time, seeded from each label's CURRENT shape. Feeding one pass's result back in as the next pass's starting point lets the shared boundary settle a little closer to a mutually-consistent position each time, instead of stopping after just the first (possibly still-settling) placement. Re-runs until **every** label in the group has a shape identical to the previous pass, or **Max stability passes** (default 50) is hit — a fresh group still needs a minimum of 2 attempts to *confirm* nothing is changing, exactly like Correct Label's own 3D stability switch.

Protect Skin as Label

Turns everything OUTSIDE the brain mask into a real, ordinary label — **always ID -1**, never the shared Label A/B fields — so it's structurally protected by the exact same foreign-label exclusion every Correct Label / Correct Adjacent Labels / auto-grow / Cellpose-SAM auto-correct call already has, instead of relying on those tools' own intensity threshold happening to stay below skin's own signal everywhere. A real cell's own correction bleeding into skin residue — a genuine, recurring failure mode even with a correctly-scoped local working area, since skin can be just as bright as real signal — is exactly what this rules out structurally: skin becomes just another label nothing else is ever allowed to grow into, absorb, or jointly split against.

How it works: one click does three steps. **First, it gets `lo`**, from the shared **Contrast low (best_lo) calibration** section above (7d): with Auto-sweep checked (the default), it runs the same contrast sweep step 1 of the auto-correct pipeline uses, on the **Signal layer**, against the labels you loaded (about 1 minute on a full-size fish; **cells only** — skin is ignored; sample crops padded by Calibrate Correct-Label Contrast's own Bbox padding), subtracts the **Reduce best_lo by** amount from the result, and uses that as `best_lo`. With Auto-sweep unchecked, it skips the sweep entirely and uses the signal layer's own current low contrast limit directly — no reduction applied. It does **not** silently fall back to the signal layer's current low limit while Auto-sweep is checked — that would be whatever an IMS load or an earlier session left behind (e.g. 90) and has nothing to do with these labels. When protection finishes, the signal layer's **low** limit is set to the value actually used and its **high** limit is set adaptively from this fish's own real signal (the 99th percentile of intensity among the layer's real labels, floored at low + 50), so the display shows real signal's own dynamic range instead of whatever unrelated ceiling the layer came in with; the status line reports both (*"lo=101 (swept 103, reduced by 2, mean IoU 0.776 on 50 samples). Contrast set to [101, 194]."* when swept, or *"lo=90 (autosweep is off — using the signal layer's current low limit)"*, with no contrast update, when not). If a sweep is attempted but genuinely impossible (for example the labels have no usable interior cells) it falls back to the layer's current low limit and says so — *"NOT calibrated (…) — fell back to the signal layer's current low limit"* — never silently. **Second**, every background voxel outside the brain mask is bulk-filled with the skin label (never overwriting a real cell's own territory, never touching anything inside the brain mask). Then that label is trimmed down from "the whole outside-brain region" to just the real, signal-supported skin territory (everything at or above the calibrated `lo`) — **each Z slice**

independently, by calling Correct Label's own single-slice (2D) engine directly, once per slice — the exact same function the button itself calls when you select the skin label by hand and click Correct Label, byte-for-byte identical slice by slice. Not the 3D whole-cell walk: skin has no need for that walk's own cross-slice machinery (built to keep ONE coherent object consistent slice to slice as it's re-derived) — skin's candidate territory on any given slice is already fully known in advance from the brain mask, and it spans nearly the entire Z range of a typical fish, so walking that machinery over every slice would be pure wasted computation for no benefit (the same reason 3D correction of an *existing* skin label is blocked in Correct Label's own dropdown, further down).

Each slice does still get its own **auto-grow** + **until-stable** — the same per-slice mechanism Correct Label's own 3D mode uses, reading the exact same **Correction Settings** (7b) Correct Label/Correct Adjacent Labels/Auto-correct Labels do: a slice whose result touches the edge of its own working window retries at a bigger pad, and once that settles, the slice keeps re-running (reseeded from its own previous result) until its footprint stops changing. Both are on by default (growth 5px up to 20 attempts/slice, until-stable up to 50 passes/slice) and are what actually let skin reach real signal well beyond its own small starting footprint — a single fixed-pad attempt alone recovers only a fraction of what's really there.

Because every slice's correction is fully independent (no cross-slice dependency at all, unlike a real cell's own 3D walk), slices are corrected **in parallel**, on up to 75% of the machine's CPU cores — confirmed via direct benchmark to cut a real fish's run time by roughly 3.5x (about a third of the time it would otherwise take), without starving the rest of napari (or another concurrent job) of every core.

Boundary against a touching real cell is jointly RESOLVED, not just excluded. Wherever a real label is present in skin's own working area, the shared boundary is re-derived via the same marker-seeded watershed Correct Adjacent Labels uses — this is what lets skin correctly grow into signal a cell's own earlier (or not-yet-run) correction over-claimed, instead of being permanently walled off wherever that cell's current boundary happens to sit. Only skin's *own* resulting territory is ever painted back, though — the touching cell's own label is never modified by this tool, structurally guaranteed by the same watershed algorithm computing the split (a cell's own existing pixels are always used as *its own* marker/seed, and a marker pixel can never be reassigned to a different label's result). This is deliberately one-sided: at the point skin is protected, a Cellpose-SAM cell hasn't

been through its own correction yet, so it isn't skin's place to reshape it — once that cell's own turn comes up (Cellpose-SAM auto-correct's per-cell stage, or Correct Label run on it directly), *it* is free to adjust both itself and whatever it touches, skin included.

Does NOT clamp its result against the brain mask. Skin's own working area, once bulk-seeded, already touches the image's own edges on essentially every slice, so the generic correction engine's "real signal, not already claimed by a different label" rule has no way, on its own, to tell real skin residue apart from an unrelated unclaimed signal blob sitting *inside* the brain — instead, a **Remove Debris** pass runs automatically right after protection (this button, and both auto-correct entry points) and sweeps up whatever small stray blob skin absorbed by size alone (`Final min-size fraction × Min volume`), the same way debris in any other label is caught, without forbidding a large, legitimate inward correction along skin's real inner boundary near the brain edge.

Automatically repairs a slice that comes back with no skin at all. On rare occasions a single slice's own per-slice correction can come back empty even though the brain mask leaves real outside-brain territory there to protect. Every such slice is detected automatically and given up to 2 independent, from-scratch attempts — re-seeded fresh (everywhere outside the brain mask that isn't already a real cell's own territory) and re-corrected — before being accepted as a genuine gap. If any slice needed this, the status line and the report box below both say so (*"N slice(s) auto-repaired"*); if a slice still has no skin after every repair attempt, a clear warning names it — that means there's genuinely no signal-supported skin territory there at this `lo`, or every bit of outside-brain territory on that slice is already claimed by a real cell, worth a manual look rather than something silently swept under the rug. The exact same repair check also runs on any real cell corrected in 2D against a touching skin label (see Auto-correct Labels below) — a cell's own version re-seeds from its own original footprint instead of the brain mask.

The status line reports how many debris fragments of stray skin that pass removed, when any were.

It never protects twice. If the selected labels layer already carries the skin label (any voxel equal to -1 — however it got there: this button, the auto-correct pipeline, or a labels file loaded from disk), clicking again does nothing and says so: *"Skin is ALREADY protected on '' (label -1, N*

voxels) — nothing done. To protect it again from scratch, click 'Remove Skin Label' first." This is judged from the layer's own contents, not from an earlier click.

Typical time on one real fish (101×2048×2048): about **7 minutes**, almost all of it the per-slice skin trim; the contrast sweep is about 50 s and the debris pass about 9 s.

Signal layer, **Labels layer**, and **Brain mask layer** — all read from the shared selector at the top of this tab (7a), not fields of their own here. If the Signal layer is an `_ExtRm`/`_NoBG`/`_RndFill` layer, the skin trim (not the contrast sweep) automatically switches to that layer's raw channel — the open layer with the `_brain_only…` suffix stripped, or else `<output folder>/<that name>_original.tif` — since `_ExtRm`/`_NoBG` layers are zeroed outside the brain; it prints which source it used, or a warning if it found none. The simplest setup is to pick the raw channel as Signal layer directly. The signal layer's contrast **low** limit is set to the value used afterwards, and **high** adaptively from this fish's own real signal (see 7d).

Bbox padding is read from the shared **Correction Settings** (7b) — the same value every other correction tool uses (default 15) — as the STARTING pad for each slice; with auto-grow on (the default), a slice that needs more room grows past this on its own.

Click **Protect Skin as Label**. The status line reports the skin label's own final pixel count and how many real labels touch skin; the report box below it lists **which ones** touch or sit near skin anywhere in the fish (plus any repaired or still-missing slices), so you know exactly which cells sit closest to the brain edge and are worth a closer manual look (they're the ones most at risk of already having a pre-existing bleed from before this run, which protecting skin now doesn't retroactively undo on its own).

Remove Skin Label — clears every voxel currently equal to **-1** back to background, a plain bulk clear (not connected-component-restricted); always -1, never the shared Label A/B fields. Use it whenever the skin label is no longer needed, e.g. right before Save Labels or Statistics if you don't want it in the final output.

Hide skin label in the viewer — a checkbox right below Remove Skin Label. Purely visual: makes label **-1** fully transparent in the display of the Labels layer selected in the shared selector, without touching any data at all — unlike Remove Skin Label, nothing is deleted. Every OTHER label keeps its own real, distinct color exactly, both while

hidden and after toggling back off (skin's own color returns too). Useful whenever skin's own territory is visually cluttering the view of the real cells you're correcting.

napari has no built-in property for "hide just this one label, show everything else" — its own native per-label isolation feature (`show_selected_label`) only does the *opposite* (show just one, hide the rest), and that direction is hard-wired directly into napari's own GPU shader, which has no equivalent branch for excluding a single value. The only mechanism whose per-label transparency genuinely reaches the real rendered image is a fully custom color mapping (one explicit color per label) — built fresh each time this checkbox is toggled on, from the highest label ID present (`labels.max()`) up: IDs that don't actually occur cost nothing extra, since an unused color-table entry is simply never looked up. Toggling stays well under a second even the very first time in a session, and near-instant on every toggle after that, with full color fidelity preserved throughout.

3D display mode — not the toggle click itself, but every subsequent refresh while hidden — uses a custom `_FastDirectLabelColormap` that forces napari's fast array-index cast path for this checkbox's own color mapping specifically (not napari's colormaps generally): whenever the displayed view is recomputed, napari casts the raw label array into a color-lookup texture; in 2D that's just the current slice, but in 3D display mode it's the *entire volume*, so which cast path napari's own backend selection routes a `DirectLabelColormap` through (a per-voxel hashmap lookup vs. a plain array index, depending on what's installed in the environment) has a real, measurable effect on every 3D refresh while skin is hidden.

Auto-correct Labels

Runs the exact same 6-step pipeline the Cellpose-SAM Segmentation section's own Auto-correct labels via contrast sweep checkbox chains onto a fresh run — but against the **Labels layer selected in the shared selector above** (7a), as it currently stands, instead of a just-produced one. This means a labels layer segmented earlier — either route, an earlier session, or one loaded via Load Labels layer (.tif) — can get the same treatment (self-calibrated contrast sweep, skin protection + debris cleanup, per-cell correction — 2D against skin for a touching cell, 3D otherwise — then a final debris cleanup) without re-running Cellpose-SAM segmentation from scratch.

Uses the shared **Signal layer** / **Labels layer** / **Brain mask layer** selector (7a), the same **Contrast low (best_lo) calibration** controls Protect Skin as Label reads (7d), and the shared **Correction Settings** (7b) — for every real cell's own correction, for skin's own trim (same starting pad as the cells), and for the contrast sweep's own sample crops — no separate, duplicate set of fields to fill in here. The contrast sweep and every cell correction read the Signal layer; skin's trim switches to the raw channel exactly as Protect Skin as Label does.

A cell corrected in 2D against a touching skin label gets the same automatic repair described above for skin: if it comes back with no footprint at all on some slice it used to occupy, it's independently re-seeded (from its own original footprint on that slice, never touching what a different real label now legitimately owns there) and re-corrected, up to 2 tries, before being reported as a genuine gap. A cell corrected in 3D (the normal case for a cell that never touches skin) doesn't need this — its own cross-slice seeding already keeps every slice non-empty by construction.

Click **Auto-correct Labels**. On success, the selected Labels layer is updated in place (skin protected as its own label, every real cell corrected), the signal layer's own contrast limits move to the calibrated threshold and an adaptive high limit measured from this fish's own real signal (see 7d), Protect Skin as Label's own report box is populated, and the full consolidated report (same format the chained Cellpose-SAM pipeline produces, including the *possible non-microglia blobs* list) appears in the log box below the status line.

If Soften label contours (sanding) is checked (Common Settings, default on) — same as the Cellpose-SAM-triggered flow — a further sanding stage chains on right after, over every corrected cell: interior cavities/vesicles are filled first if that checkbox is also on, then each cell's contour is softened. The button stays disabled and the status line reads *"…Sanding label contours…"* until that stage finishes too; the full sanding report is appended to the log box underneath the auto-correct report. This button never saves the labels to a file on its own (before or after this stage) — it only ever mutates the selected Labels layer in place (use Save Labels), unlike the Cellpose-SAM Segmentation route, which does save `_cp_krendl_ac.tif`/`_cp_krendl_ac_snd.tif`. It does save the **report**: as plain text to `<output folder>/<labels layer name>_autocorrect_report.txt` (header with date, labels layer and signal layer, then the report; the sanding report is appended when that stage runs), replacing that layer's previous report. Build GT-Correction Package picks the newest one up automatically.

Order with Protect Skin. Either order works. If you click **Protect Skin as Label** first, this button finds the skin already in the layer and **reuses it exactly as it is** (the report says *"Skin was ALREADY protected… reused exactly as it was, not re-protected or re-trimmed"*); it still runs its own contrast sweep for the cells, and because both buttons run the same sweep on cells only, they calibrate to the same `lo` (both crop their samples with Calibrate Correct-Label Contrast's Bbox padding). If skin is not there, this button protects it first. If skin ends up not protected (for instance a brain mask that covers the whole frame, so there is no outside to protect), the run **stops with an error before touching any cell**.

Save Labels

Saves the Labels layer selected in the shared selector (7a). Opens a file-save dialog pre-filled with the output folder (see Section 11) and the layer name as the filename. Choose a location and filename, then click Save.

Labels are saved as zlib-compressed `int32` TIFF. Each voxel value = label number (0 = background, -1 = skin if protected).

> **Save Labels is separate from Create Labels by design.** This lets you edit labels in napari (split, delete, merge) before saving the final result.

> After saving labels, switch to **Tab 4 — Statistics** to compute measurements for each cell.

---

8. Tab 4 — Statistics

This tab computes a comprehensive set of morphological, spatial, intensity, and brain-region measurements for every label and saves them to a CSV file. It is intentionally separate from Tab 2 so there is room to configure all options comfortably before clicking Generate.

For what each output column means, see Section 12 below. For the algorithm/formula/library behind each one — useful if you're auditing a result or citing the method — see the separate STATISTICS_GUIDE.md.

> **Which Labels layer is measured:** the one currently selected in napari's layer list, or — if the selection isn't a Labels layer — the topmost Labels layer. (This tab doesn't use Tab 3's shared selector.) Select the right layer before clicking Generate Statistics.

Description backend

Selects the engine used to generate the plain-language `description` column in the CSV.

Option	Internet	Cost	Notes
Rule-based (offline)	No	Free	Always available; template-based sentences
Ollama (local, free)	No	Free	Runs a local LLM on your machine
OpenAI API (paid)	Yes	Pay-per-token	GPT-4o-mini recommended for low cost
Claude API (paid)	Yes	Pay-per-token	claude-haiku-4-5 recommended for low cost

See Section 13 for detailed setup instructions for each backend.

Ollama sub-panel (shown when Ollama is selected)

- **Endpoint:** URL where Ollama is running. Default: `http://localhost:11434`. Change this if Ollama runs on a different machine or port.
- **Model:** The Ollama model name to use (e.g. `llama3`, `mistral`, `phi3`). Must be pulled first (`ollama pull llama3`).

API sub-panel (shown for OpenAI or Claude)

- **API Key:** Your secret API key. Shown as dots (password field). **Saved encrypted in your OS's credential store** (not the plugin's plaintext config file) and prefilled next session — see the note under Step 2 below.
- **Model:** The model identifier (e.g. `gpt-4o-mini` for OpenAI, `claude-haiku-4-5-20251001` for Claude).
- **Base URL:** Optional. Leave blank unless you use an OpenAI-compatible proxy or self-hosted endpoint.

Intensity statistics (optional)

Image layer dropdown — select an Image layer from your napari session (or leave as "None" to skip).

When an image layer is selected, three additional columns are computed per label using the raw intensity values inside each cell's mask:

- `mean_intensity` — average pixel intensity inside the label
- `integrated_intensity` — total sum of all pixel values (proportional to total fluorescent material)
- `intensity_cv` — coefficient of variation (std / mean) — a measure of how uniform the signal is; 0 = perfectly uniform, high values = heterogeneous staining

> Select the microglia channel (usually the green channel, ch1) for biologically meaningful results.

Brain regions (optional)

Assigns each cell to a named anatomical brain region and computes its distance to the nearest region boundary.

Boundary lines dropdown — select a Shapes layer containing one or more `line` or `path` shapes (other shape types are ignored), or leave as "None" to skip.

Region names text field — enter the region names separated by commas, listed anterior to posterior. For N boundary lines, provide N+1 names. Left blank, regions are named `Region 1`, `Region 2`, …; too few names are padded with `region_<n>`.

Example: If you draw one line separating the optic tectum from the hindbrain, enter:

```
Optic tectum, Hindbrain
```

Fish orientation and axis convention:

In these stacks, the fish lies along the **X axis** with the head pointing toward X = 0 (anterior = small X, posterior = large X). Y runs from 0 to 2048 top-to-bottom. The optic tectum / hindbrain boundary therefore runs roughly **top to bottom along Y**, separating the left part of the image (optic tectum, small X) from the right part (hindbrain, large X).

How to draw region boundaries:

1. In the napari toolbar, click **New shapes layer** (or add via Layers → Add shapes layer).
2. Select the **path** tool in the toolbar (a polyline — click once per vertex, double-click to finish). Use **path** rather than **line** so you can follow the curved anatomy of the optic tectum / hindbrain boundary.
3. Click along the boundary curve **from top to bottom** (Y = 0 toward Y = max), following the anatomical contour. The optic tectum will be on the left of your drawn path (smaller X), the hindbrain on the right (larger X). Double-click on the last point to finish.
4. For multiple regions, draw one path per boundary, each running top to bottom.
5. Select the Shapes layer in the **Boundary lines** dropdown and type your region names.

The boundaries are sorted automatically by mean X position of their vertices (leftmost = most anterior). For each cell, the plugin finds the nearest segment on the boundary curve and uses its orientation to determine which side the cell falls on (left = more anterior region, right = more posterior region).

Two additional columns are added to the CSV:

- `brain_region` — name of the region this cell belongs to
- `region_boundary_dist_um` — distance in μm to the nearest region boundary line

Output columns (checklist)

A scrollable list with one checkbox per CSV column (or column group — e.g. `centroid_vox` covers the three Z/Y/X columns, `bbox_vox` the six corner columns, `nearest_neighbor 1st` the label + distance pair). The defaults cover core morphology; several marginal or unvalidated columns start unticked. **All** ticks every box, **Reset** restores the defaults. `label` is always included. Intensity and brain-region columns are only produced when their inputs are set; if one is ticked without its input, the status line warns and the column is skipped.

Outlier bounds (read-only)

Above the button, a note shows the **is_volume_outlier ceiling** (the largest GT-confirmed cell so far, or "not yet measured"); the floor is the Min volume shown in Create MG Labels' Common Settings. This tab only ever reads both bounds — they're set exclusively by the GT Toolkit Tuning Tool (10l).

Generate Statistics (button)

Click to compute. Runs in a background thread. When complete:

- A CSV file is saved to the output folder (see Section 11), named `{opened_file_stem}_statistics.csv` (or the Labels layer's name if no file was opened).
- The status line shows how many labels were processed.

The CSV contains one row per label with up to 52 columns, depending on the checklist and which optional inputs are set. See Section 12 for a full description of every column.

The `is_volume_outlier` column (when ticked, the default) is `True` for a cell on **either** side of the two GT-tracked bounds (a bound that hasn't been measured yet flags nothing):

- **Bigger than the outlier ceiling.** Neither route ever deletes an oversized cell automatically — it's flagged for a human to look at instead, since "unusually large" is far more often a real biological outlier or an under-corrected merge of two touching cells than definite debris.
- **Smaller than the Min volume floor.** Both routes only auto-remove a cell once it drops below `Final min-size fraction × Min volume` (the golden-ratio cutoff, 6a — Cellpose-SAM's final safety-net stage

in 6c, the Pixel Classifier's own volume filter in 6b) — anything above that deletion line but still under the confirmed floor itself survives untouched on either route, yet is still smaller than any cell ever proven real. Flagging it here (rather than silently trusting it) surfaces exactly that gray zone for review.

Score Against GT — in Section 10e, Tab 6 — Sweeps & Utilities. Scores any two Labels layers already in the viewer against each other.

---

9. Tab 5 — AI Tools

Always visible, regardless of GPU. A banner at the top reports your GPU situation and adjusts its tone accordingly — checked once when napari starts:

Situation	Banner
No CUDA GPU detected	Red, bold: training/ inference will run on CPU — can take **days to months** for a full run instead of hours. Still usable for small experiments.
CUDA GPU present, under 8GB VRAM	Amber, bold: training may still work with a **reduced `batch_size`** (try 2, or even 1) but could be slow or hit out-of-memory errors.
CUDA GPU present, ≥8GB VRAM	Green, quiet confirmation — no action needed.

A smaller GPU, or none at all, doesn't mean the tools are useless — just slower, or in need of a smaller `batch_size`. GT Annotation itself needs no GPU at all.

Email notification (optional) — in Section 10h, Tab 6 — Sweeps & Utilities, General category. When its switch is on, every training run below emails when it stops.

A switch below that picks one of two mutually-exclusive groups. Only one is shown at a time.

9a. GT Annotation

Hand-draw polygon boundaries on key slices to create brain/skin ground-truth masks — the manual annotation step that produces training data for the MONAI model.

1. **Image layer** — pick the Image layer to annotate from the dropdown (populated from whatever's open in the viewer — use **Open TIF / IMS file** in Tab 1 first if nothing is loaded yet). Selecting a layer here automatically creates a `brain_polygons` Shapes layer (yellow) if one doesn't already exist.
2. **Draw polygons** — select the `brain_polygons` layer in the Layers panel, choose napari's polygon tool, and trace the brain boundary on key slices — roughly every 10 slices (e.g. 0, 10, 20, 30…). At least 2 polygons on different slices are needed. If one of your key polygons is on **slice 90**, it is copied unchanged to every slice beyond 90; without a slice-90 polygon that propagation is switched off (with a console warning) and slices past your last key slice reuse the last key polygon.
3. **1. Interpolate Polygons** — resamples each drawn polygon to 96 evenly spaced points, gives every polygon the same winding direction as the first one, then interpolates point-to-point between the nearest key slices along Z. Produces a `brain_polygons_interpolated` layer (cyan) — review it before continuing.
4. **2. Generate Masks** — rasterizes the interpolated polygons into a brain mask (median-filtered over 3 slices along Z to smooth slice-to-slice jitter), saves `<stem>_brain_mask.tif`, `<stem>_skin_mask.tif` (both 0/255), `<stem>_original.tif`, `<stem>_brain_only.tif`, `<stem>_skin_only.tif`, and both polygon `.npz` files to `<source_folder>/<source_stem>/` (the same output-folder convention as every other tab — see Section 11). Also adds `brain_mask`, `skin_mask`, and a `<stem>_brain_only` layer to the viewer, without touching or hiding any of your other layers.

> **Note:** this section has no file-open button of its own. It annotates the Image layer you pick in its dropdown, but the output folder and the `<stem>` in the file names come from the file most recently opened via **Open TIF / IMS file** in Tab 1 — Generate Masks refuses to run if no file has been opened that way.

9b. MONAI Training

Prepares training data and launches MONAI U-Net training — the model Tab 1 uses for skin removal.

Prepare Training Data converts raw+GT fish folders (the output layout GT Annotation produces) into the HDF5 dataset the trainer needs. Leave the brain/skin directory fields blank to use the training script's own built-in defaults, or list your own comma-separated paths; also set the output dir (default `training_data_v2`), tissue (brain/skin), `n_val`/`n_test` (5/5), `split_seed` (16) and `num_workers` (75% of CPU cores). Runs `prepare_data.py` through `conda run` in the background (the status line says it can take ~15 min) without blocking the UI. On success, auto-fills the Train MONAI section's data directory.

Train MONAI U-Net launches the actual training run — configure the data dir, model dir (default `models_v2`), `epochs` (1500) / `batch_size` (2) / `lr` (1e-4) / `val_every` (5) / `ckpt_every` (50) / GPU index (0), optionally point `resume` at an existing checkpoint to continue training instead of starting fresh, then click **Launch Training**. See How training launches work below for the shared **Patience (checkpoints)** early-stopping field and what happens next.

9c. Cellpose-SAM Training

Extracts fine-tuning crops and launches Cellpose-SAM training — the model Tab 2's Cellpose-SAM Segmentation uses.

Extract X/Y/Z Patches generates 2D training crops in all three orientations from a full-fish image + GT labels pair: **XY** at native resolution, **XZ**/**YZ** with the Z axis stretched by the voxel-scale-derived anisotropy so all three orientations end up at the same effective pixel scale. Cell crops are placed around GT cells, up to `max/orientation` per orientation. A skin label (-1) in the GT is written as background (0) in every mask crop, since Cellpose only knows background and cells; the GT file itself is not changed. This is the method behind every real Cellpose-SAM training dataset in this project (`train_cellpose_512`, `_multi`, `_multi3`).

- **crop_size** / **crops/slice** / **max/orientation** / **min_gt_pixels** / **seed** — control crop dimensions and how many are sampled; defaults (512 / 5 / 320 / 10 / 42) match what's been used for every real dataset so far.

- **Add skin crops (from the GT's skin label -1)** (checked by default) — teaches the model that bright skin is not a cell. It needs a GT that still carries the skin label from Protect Skin as Label; with no -1 in the GT, no skin crops are made and the status line says so. Per orientation, on top of the cell crops, it adds:
  - **Cell-skin boundary** crops (default **20** % of that orientation's cell crops): placed around a cell lying within 3 px of skin, so the crop holds the cell and the skin it touches — the hardest case, where the model must draw the cell up to the skin and stop.
  - **Skin-only** crops (default **10** %): placed on skin, with no cell in the crop, so the mask is empty.

  Take the **Image** from the raw channel (`<stem>_original.tif` or the `.ims` channel), not `_ExtRm`: `_ExtRm` has the skin zeroed, so there would be nothing to learn from. Skin crops are numbered from index 100 (boundary) and 200 (skin-only) in the file name (e.g. `xy_045_200.tif`); Train Cellpose-SAM keeps an empty-mask crop only when its index is 200 or more. The cell crops are the same with or without skin crops (the skin crops use their own random sequence).
- **Z (µm)** / **XY (µm)** — the fish's voxel size (default 1.0 / 0.174), which sets the Z-stretch factor.
- **Clean truncated labels** (checked by default) — a crop framed around one target cell can also graze the corner of a *different* nearby cell purely by chance, sometimes showing only a tiny sliver of it. Left in, that sliver is still a valid-looking label with a wildly wrong flow-center target (Cellpose points flow vectors toward each object's own centroid — a fragment's visible centroid is nowhere near the real cell's center). With this on, any label whose crop-visible pixel count falls below **Min visible fraction** (default 0.90) of its true full-slice cross-section gets zeroed out of that crop — automatically, right after generation, not as a separate step you have to remember. A crop's own intended target cell is essentially never affected (it's already well above this threshold in the crop it was generated for); this specifically catches incidental neighbors. Before writing anything, the whole crop folder is backed up to `<folder>_pretrunc_backup` — skipped on a second run if that backup already exists, so re-running never overwrites an earlier backup with already-cleaned files.

Combine Crops into Shared Training Folder pools one fish's own crop folder (from Extract X/Y/Z Patches above) into a single shared folder every fish's crops get pooled into for a multi-fish training run — browse to **Fish crop folder** and **Combined folder**, then click **Combine**. Every file is symlinked in (nothing copied), prefixed with the

name of the crop folder's **parent** folder — normally the fish's own full data-folder name (e.g. `NT39-3dpf-Crispr-ctrl-D1F4_2024-09-05_15.38.01`), so keep each fish's crop folder inside its own fish folder — never a short Dish/Fish tag like `D1F4` alone, since that shorthand repeats across different experiments and ages in this project and isn't a safe identifier in a shared pool on its own. Safe to run again later for a new fish: an already-linked fish's crops are left untouched. On success, the combined folder is filled into the **Data dir** field below, ready for Train Cellpose-SAM.

Train Cellpose-SAM launches fine-tuning — set the **Data dir** (the crop folder; required), `model_name` (default `cpsam_microglia_xzyz`), `n_epochs` (200) / `batch_size` (4) / `save_every` (10) / `log_every` (5) / `lr` (1e-4), then click **Launch Training**. It runs `train_xzyz.py` in the conda env `cellpose` by default (config key `cellpose_conda_env`), logging to `<data dir>/logs/gui_launch_<timestamp>.log`. The `pretrained` field defaults to whatever checkpoint is already loaded in Tab 2's Cellpose-SAM Segmentation section — i.e. by default this **continues training from where Tab 2 left off**, though you can browse to a different starting checkpoint (or type a builtin name like `cpsam`) if you want to start fresh. Crops whose mask ends up empty are skipped (for example, a crop whose every label was zeroed by Clean truncated labels), except skin-only crops, which are kept as background-only examples; the log reports both counts. `branch_weight`/`branch_radius` control the project's branch-weighted loss (weights thin/branch-tip pixels more heavily during training so the model doesn't under-segment fine processes) — set `branch_weight` to `0` to disable it and use the standard Cellpose loss instead.

Calibrate branch_radius (from GT) measures the real branch thickness of actual GT-labeled cells instead of guessing `branch_radius` by hand. Browse to a GT labels volume, set **scale Z**/**scale XY** to match its voxel scale, and click **Calibrate branch_radius**. The tool 3D-skeletonizes every labeled cell, decomposes each skeleton into branch segments, measures each segment's mean diameter via an anisotropic distance transform, and takes the **thinnest quartile** (the distal branch tips — the fine processes `branch_weight` exists to protect, as opposed to thick soma-adjacent segments) as the basis for the recommendation, converting that radius from microns to pixels at the given scale. Run from here, it only reports what it measured and changes nothing — its "This is verified ground truth" box is hidden. Only when the **GT Toolkit Tuning Tool** (10l) runs it with GT verified does the result feed a never-falling ceiling tracked across every fish calibrated so far (a thicker "thin branch" measured in any fish sets a higher bar the field must still meet, the opposite direction from Min volume's never-rising

floor) — that ceiling, not just this run's own measurement, is then applied to the `branch_radius` field above (and shown in a read-only **Recommended branch_radius** line underneath it) and saved to config. This can take anywhere from several seconds to a couple of minutes depending on how many cells are in the GT volume; it runs in a background thread so napari stays responsive.

Verify Best Epoch (GT Sweep) — in Section 10d, Tab 6 — Sweeps & Utilities. Confirms or corrects the recommended-checkpoint pointer above against real GT.

How training launches work

Both "Launch Training" buttons (MONAI and Cellpose-SAM) start a **detached background process** rather than running inside napari itself — the command runs via `conda run -n <env> --no-capture-output <script> ...`, launched so it keeps running even if you close napari (technically: `setsid()` on Linux/Mac, `CREATE_NEW_PROCESS_GROUP | DETACHED_PROCESS` on Windows). This works identically on all three platforms without needing `tmux` (which isn't available on Windows at all).

- **Live status** — a log-tail view in the GUI refreshes every 8 seconds (deliberately coarse — there's no benefit to checking more often on an hours-to-days job). MONAI's log is `<model dir>/gui_launch_<timestamp>.log`; Cellpose-SAM's is `<data dir>/logs/gui_launch_<timestamp>.log`.
- **Patience (checkpoints)** — an integer field in both groups, and it's the *same rule* for both: stop automatically once N checkpoints in a row pass with no improvement in the model-selection metric (Full-brain Dice for MONAI — higher is better; `test_loss` for Cellpose-SAM — lower is better; the plugin handles the direction per metric automatically). `0` disables early stopping entirely. This is enforced by the plugin itself, reading each checkpoint as it lands in the log — `train.py`'s own built-in `--patience` flag is always overridden to an effectively-infinite value so it can't quietly stop training before the GUI's check does; there's exactly one early-stopping mechanism, not two that happen to look the same in the UI.
- **Reopening napari mid-training** — the plugin remembers the running job (including the patience setting) and automatically reconnects to it, so you don't lose visibility into a training run just because you closed and reopened napari. You'll see "Resumed monitoring PID …" instead of an empty status.

- **Reopening napari *after* the job already finished** — if the training process is no longer running by the time you reopen napari (e.g. it ran to completion, or crashed, while napari was closed), the status line reports that immediately: "Training (PID …) finished while napari was closed. Best … at epoch …". For Cellpose-SAM this is also the point where the recommended-checkpoint pointer (see below) gets written, if it wasn't already. Either way you don't need to have napari open at the exact moment a run finishes — the next time you open it, it tells you what happened.
- **Email notification** — see Section 10h, Tab 6. Unlike the previous two bullets, this one doesn't depend on ever reopening napari at all — the email arrives on its own schedule regardless.
- **Stop Training** — kills the training process and everything it spawned (the `conda run` wrapper spawns a child `python` process, and both are terminated together). Early stopping uses this same kill mechanism internally.
- **Which checkpoint to use afterwards** — MONAI's `train.py` already tracks and saves its own best checkpoint as `best_model_fullstack.pth`, so nothing extra is needed there. `train_xzyz.py` (Cellpose-SAM) has no such tracking — it only saves periodic epoch checkpoints — so whenever the plugin observes a Cellpose-SAM run has stopped (finished on its own, early-stopped, or discovered already-finished the next time you reopen napari — see above), it writes a small pointer file, `<model_name>_best_recommended.txt`, into the run's `models/` folder next to the checkpoints. It's a one-line text file naming the best-scoring checkpoint (by `test_loss`), e.g. `cpsam_microglia_xzyz_epoch_0150` — not a copy of the (often 100s-of-MB) checkpoint itself, and not an OS symlink either (those need elevated privileges/Developer Mode on Windows), so it works the same way on every platform with no special permissions. The GUI's status line also reports the best epoch directly once the run stops.
- **If a script isn't found** — `prepare_data.py`/`train.py`/`train_xzyz.py` ship with the plugin (bundled under `napari_zf_microglia_ai/training_scripts/`, installed as package data), so this shouldn't normally happen. If you want to point at a locally modified copy instead, override the path via the `monai_prepare_script_path`/`monai_train_script_path`/`cellpose_train_script_path` keys in `~/.config/napari-zf-microglia-ai/config.json`.

---

10. Tab 6 — Sweeps & Utilities

Twelve tools, gathered here so Tabs 1-5 stay focused on running the pipeline rather than tuning it. Each is individually collapsible: click a section's title checkbox to hide its contents, so you can keep only the one you're actively using expanded. Every sweep operates on its own Tab 1/2/5 sliders/fields — this tab is where the sweep runs, not where its results live. Five are GT-*sweep* tools (test a small parameter grid against a handful of proxy cells or one mask, as a fast approximation); the **GT Toolkit Tuning Tool** runs those (and the branch_radius calibration) for one fish in one go and is the only thing that applies their findings; the rest (Score Against GT, Build GT-Correction Package, Calibrate Correct-Label Contrast, and the General-category utilities below) are related tools that don't fit that "sweep against ground truth" shape.

On-screen order, top to bottom: the "Show tools for…" filter, then the **GT Toolkit Tuning Tool** (10l, runs every applicable sweep in one shot), then the individual GT sweeps in pipeline order — **Verify MONAI Threshold / Erosion** (10a), **Verify BG Threshold / Signal Erosion** (10b), **Verify Smooth σ XY / σ Z** (10g), **Calibrate Cellprob + Krendl + Min size + Large-contact** (10c), **Calibrate Correct-Label Contrast** (10i), **Verify Best Epoch** (10d) — then the GT utilities, **Score Against GT** (10e) and **Build GT-Correction Package** (10f), and last the session utilities, **Autosave Labels Layer** (10k), **Email notification** (10h) and **Drift View in 3D** (10j).

"Show tools for…" filter — with this many tools stacked in one tab and no indication of which pipeline each belongs to, it isn't obvious at a glance what any given tool is even for. Four checkboxes at the top of the tab let you hide the ones you don't need:

Category	Tools shown
Skin Removal (MONAI)	10a. Verify MONAI Threshold / Erosion
Pixel Classifier segmentation	10b. Verify BG Threshold / Signal Erosion, 10g. Verify Smooth σ XY / σ Z
Cellpose-SAM segmentation	10c. Calibrate Cellprob + Krendl + Min size + Large-contact, 10d. Verify Best Epoch, 10f. Build GT-Correction Package
General (any pipeline)	

Category	Tools shown
	10e. Score Against GT, 10i. Calibrate Correct-Label Contrast, plus the other General-only utilities (Email notification, Drift View in 3D, Autosave Labels Layer, GT Toolkit Tuning Tool)

All four are checked by default (nothing is hidden until you actually uncheck something), and your choice is saved to config and restored next time you open napari. Unlike Tab 5's MONAI/Cellpose-SAM training switch, these are independent checkboxes, not a mutually-exclusive radio choice — you can leave several checked at once if you work with more than one pipeline.

Every sweep here keeps a running history across every fish you've ever swept, not just the one you swept last. A single sweep run only ever proves something about the one fish it ran against; treating that one result as the final answer, and overwriting whatever an earlier sweep on a different fish had found, throws away real evidence for no reason. Each tunable value therefore falls into one of two categories, and the tool remembers a per-fish history for whichever one applies:

- **Floors and ceilings** (Min volume, Min hole size, branch_radius) protect against a specific failure — discarding a real cell, erasing a real hole, or under-protecting a real thin branch. These track the single most demanding piece of evidence seen across every fish: Min volume/Min hole size only ever get *stricter* (lower) as a new fish proves an even smaller real example exists; branch_radius only ever gets *more generous* (higher) as a new fish proves an even thicker "thin" structure needs covering. A value already proven safe by one fish is never silently relaxed by a later sweep that simply didn't happen to test anything as extreme.
- **Everything else** (MONAI Threshold, MONAI Erosion, BG Threshold, Signal Erosion, Smooth σ XY/Z, Cellprob) has no safety direction — each fish's sweep just finds that one fish's own local optimum, and there's no reason to trust the most recent fish over any earlier one. These are **averaged** across every fish swept so far instead. (Safe-merge max gap/min contact, Cellpose Min size and Large-contact merge are a third kind: their raw samples are pooled across fish and re-optimized together — see 10c.)

Re-running a sweep against a fish you've already swept before, for example after correcting that fish's ground truth, updates that fish's own entry in place rather than adding a duplicate — the history is

keyed by the GT file's name (the GT brain mask for 10a, the GT labels file otherwise, the fish folder for the GT Toolkit's own first step), so it stays accurate rather than growing stale entries forever. When a value is applied, the sweep's status message reports both numbers: what this specific fish's sweep found, and what the aggregated recommendation is once that fish's result is folded in.

Who is allowed to apply a result. A sweep run on its own — by clicking its own Run button here — only **reports** (best point, score): its "This is verified ground truth" checkbox is hidden, so it never moves a shared value, a slider, or a recommended line. Results are applied only by the **GT Toolkit Tuning Tool** (10l), and only when its own **"This is verified ground truth"** box is ticked (off by default, un-ticks itself after every run) — the GT labels a sweep takes could be anything, a raw uncorrected prediction just as easily as a hand-verified fish. The one exception is **Verify Best Epoch** (10d), which picks a training checkpoint rather than a pipeline value and keeps its own visible one-shot checkbox.

Sliders/fields a sweep can apply a value to show a separate, read-only "Recommended: X" line underneath them, in whichever tab that control actually lives (Tab 1 for MONAI Threshold, MONAI Erosion, BG Threshold and Signal Erosion; Tab 2 for Min hole size, Smooth σ XY/σ Z, Min size, Cellprob threshold, Safe-merge max gap/ min contact and Large-contact merge; Tab 5 for branch_radius) — distinct from the slider itself, which stays freely editable at all times. An applied result both sets the live slider *and* updates this read-only line; moving the slider afterward to test something else never touches the recommended line, so what the evidence actually supports is never silently lost. Min volume/Max volume (Common Settings, 6a) already work this way by design — informative value only, no slider at all — since nothing should ever hand-tune them in the first place.

10a. Verify MONAI Threshold / Erosion (GT Sweep)

The cheapest of the sweepers here. Checks MONAI's own brain segmentation — is the current **MONAI Threshold** and **MONAI Erosion** (both Tab 1 fields) combination actually the one closest to a hand-corrected brain mask? Unlike the other sweepers, this scores a single whole-volume mask, not multiple per-cell labels — Dice/IoU/precision/ recall between the predicted brain mask and a GT mask directly, no complex-cell selection or bounding-box cropping needed.

1. **Image** — the raw volume to run inference on. Must be a **TIFF, not `.ims`** (loaded via `tifffile.imread`, unlike Tab 1 itself which does

support `.ims`), and must be the true pre-MONAI raw image — feeding it an already brain-masked image would bias the very segmentation this tool is scoring.
2. **GT brain mask** — a *hand-corrected* brain_mask.tif, e.g. from **GT Annotation** in Tab 5 (the polygon annotation tool's own rasterized output) — not a MONAI prediction.
3. **Threshold min/max/step** and **Erosion min/max/step** — define the grid. Defaults (0.15–0.35 step 0.05, 0–4 step 1) span 5×5=25 points.
4. **Sieve** (default on) — auto-narrows Threshold coarse → fine across 3 stages instead of running one flat grid: stage 1 is the min/max/step above, stages 2 and 3 each re-center on the previous stage's winner using their own +/- width and step fields (defaults ±0.05 step 0.01, then ±0.02 step 0.005), clipped to stage 1's min/max. Erosion's own grid stays the single range given above at every stage.
5. Click **Run Threshold/Erosion Sweep**.

MONAI's sliding-window inference (the only genuinely expensive, GPU-bound step) runs **exactly once**, producing a raw probability map. Every threshold and erosion value in the grid is then just a cheap re-threshold + largest-component/fill-holes + optional erosion on that same probability map — no reloading the model, no repeat sliding-window passes. A full 25-point grid typically finishes in well under a minute on GPU, and still works (just slower) on CPU/MPS since it uses the same device selection as **Run Skin-Remover**.

The report is a 2D grid (rows = Erosion, columns = Threshold, cells = Dice%), with your current Tab 1 slider values marked; the best point is the highest Dice. Run through the GT Toolkit with GT verified, this fish's best point is folded into a running average across every fish swept so far, and **that average is applied to the Tab 1 MONAI Threshold/ MONAI Erosion sliders (and their Recommended lines) and saved to config**.

10b. Verify BG Threshold / Signal Erosion (GT Sweep)

Answers the same kind of question as 10d below, but for the Pixel Classifier path instead of Cellpose-SAM: is the current **BG Threshold** and **Signal Erosion** (both Tab 1, Background mode 2) combination actually the one that produces microglia labels closest to ground truth, or does a nearby combination do better?

1. **GT image** — the raw image or its `_ExtRm` (both give identical results, since only pixels inside the brain mask are read) — not `_NoBG`/ `_RndFill`, which already had a different background step applied.

2. **brain_mask.tif** — the *raw* (un-eroded) mask Tab 1 saves. MONAI inference itself is **not** re-run by this sweep — it only varies what happens *after* inference (background thresholding, signal erosion, labelling), so it needs an already-computed mask from a normal Tab 1 run rather than the model checkpoint.
3. **GT labels** — the corrected ground-truth microglia label volume for that fish (typically `<stem>_GROUND_TRUTH.tif`).
4. **BG Threshold min/max/step** and **Signal Erosion min/max/step** — define the grid. Defaults (1.0–1.8 step 0.2, 0–4 step 1) span 5×5=25 points.
5. **Sieve** (default on) — auto-narrows BG Threshold coarse → fine across 3 stages (defaults ±0.2 step 0.05, then ±0.05 step 0.01), same idea as the MONAI Threshold sweep above; Signal Erosion's grid stays fixed at every stage.
6. **Complex cells to test** (default 5), **Pad Z** / **Pad XY** (default 15 / 40 voxels around each cell's bounding box) and **Z (µm)** / **XY (µm)** (this fish's voxel size, default 1.0 / 0.174).
7. Click **Run BG Threshold/Erosion Sweep**. It finds the N most complex GT cells once (most skeleton branches first, least spherical as tiebreak), and for each grid point applies that BG Threshold + Signal Erosion to each cell's cropped region, runs Create Labels on the crop (using Tab 2's current σ XY / σ Z and Final min-size fraction, plus **Min volume** and **Min hole size**, both measured automatically from the GT itself — see below), and best-IoU-matches the result against GT.

Min volume and Min hole size are both measured from the GT, not read from Tab 2. The small-blob cleanup threshold used during the sweep is the true smallest labeled cell's own voxel volume in the GT labels you provided, not whatever Min volume Tab 2 currently shows — a fixed guessed number risks discarding a real small cell as noise if it's too high. Min hole size works the same way in reverse: it's the smallest genuinely real internal gap found anywhere in that GT's own cells (scanning each cell's per-slice footprint for background regions a human annotator deliberately left unlabeled), so the sweep never fills in a gap the ground truth itself confirms is real. Single-pixel gaps are treated as annotation noise rather than evidence when measuring this, since real GT checked during development showed those are common and unrelated to genuine structure.

Tab 2's read-only **Min volume** value, and the **"Recommended floor (from GT sweeps so far): N vox"** line under the Min hole size slider, track the running floor across every applied sweep. Each only ever **decreases** — once one fish's GT proves a cell (or a hole) of a given size is real, a *different* fish's sweep (which may simply lack any cell or hole

that small) can't raise it back above that. An applied run (GT Toolkit, GT verified) also sets the Min hole size slider to that floor — the slider stays freely editable afterward for your own testing; the recommended line keeps the real number safe regardless.

This sweep is considerably cheaper than the Cellpose-SAM one: MONAI inference only ever runs once (outside this tool, via a normal Tab 1 run), and neither Erosion nor BG Threshold require reloading a model. A full 25-point grid typically finishes in minutes, and works on CPU too (Create Labels already has a CPU fallback).

The report is a 2D grid (rows = Signal Erosion, columns = BG Threshold, cells = average IoU%), with your current Tab 1 slider values marked and compared against whatever the sweep found best (highest average IoU). Run through the GT Toolkit with GT verified, this fish's best BG Threshold/Signal Erosion point is folded into a running average across every fish swept so far (Signal Erosion has its own history, separate from MONAI Erosion's), and Min volume/Min hole size are updated as never-rising floors — **the resulting averages and floors are applied to Tab 1's BG Threshold/Signal Erosion sliders, Tab 2's Min volume value and Min hole size slider, and saved to config**.

10c. Calibrate Cellprob + Krendl Merge Parameters + Min size + Large-contact (GT Sweep)

One button, one `do_3D` network pass, five calibration steps against a full-fish GT labels volume — every Cellpose-SAM Segmentation parameter this plugin has (per the standing policy: no pipeline value is left uncalibrated).

Uses the Cellpose-SAM checkpoint loaded in Tab 2 and Tab 2's current **Min size**.

1. **Image** / **GT labels** — a full-fish `_ExtRm` `brain_only` image (the Cellpose-SAM route's input — not `_NoBG`/`_RndFill`, and not the raw pre-Tab-1 image) + its corresponding GT labels volume.
2. **Z (µm)** / **XY (µm)** — this fish's voxel size (default 1.0 / 0.174); drives the do_3D `anisotropy` parameter (Z/XY ratio) and the physical max_gap; independent of whatever's open in the viewer.
3. **Cellprob min/max/step** — define the grid for step (1) below (default −2.0 to 2.0, step 0.25).
4. **Sieve** (default on) — auto-narrows Cellprob coarse → fine across 3 stages (defaults ±1.0 step 0.25, then ±0.5 step 0.1); the one `do_3D` network pass is reused across all 3 stages, not repeated.

5. You get an email when it finishes or fails (~3h is well past the default 10-minute threshold) — see Section 10h.
6. Click **Run Full Calibration**. Run this way it only reports; to apply its findings to Tab 2, run it through the GT Toolkit Tuning Tool (10l) with GT verified. It never changes the shared Min volume/Min hole size floors.

What the five steps actually measure, in order:

1. **Cellprob** — sweeps against GT on raw voxel-level Dice/IoU (binarized foreground vs. background, instance identity ignored) — no GMM/Krendl/large-contact run at all here, to avoid entangling the Cellprob pick with not-yet-calibrated merge parameters. `min_hole_size` is measured once from this GT's own real holes and reused by every mask-formation call below.
2. Forms raw `cp_masks` at the winning Cellprob, reusing the same cached `do_3D` flows from step 1 — no extra network pass.
3. Compares those `cp_masks` against GT to jointly calibrate **Safe-merge max gap** / **Safe-merge min contact** against Krendl safe-merge's real OR-combined rule (`gap<=max_gap OR contact>=min_contact`), minimizing total corrections (missed + false merges) rather than avoiding one error type at any cost — an over-merge is fixable with Split Label, an under-merge with Join Labels.
4. Forms the SAME raw masks again but fully **unfiltered** (`min_size=0`, still reusing step 1's cached flows), and compares those against GT to calibrate the early **Min size** noise filter. Unlike step 3, a real fragment `min_size` discards is simply gone — no "restore a deleted fragment" tool exists — so this step weighs a missed real fragment far more heavily than a noise fragment that merely survives a little longer (GMM/Krendl/the final min-size safety net still get three more chances at that one).
5. Runs step 2's raw masks (`min_size=15`, matching production exactly) through GMM cleanup + Krendl safe-merge at step 3's own result, then compares what's **still split** against GT to calibrate **Large-contact merge**. Same fewest-total-corrections objective as step 3 (a false merge is fixable with Split Label, a missed one with Join Labels) — a genuinely symmetric failure mode, unlike step 4's.

Flow is not swept, and has no user control anywhere in this plugin: reading `cellpose/dynamics.py` shows its flow-error QC filter only runs when `do_3D=False` (2D/stitch mode) — under `do_3D=True`, which this plugin always uses, changing Flow threshold changes nothing about the result. It's fixed internally purely because `do_3D`'s call signature still accepts it — see Flow threshold in Section 6c.

A single `do_3D` network pass on a full-size fish can take a few hours, and that's this tool's entire cost regardless of how many Cellprob values are in the grid — steps 2-5 all reuse that one pass's cached flows, never a separate network pass each. **Stop Sweep** only cancels step 1, between grid points; since the network pass happens once upfront, it can't itself be interrupted mid-pass, and steps 2-5 (all cheap, CPU-side) always run to completion once step 1 picks a winner. This does **not** run detached — it won't survive closing napari. The report box streams Cellpose's internal `do_3D` progress live during step 1 rather than sitting on one static message — see the note under Run Cellpose-SAM Segmentation in Section 6c.

Pooling across fish (applied runs only): Cellprob is averaged across every fish swept so far (no safe direction to bias toward). Max gap/min contact, Min size, and Large-contact are each jointly re-optimized over the full pooled sample set across every fish calibrated so far for that parameter (not averaged per-fish then combined) — see `_update_merge_stats_history`/`_update_minsize_stats_history`/`_update_largecontact_stats_history`. The averaged Cellprob and every pooled recommendation are applied to the Tab 2 sliders (and their Recommended lines) and saved to config in one pass; a parameter with no samples yet across any fish is left unchanged.

10d. Verify Best Epoch (GT Sweep)

Answers a specific question the recommended Cellpose-SAM checkpoint alone can't: `test_loss` (what picks the recommendation, in Tab 5's Train Cellpose-SAM section) is a proxy for segmentation quality, not the real thing, and checkpoints often plateau within noise of each other. This tool checks the recommendation against actual ground truth on a small, deliberately hard sample:

1. **GT image** / **GT labels** — browse to a full-fish `_ExtRm` brain_only image (the same input do_3D uses in production) and its corrected ground-truth label volume (the same pair used to build training crops). Doesn't need to be a fish the current model was trained on, but usually is.
2. **Recommended epoch** — click **From pointer file** to pull it from `<model_name>_best_recommended.txt` (uses Tab 5's Train Cellpose-SAM section's own Data dir/model_name), or type it in manually if no pointer exists yet. A checkpoint for that epoch must exist in `<Data dir>/models/`.
3. **Checkpoints below** / **above** (default 2 / 2) and **save_every** (default 10 — must match the interval the training actually saved at),

Complex cells to test (5), **Pad Z / Pad XY** (15 / 40), **Z / XY (µm)** (1.0 / 0.174), and **Cellprob threshold** (defaults to Tab 2's current value).

4. Click **Run Epoch Sweep**. The tool:
   - Finds the **N most morphologically complex cells** in the GT volume (default 5) — ranked by skeleton branch count (most-branched first), sphericity as a tiebreak, *not* by cell size.
   - Crops each to its bounding box + padding (default 15 vox Z, 40 vox XY).
   - Runs `do_3D` inference at the recommended epoch plus **N checkpoints below and above it** (default 2 and 2 — a 5-epoch × 5-cell = 25-inference sweep by default), best-IoU-matches each prediction against its GT cell, and averages.
   - Reports a table plus a plain confirm/disagree verdict against the recommended epoch.

This can take a while — each `do_3D` call is a few minutes, so a default 5×5 sweep is roughly 30 minutes to a couple of hours. **Stop Sweep** cancels between checkpoints (not mid-inference). Unlike Launch Training, this does **not** run as a detached process and does **not** survive closing napari. You get an email when it finishes or fails — see Section 10h.

If the sweep disagrees with the recommendation, applying it — rewriting the recommended-checkpoint pointer to the sweep-confirmed epoch and loading that checkpoint as Tab 2's active Cellpose-SAM model — requires this tool's own **"This is verified ground truth"** checkbox (visible here, unlike the other sweeps; off by default, un-ticks itself after every run) to be ticked for that run. This tool is not part of the GT Toolkit Tuning Tool. Left unticked, a disagreement is still reported in full, it just doesn't swap the active model out from under you on the strength of GT labels you haven't actually confirmed are correct.

10e. Score Against GT

Whole-fish instance-segmentation scoring: Hungarian-matched TP/FP/FN/Score plus mean IoU/Dice (over matched pairs only), between any two Labels layers already loaded in the viewer — for validating a checkpoint pick, a cellprob/large_contact setting, or any other model/parameter comparison directly against ground truth.

1. **Predicted labels** / **GT labels** — pick any two Labels layers from the dropdowns (same shape required).

2. **IoU threshold for a match** — the minimum IoU for a predicted object to count as a true positive for a given GT object (default 0.5).
3. Click **Score Against GT**.

Runs synchronously (pure CPU, `scipy.optimize.linear_sum_assignment`) — fast enough at typical whole-fish object counts that a background thread isn't needed. **Score = TP − 0.5×(FP + FN)**. The report lists every matched pair (IoU%, Dice%, voxel counts, size delta), plus the FN (missed GT) and FP (spurious predicted) object IDs.

This is a genuinely different tool from the sweepers above: those each test a handful of parameter combinations against a handful of complex cells (or one mask) as a fast proxy; this scores one specific pair of label volumes completely, the way a final reported result would be scored.

10f. Build GT-Correction Package

Packages a Cellpose-SAM correction result for external manual correction — the exact file layout this project has assembled by hand for every fish sent out for ground-truth creation. The corrected result becomes future training/GT data, closing the loop between inference and the training tools in Tab 5.

1. **Fish stem** — the identifier used to name every file in the package (e.g. `NT39-3dpf-D1F4_2024-09-05_15.38.01`).
2. **Source image** — the `brain_only` image the segmentation ran on. **Browsing here also auto-fills the other fields** from that fish's own folder (Fish stem, Corrected masks, Raw Cellpose masks, Brain mask, Raw signal channel, Auto-correct report, Output folder) using the established `<parent>/<stem>/<stem>_<artifact>.tif` convention — override anything by hand afterward if it picked the wrong file. A GT Toolkit **Scan Folder** (10l) fills the same fields.
3. **Corrected masks** — the most-advanced Cellpose-SAM correction stage this fish actually has: **sanded** > **auto-corrected** > **Krendl-only**, whichever exists (auto-picked when Source image is browsed above; see 6c's Auto-correct/Sanding sections for what each stage means). Becomes `<stem>_cp_corrected.tif` — the file the reviewer edits first ("start here" per the guide).
4. **Raw Cellpose masks** (optional) — the pre-merge `do_3D` output (`*_cp.tif`), if you have it, included as `<stem>_cp_masks_3D.tif` for reference only (not corrected).
5. **Brain mask** — this fish's own raw (un-eroded) `brain_mask.tif`, included as `<stem>_brain_mask.tif` — auto-filled the same way as Raw Cellpose masks. **Without it, whoever corrects this package**

can't use Protect Skin as Label at all (that tool needs a brain mask layer).

6. **Raw signal channel** — the raw, unmasked microglia channel (`<stem>_ch1_original.tif`, which Run Skin-Remover saves next to its other outputs whenever a background mode is used), auto-filled from the fish's folder. Written into the package as `<stem>_original.tif`, zlib-compressed (a full fish is typically well under 100 MB). Protect Skin as Label needs it as its **Signal layer**: the source image (`_brain_only_ExtRm`) is zeroed outside the brain, so without this the reviewer has to open the fish's original `.ims` instead.
7. **Auto-correct report** — the `<...>_autocorrect_report.txt` saved by Auto-correct Labels or by Cellpose-SAM Segmentation's chained auto-correct stage. Auto-filled with the **newest** one in the fish's folder — check it belongs to the Corrected masks you are sending. Copied in as `<stem>_autocorrect_report.txt`, so the reviewer can see which cells the report flags (joint `group=[A, B]` corrections, cells touching skin, possible non-microglia blobs) before starting.
8. **Creation guide** (optional override) — defaults to the plugin's own `docs/GROUND_TRUTH_CREATION_GUIDE.md`; only set this if it lives somewhere else on your machine.
9. **Output folder** — where the package folder and `.zip` are created.

Brain mask, Raw signal channel and Auto-correct report can be left empty — the package still builds — but the status line then warns which of them the reviewer will be missing. Rebuilding into the same output folder replaces every file this tool writes, so a field left empty this time never ships a previous build's copy.

Click **Build GT-Correction Package**. Output:

```
<output folder>/
├── <stem>_GT_package/
│   ├── GROUND_TRUTH_CREATION_GUIDE.md
│   ├── <stem>_cp_corrected.tif
│   ├── <stem>_cp_masks_3D.tif        (only if provided)
│   ├── <stem>_brain_mask.tif         (only if provided)
│   ├── <stem>_original.tif           (raw signal channel, only if
provided)
│   ├── <stem>_autocorrect_report.txt (only if provided)
│   ├── <stem>_cell_statistics.csv    (label/volume/centroid/bbox
– quick reference, not the full Tab 4 output)
│   └── <stem>_brain_only_ExtRm.tif
└── <stem>_GT_package.zip             (the folder above, zipped)
```

The statistics CSV is deliberately minimal (label, volume in voxels and µm³, centroid, bounding box; largest cell first) — a quick reference for someone correcting labels, not the full Tab 4 Statistics output. It uses the voxel scale of the stack currently open in the viewer ((1, 1, 1) if none). The Source image is copied in as `<stem>_brain_only_ExtRm.tif` whatever its own name.

> **On-disk naming for every Cellpose-SAM stage** is cumulative and self-documenting — each stage's filename is the previous one with one more suffix appended, so reading it left to right tells you exactly which processing steps ran:
>
> | File | Stage |
> |---|---|
> | `<stem>_cp.tif` | raw `do_3D` output, pre-merge |
> | `<stem>_cp_krendl.tif` | + Krendl safe-merge + large-contact merge (always saved by a normal run) |
> | `<stem>_cp_krendl_ac.tif` | + auto-correct (only if that stage was left enabled) |
> | `<stem>_cp_krendl_ac_snd.tif` | + sanding (only if that stage was left enabled too) |

10g. Verify Smooth σ XY / σ Z (GT Sweep)

Checks the Pixel Classifier's pre-threshold Gaussian smoothing (**Smooth σ XY** / **Smooth σ Z**, Tab 2 — defaults 2.6/1.0) against real ground truth, the same way every other GT-sweep tool here checks its own parameter(s).

1. **GT image** / **brain_mask.tif** / **GT labels** — same three inputs as 10b above (the raw or `_ExtRm` image, the raw un-eroded mask, and the corrected GT label volume), plus **Z / XY (µm)** (default 1.0 / 0.174).
2. **sigma XY min/max/step** (default 0.5–2.5 step 0.5) and **sigma Z min/max/step** (default 1.0–5.0 step 1.0) — define the grid.
3. **Sieve** (default on) — auto-narrows sigma XY coarse → fine across 3 stages (defaults ±0.5 step 0.1, then ±0.2 step 0.05), same idea as the other sweepers; sigma Z's own grid stays fixed at every stage.
4. **Complex cells to test** (5) and **Pad Z** / **Pad XY** (15 / 40), as in 10b.
5. Click **Run Sigma Sweep**. **BG Threshold and Signal Erosion are held fixed** at whatever Tab 1's sliders currently show — this sweep isolates sigma specifically.

Cheaper per grid point than the BG Threshold/Signal Erosion sweep: since BG Threshold and Signal Erosion don't change here, each cell's thresholded `brain_only` crop is computed once and reused across every sigma combination — only the `create_labels()` call itself (smoothing + 3D connected components) varies per grid point.

Same apply and floor-recalibration behavior as 10b (GT Toolkit, GT verified): this fish's best (sigma XY, sigma Z) point is folded into a running average across every fish swept so far and that average is applied to Tab 2's Smooth σ XY/Z sliders, and both Min volume and Min hole size are recalibrated as the same never-rising floors described there.

10h. Email notification (optional)

Not a sweep — a General-category utility, alongside Score Against GT, since it isn't tied to one pipeline. This panel holds **the plugin's one email switch** and the SMTP settings behind it. There are no per-tool email checkboxes:

- **Email me when a job finishes or fails** (default: **on**) — any job the plugin runs in the background (Run Skin-Remover, Create Labels, Cellpose-SAM Segmentation, every Edit MG Labels correction, Generate Statistics, every GT sweep and calibration, Build GT-Correction Package, patch extraction, …) sends one email when it ends, success or failure.
- **only if it ran longer than** (default: **10 min**) — jobs shorter than this never email, so quick edits stay silent. `0` emails after every job.

The email's subject says the job and whether it **finished** or **FAILED**; the body gives start/end time, duration, and the job's own result or error message. A tool that chains stages (Cellpose-SAM Segmentation → auto-correct → sanding, or Auto-correct Labels → sanding) sends **one** email for the whole chain, timed from the first click. GT Toolkit Tuning sends one email at the end, not one per step. **Training runs (Tab 5)** always email when they stop while the switch is on — no threshold, since they run for hours — even if napari is closed by then (see below).

Nothing breaks without email set up. With **Notify email** blank (the default) nothing is sent. If it is filled in but the SMTP username/password is missing, or the server can't be reached, the job still runs

and finishes normally; the plugin only prints a one-line note to the console. A training launch in that state starts anyway and says “no email will be sent” in its status line.

Fields: Notify email, **SMTP server/port**, **SMTP username**, **SMTP password**. Leave **Notify email** blank to send nothing — that’s the default. Free with any Gmail account, no other signup needed. **See Section 13a for the full step-by-step setup** (turning on 2-Step Verification, generating a Google App Password, and what to type into each field) — the short version: server `smtp.gmail.com`, port `465`, username = your Gmail address, password = a Google App Password, not your normal Gmail password, which won’t work here.

All fields are saved between napari sessions (each one when you finish editing it), along with the switch and the threshold — set this up once. The **password specifically is saved encrypted in your OS’s credential store** (Windows Credential Manager / macOS Keychain / Linux Secret Service, via the `keyring` package) rather than the plugin’s own plaintext `config.json` — real encryption at rest with an OS-managed key, not something the plugin manages itself. This is reasonable to persist at all because a Gmail App Password is a separate, revocable credential Google issues specifically for unattended third-party use like this, not your real account password.

On Linux, the OS credential store needs a running, *unlocked* Secret Service session (GNOME Keyring or KWallet) — present on a normal desktop login, but not guaranteed over SSH, on a headless machine, or before you’ve logged into the desktop once. This is common enough on Linux workstations used mainly over SSH/tmux (confirmed directly on this project’s own machine) that the plugin has a second layer for it: if the OS store isn’t reachable, the password is saved instead to a local file encrypted with Fernet/AES (via `cryptography`), keyed by a machine-local key file the plugin generates once — no password prompt, no plaintext on disk, works the same way every session. This is a **weaker guarantee than the OS store**, worth being honest about: the key protecting that file lives right next to it, on the same machine and account, so it defends against casual exposure (an accidental `cat config.json`-style leak, a stray backup of just one file, a screen-share) rather than someone with full read access to your home directory. You’ll see a one-line note printed to the console the first time this fallback kicks in (“OS credential store unavailable – saved to a local encrypted file instead”). Only if *both* the OS store and this fallback fail

(e.g. the config directory itself is unwritable) does the plugin give up and not persist the value at all. Windows and macOS almost always have a working OS-level backend, so this mostly matters on Linux.

Send Test Email — verifies your SMTP settings actually work without waiting on a real long job. Fill in the fields above, then click it: sends one email immediately with a fixed test subject/body, using the same SMTP settings and send call every job email uses. Runs in a background thread (a misconfigured host/port can hang for up to the 30-second SMTP timeout rather than fail instantly) so napari stays responsive while it tries. Reports "Sent to … — check your inbox" or the specific SMTP error (wrong password, unreachable host, etc.) right there — the fastest way to confirm setup before relying on it for a long run.

Why Tab 5's training notifications still work even if napari is closed the whole time: unlike every other job (which runs in a background thread inside napari itself, so it needs napari to stay open the whole time regardless of email), a launched training run's notification isn't sent by the GUI's live polling — instead, when the switch is on and SMTP is set up at launch time, the launched background process itself is a small wrapper that runs the real training command, waits for it to finish, *then* sends the email, before exiting. That wrapper is what's detached and survives napari closing, exactly like the training script itself, so the email still arrives on schedule whether or not you ever reopen napari to see it. Clicking **Stop Training** kills the whole thing (wrapper included) before it reaches the email step, so a manual stop doesn't send one — only unattended completions/crashes do.

10i. Calibrate Correct-Label Contrast

Finds the lower-contrast value Correct Label (Tab 3 — Edit MG Labels) should start from — automatically, instead of by dragging the contrast slider until a silhouette "looks right" by eye. Unlike every other Tab 6 sweep, this one is **not** checked against independent ground truth — it finds whichever value best **reproduces what the active Labels layer already segmented**, since that's the actual question Correct Label needs answered before a user nudges it further for one specific problem cell.

This is the canonical best_lo sweep — every other autosweep-driven tool correlates to it. Protect Skin as Label, Auto-correct Labels, and the Cellpose-SAM Segmentation route's own chained auto-correct

stage all run this exact same sweep (not a second, independently-tunable copy, and with this tool's own Bbox padding for the sample crops) whenever their own **Contrast low (best_lo) calibration** section (Tab 3 — Edit MG Labels, 7d) has Auto-sweep checked. Changing **Cells**, **Slices/cell**, **Edge margin (µm)**, or **Sweep steps** here changes the sweep every one of those tools runs, not just this standalone button — this is the one place to tune how many cells/slices the sweep samples.

How it works: picks the **Cells** most morphologically complex cells (same skeleton-branch-count "Complexity" measure Resort Labels uses) whose centroid sits at least **Edge margin (µm)** away from the volume's own outer boundary — a proxy for "not close to skin", since a cell right at the boundary is exactly the one most likely to already carry a skin-residue artifact merged in (the thing this calibration should be scored *against*, not accidentally learn from). For each selected cell it samples up to **Slices/cell** Z-slices (default 20 cells × 20 slices = **400 samples**, not 20 total), spread across the middle of that cell's own Z-extent. It then sweeps **Sweep steps** candidate lower-contrast values, evenly spaced between the 1st and 99.5th percentile of intensity inside the samples' own boxes (each sample's bounding box on its slice, grown by **Bbox padding (px)**), then re-tests every whole-number intensity value within one step either side of the best one (a tenth of a step apart for a non-integer image), and keeps whichever value best reproduces the most existing 2D footprints, jointly across all samples (mean IoU) — not each sample's own independent best, which would let one outlier pull the result around.

Only cell labels count, both as targets and as context: any skin label (-1) present in the layer is treated as plain background during calibration, so the result is the same whether or not skin has been protected.

Signal layer / **Labels layer** — read from Tab 3 — Edit MG Labels' shared "Layers and label(s) being edited" selector (7a), not a separate pair of combos here — pick them once there, and every tool that needs them (this one included) uses the same choice.

Cells / **Slices/cell** — default 20 / 20 (up to 400 samples; ranges 1–20 / 1–30). **Edge margin (µm)** — default 50.0 (each axis's margin is capped at 30% of that axis's extent). **Sweep steps** — default 20 candidate values for the coarse pass (range 5–200); the fine pass around the winner adds about a dozen more. **Bbox padding (px)** — default 15, the sample-crop padding for this sweep and for every sweep that correlates to it

(Protect Skin as Label, Auto-correct Labels, the chained Cellpose-SAM stage) — not Correction Settings' Bbox padding, which is the per-cell correction padding.

Click **Run Contrast Calibration Sweep**. On success, the report below shows every candidate tried and its mean IoU, and the winning value is **applied directly** to the selected signal layer's contrast limits — low set to the winning value, high set adaptively from this fish's own real signal (see 7d) — no manual step needed afterward.

10j. Drift View in 3D

A General-category utility with nothing to do with any GT sweep or pipeline — a small quality-of-life extra for looking at a 3D result hands-free. Click **Start Drift** and the camera slowly, continuously tumbles around whatever's currently in the viewer, like drifting past it in space; switches the viewer to 3D display automatically if it wasn't already. **Speed** (1–100, default 20) scales how fast it turns — adjust it at any time, even while already drifting, no need to stop first. Click the same button (now labeled **Stop Drift**) to freeze the camera exactly where it is, at whatever orientation it's currently at.

Rotates freely on all three axes (pitch, roll, and yaw) at once, never stalling on any of them. Each tick composes a real 3D rotation (`scipy`'s own `Rotation`, the same class napari's own camera model is itself built on) rather than incrementing napari's three Euler angles independently — napari's own `camera.angles` is an `'xyz'`-order Euler decomposition, which mathematically restricts its own MIDDLE angle to ±90° (the other two get a full 360° range), so composing a true 3D rotation each tick is what lets the camera keep advancing indefinitely on every axis at once.

Purely a camera effect — it only ever changes `viewer.camera.angles`, never touches any layer's own data, so it can't interact with, be interrupted by, or leave any trace on a correction tool's own result. Each of the three rotation axes advances at a slightly different rate rather than a shared one, so the motion reads as an organic drift rather than a mechanical single-axis spin that visibly repeats on a short cycle.

10k. Autosave Labels Layer

A General-category safety net for manual label editing (Correct Label, Correct Adjacent Labels, Split/Join Labels, etc.), independent of anything else in the plugin — a crash loses at most one interval's worth of work, not the whole session.

Autosave the Labels layer being edited — checkbox, **default on**. While checked, periodically saves the one Labels layer currently picked in Tab 3's own shared Layers and label(s) being edited selector to `<output folder>/<layer name>_recovery.tif`, overwriting the previous recovery save each time. Unchecking it disables autosaving entirely; it also does nothing if no Labels layer is currently selected there.

Every: — interval spinbox + **Minutes**/**Hours** unit dropdown, **default 30 minutes**. Changing either takes effect immediately, not on the next already-scheduled save.

Scoped to only that one layer — no other Labels layer (a `brain_mask` layer included) is ever saved by this tool.

10l. GT Toolkit Tuning Tool

The first tool in this tab (General category), and the **only** place in the plugin that applies GT-sweep results to the shared values. Point it at one fish and it tunes the whole plugin against that fish's ground truth in one run, reusing every individual sweep below unchanged.

1. **Fish original file or folder** — the raw source file (its sibling folder `<parent>/<stem>/` is found automatically) or the fish folder itself; **File…** / **Folder…** browse for either. The path is remembered.
2. **Scan Folder** — looks for this fish's files by the `<stem>_<artifact>` convention (`_original.tif`, `_brain_mask.tif`, `_brain_polygons_interpolated.npz`, `_brain_only_ExtRm/NoBG/RndFill.tif`, `_cp.tif`, `_cp_krendl*.tif`, `_GROUND_TRUTH.tif`, `_statistics.csv`, `_autocorrect_report.txt`), listing each as found/missing. A file is found if it's named exactly `<stem><suffix>`, or if it's the only file in the folder with that suffix (for the auto-correct report: the newest one). It then fills the path fields of every sweep below, of Calibrate branch_radius (Tab 5) and of Build GT-Correction Package. Safe to run any number of times.

3. **This is verified ground truth** — off by default; un-ticks itself when the run finishes. Only when ticked does the run change anything: every step still runs and reports without it.
4. **Run Full GT Toolkit Tuning** — scans again, then:
   - **Min/Max volume** + **Min hole size** — measured instantly from `<stem>_GROUND_TRUTH.tif` (smallest cell, largest cell, smallest real hole) and, when GT-verified, folded into the never-rising Min volume / Min hole size floors and the never-falling Max volume ceiling (Common Settings, 6a; the Statistics outlier bounds).
   - then each of these steps, in order, whose files were found — each one run with its hidden "This is verified ground truth" box set to match yours: **MONAI Threshold/Erosion** (10a; needs `_original.tif` + a hand-corrected `_brain_mask.tif`, see below), **BG Threshold/Signal Erosion** (10b; needs a raw or `_ExtRm` image + `_brain_mask.tif` + `_GROUND_TRUTH.tif`), **Smooth σ XY/Z** (10g; the same, plus `_brain_only_NoBG.tif`), **Cellprob** + **Krendl** (10c; needs `_brain_only_ExtRm.tif` + `_GROUND_TRUTH.tif`) and **branch_radius** (Calibrate branch_radius; needs `_GROUND_TRUTH.tif`). Skipped steps and the reason are listed in the report.
5. **Stop** — cancels the step that is running (between its grid points) and skips the rest of the queue.

Each step uses the ranges, Sieve settings and cell counts currently set in its own section. Run Skin-Remover writes `_original.tif` and `_brain_mask.tif` too, so the MONAI step needs proof that the brain mask is GT Annotation's hand-drawn one: `<stem>_brain_polygons_interpolated.npz` (written by GT Annotation's Generate Masks in the same call) must sit beside it with the same stem, and the mask must not have been written more than 5 minutes after it (a later write means something, e.g. Run Skin-Remover, overwrote it). Otherwise the step is skipped with that reason in the report. Scan Folder lists the polygons file as `brain_polygons`. The report box collects every step's final status line. One email is sent for the whole run (if it ran past the email threshold), not one per step.

---

11. Output files and folder structure

All files saved by the plugin go into a dedicated folder named after the most recently opened input file (`<file's folder>/<file stem>/`). File names inside start with the **layer** name, which for a multi-channel file includes the channel (e.g. `NT54_ch1`):

```
/path/to/your/data/
├── NT54.ims                                          ← original
input file (channels → layers NT54_ch0, NT54_ch1, …)
└── NT54/                                             ← output
folder (created automatically)
    ├── NT54_ch1_brain_mask.tif                       ← binary brain
mask (0/255, uint8)
    ├── NT54_ch1_brain_only_ExtRm.tif                 ← brain only
(suffix = background mode, below)
    ├── NT54_ch1_original.tif                         ← raw input
channel (whenever Background mode ≠ Off)
    ├── NT54_ch1_brain_only_ExtRm_cp.tif              ← Cellpose-SAM
raw prediction
    ├── NT54_ch1_brain_only_ExtRm_cp_krendl.tif       ← Cellpose-SAM
final segmentation
    ├── NT54_ch1_brain_only_ExtRm_cp_krendl_ac.tif    ← + auto-
correct (if enabled)
    ├── NT54_ch1_brain_only_ExtRm_cp_krendl_ac_snd.tif ← + sanding
(if enabled)
    ├── NT54_ch1_brain_only_ExtRm_autocorrect_report.txt ← auto-
correct (+ sanding) report
    ├── <labels layer>.tif                            ← Save Labels
(default name)
    ├── <labels layer>_recovery.tif                   ← Autosave
Labels Layer
    └── NT54_statistics.csv                           ← per-label
statistics (named after the file)
```

The folder is created the first time a file is saved. If no file was opened with **Open TIF / IMS file**, the plugin uses the file behind any layer napari opened itself (File → Open, drag-and-drop); if there is none, files are saved in the current working directory (with a console warning).

Brain-only suffixes depending on background mode:

Mode	Suffix
Off	*(none)*
1 — Exterior Removed	`_ExtRm`
2 — No Background	`_NoBG`
3 — Random Fill	`_RndFill`

---

12. Statistics CSV — all columns explained

The CSV produced by Generate Statistics has one row per label, with up to 52 columns: 47 are always computed, 3 more need an Image layer and 2 more a brain-region Shapes layer. Which ones actually land in the CSV is chosen in the tab's **Output columns** checklist — several start unticked. This section explains what each column *means*; for the algorithm/formula/library behind each one, see the separate STATISTICS_GUIDE.md.

Identification

Column	Type	Description
`label`	integer	Label number matching the napari Labels layer (1, 2, 3, …)

Volume

Column	Type	Description
`volume_vox`	integer	Number of voxels belonging to this label
`volume_um3`	float (µm³)	Physical volume in cubic micrometres. Computed as `volume_vox × Z_size × Y_size × X_size`. A typical zebrafish microglia is 1,000–10,000 µm³.
`is_volume_outlier`	boolean	`True` if `volume_vox` falls outside the two GT-tracked bounds — bigger than the largest cell any GT fish has ever confirmed real, or smaller than the smallest one (Common Settings' Min volume floor). Both bounds are measured only by the GT Toolkit Tuning Tool (10l); a bound that hasn't been measured yet flags nothing. Flags for human review; never deletes anything.

Position (centroid)

The centroid is the 3D centre of mass of the label — the average position of all its voxels.

Column	Type	Description
`centroid_z_vox`	float	Z position in voxel units
`centroid_y_vox`	float	Y position in voxel units
`centroid_x_vox`	float	X position in voxel units
`centroid_z_um`	float (µm)	Z position in micrometres
`centroid_y_um`	float (µm)	Y position in micrometres
`centroid_x_um`	float (µm)	X position in micrometres

Bounding box

The smallest rectangular box (aligned with the axes) that completely contains the label.

Column	Type	Description
`bbox_z0_vox`, `bbox_y0_vox`, `bbox_x0_vox`	integer	Start corner of the box, in voxels (inclusive)
`bbox_z1_vox`, `bbox_y1_vox`, `bbox_x1_vox`	integer	End corner of the box, in voxels (exclusive)
`bbox_dz_um`	float (µm)	Height of the bounding box in Z (depth of the cell in the axial direction)
`bbox_dy_um`	float (µm)	Height of the bounding box in Y
`bbox_dx_um`	float (µm)	Width of the bounding box in X

Size and shape

Column	Type	Description
`eq_diam_um`	float (µm)	**Equivalent sphere diameter** — the diameter of a perfect sphere with the same volume as this label. Formula: `(6V/`

Column	Type	Description
		π)^(1/3). Useful as a single "size" number regardless of shape.
axis1_um	float (µm)	**Longest principal axis** — the maximum extent of the label along its longest geometric direction. Derived from the inertia tensor eigenvectors.
axis2_um	float (µm)	**Middle principal axis** — approximated as the average of axis1 and axis3.
axis3_um	float (µm)	**Shortest principal axis** — the minimum extent perpendicular to the longest axis.
elongation	float	**Elongation ratio** = axis1 / axis3. A perfect sphere = 1.0. A cigar-shaped cell = 3.0 or more. The higher the number, the more stretched out the cell is.
principal_axis_dir	string	The anatomical direction of the longest axis: "Z" (axial), "Y" (coronal), or "X" (sagittal). Tells you which direction the cell is elongated in.

Surface and compactness

Column	Type	Description
solidity	float (0–1)	**Solidity** = volume / convex_hull_volume. The convex hull is the smallest convex shape enclosing the label (like shrink-wrap). A solid, convex cell = 1.0. A lobulated or branchy cell with lots of indentations < 1.0. Typical range for microglia: 0.5–0.9.
extent	float (0–1)	

Column	Type	Description
		Extent = `volume / bounding_box_volume`. How much of the bounding box is actually filled. A cube = 1.0. A sphere ≈ 0.52. Highly branched cells = much lower.
`surface_area_um2`	float (μm²)	**Surface area** in square micrometres, computed using marching cubes — a 3D mesh is generated from the label boundary and the triangle areas summed. A cell with long thin branches has a much larger surface area than a smooth sphere of the same volume.
`sphericity`	float (0–1)	**Sphericity** = `π^(1/3) × (6V)^(2/3) / A` where V = volume and A = surface area. A perfect sphere = 1.0. Anything less than 1.0 is less spherical. Microglia: typically 0.3–0.8 depending on branch complexity.
`surface_to_volume_ratio`	float (μm⁻¹)	**Surface-to-volume ratio** = surface_area / volume. Higher values indicate more complex, surface-rich morphology relative to cell size. Branches and protrusions increase this dramatically.

Skeleton (branching structure)

These columns require the `skan` package. If `skan` is not installed, they will be 0.

The algorithm skeletonizes the label (reduces it to a 1-voxel-wide skeleton) and analyses the resulting graph of branches.

Column	Type	Description
`n_branches`	integer	Number of skeleton branches. A sphere = 1 branch. A microglia with 4 protrusions = roughly 4–8 branches depending on how they connect.
`n_endpoints`	integer	Number of free-end branch tips (branches that don't loop back). Corresponds roughly to the number of protrusion tips.
`mean_branch_len_um`	float (µm)	Average straight-line (end-to-end) length of all skeleton branches in micrometres.
`max_branch_len_um`	float (µm)	Straight-line length of the longest individual branch — an indicator of maximum protrusion reach.
`branch_tortuosity`	float (≥1)	Average ratio of path length to straight-line distance per branch. A value of 1.0 = perfectly straight branches. Higher values = winding, curved protrusions.
`branch_density`	float (per 10^6 μm^3)	Number of branches per million cubic micrometres of cell volume. Allows fair comparison between cells of different sizes.
`endpoint_density`	float (per 10^6 μm^3)	Number of branch tips per million cubic micrometres. A proxy for protrusion count normalised by cell volume.
`process_complexity`	float	Combined measure of branching complexity: `n_endpoints × mean_branch_len_um / volume_um3`. High values = many

Column	Type	Description
		long branches relative to cell volume.

Morphotype classification

Column	Type	Description
`morphotype`	string	Automatic shape classification based on elongation, sphericity, solidity, branch count, and surface-to-volume ratio. Categories: **Rod-shaped** (elongated, few branches), **Amoeboid** (round, compact, few branches), **Ramified** (many long branches, low sphericity), **Intermediate-ramified** (moderate branching), **Intermediate** (doesn't fit the above).

Spatial relationships

These columns use all cell centroids together to compute neighbourhood statistics.

Column	Type	Description
`nearest_neighbor_label`	integer	Label ID of the closest other cell (by centroid).
`nearest_neighbor_dist_um`	float (µm)	Distance to the closest other cell centroid. Small values = cells are tightly packed; large values = isolated cells.
`nearest_neighbor_2_label`, `nearest_neighbor_2_dist_um`	integer, float (µm)	The same for the second-closest cell (equal to the first if only two cells exist).
`nearest_neighbor_ratio`	float	

Column	Type	Description
		Clark-Evans 3D index for this cell: the ratio of its nearest-neighbour distance to the expected distance if cells were randomly distributed at the same density. Values < 1 = clustering; > 1 = regularity/dispersion.
`local_density_100um`	integer	Number of other cell centroids within a 100 µm radius sphere (a plain count, not normalised). A measure of local neighbourhood crowding.
`depth_normalized`	float (0–1)	Z position normalised to the full depth range of all cells: 0 = shallowest cell, 1 = deepest. Useful for comparing dorsal vs. ventral distribution across samples.

Intensity statistics *(optional — requires Image layer selection)*

Column	Type	Description
`mean_intensity`	float	Mean pixel intensity inside the label mask. Reflects overall fluorescence brightness of the cell.
`integrated_intensity`	float	Sum of all pixel values inside the label (mean × voxel count). Proportional to total fluorescent material in the cell regardless of size.
`intensity_cv`	float (0–∞)	Coefficient of variation of pixel intensities = std / mean. 0 = perfectly uniform. High

Column	Type	Description
		values = heterogeneous staining, possibly indicating internal structure or imaging artefacts.

Brain region assignment *(optional — requires Shapes layer with boundary lines)*

Column	Type	Description
`brain_region`	string	Name of the anatomical region this cell belongs to (as defined by the boundary lines and region names you provided).
`region_boundary_dist_um`	float (µm)	Distance from this cell's centroid to the nearest region boundary line, in micrometres. Cells near boundaries may have mixed characteristics.

Description

Column	Type	Description
`description`	string	A plain-language sentence summarising the cell's shape, generated by the selected description backend. Example (rule-based): *"Label 3: Elongated along Y-axis (2.8:1), volume 4,521 µm³, centroid Z=87.3 Y=142.1 X=203.5 µm. Lobulated/ irregular surface, sphericity 0.41, solidity 0.72. 6 branches, 4 endpoints (mean 8.3 µm). Morphotype: Intermediate-ramified."*

13. Setting up description backends

Rule-based (offline) — no setup needed

The default. Descriptions are generated using built-in templates based on the numeric values. No internet connection, no API key, no external software. Always available.

Ollama (local, free)

Ollama runs a large language model locally on your machine. No data is sent to external servers, and there is no ongoing cost after the initial download.

Step 1 — Install Ollama

Go to https://ollama.com/download and download the installer for your operating system. Run it.

- On Linux: `curl -fsSL https://ollama.com/install.sh | sh`
- On Mac: Download the `.dmg` and drag to Applications.
- On Windows: Download the `.exe` installer.

Step 2 — Download a model

Open a terminal and run:

```
ollama pull llama3
```

This downloads the Llama 3 model (~4.7 GB). You only need to do this once. Other models you can use:

```
ollama pull mistral      # ~4 GB, fast
ollama pull phi3         # ~2 GB, smaller and faster
ollama pull llama3:70b   # ~40 GB, highest quality — needs 64 GB+
        RAM
```

Step 3 — Verify Ollama is running

Ollama starts automatically in the background after installation. You can confirm it is running:

```
ollama list   # should show your downloaded models
```

Step 4 — Configure in the plugin

In **Tab 4 — Statistics**:

1. Select **Ollama (local, free)** from the Description dropdown.
2. **Endpoint:** leave as `http://localhost:11434` (default). Only change this if Ollama runs on a different machine on your network.
3. **Model:** type the model name you downloaded, e.g. `llama3`.
4. Click **Generate Statistics**.

> If you get an `[Ollama error: ...]` in the CSV description column, check that Ollama is running (`ollama list`) and that the model name matches exactly what you downloaded.

OpenAI API (paid)

OpenAI's GPT models run on OpenAI's servers. You pay per token processed. For statistics descriptions (short prompts, short responses), the cost is very low — roughly $0.001–0.01 per 100 cells with `gpt-4o-mini`.

Step 1 — Create an OpenAI account

Go to https://platform.openai.com and sign up. You will need to provide a credit card for billing.

Step 2 — Generate an API key

1. Log in to https://platform.openai.com.
2. Click your profile icon (top right) → **API keys**.
3. Click + **Create new secret key**.
4. Give it a name (e.g. "napari-zf-microglia-ai").
5. Copy the key immediately — it starts with `sk-` and you can only see it once.

Step 3 — Configure in the plugin

In **Tab 4 — Statistics**:

1. Select **OpenAI API (paid)** from the Description dropdown.
2. **API Key:** paste your `sk-...` key.
3. **Model:** `gpt-4o-mini` (recommended — low cost, good quality). Other options:
   - `gpt-4o` — highest quality, higher cost
   - `gpt-3.5-turbo` — fastest, cheapest, lower quality
4. **Base URL:** leave blank unless you use an OpenAI-compatible proxy.
5. Click **Generate Statistics**.

> The API key is **saved encrypted in your OS's credential store** (Windows Credential Manager / macOS Keychain / Linux Secret Service, via the `keyring` package — not the plugin's own plaintext `config.json`) and prefilled next session, so you only need to paste it once. This key is tied directly to your OpenAI billing, with no separate "app-scoped" variant the way a Gmail App Password is — worth being deliberate about, and revoking/regenerating it from OpenAI's dashboard if you're ever unsure. On Linux specifically, the OS store needs an unlocked Secret Service session (GNOME Keyring/KWallet) — if none is available (e.g. an SSH-only session, or before your first desktop login), the plugin automatically falls back to a local Fernet-encrypted file instead of writing the key in plaintext — see Section 10h for exactly what that means and its weaker guarantee compared to the OS store.

Claude API (paid)

Anthropic's Claude models. Similar pricing model to OpenAI. Claude Haiku is very fast and inexpensive.

Step 1 — Create an Anthropic account

Go to https://console.anthropic.com and sign up with a credit card.

Step 2 — Generate an API key

1. Log in to https://console.anthropic.com.
2. Click **API Keys** in the left sidebar.
3. Click + **Create Key**.
4. Give it a name and copy the key (starts with `sk-ant-`).

Step 3 — Configure in the plugin

In **Tab 4 — Statistics**:

1. Select **Claude API (paid)** from the Description dropdown.
2. **API Key:** paste your `sk-ant-...` key.
3. **Model:** `claude-haiku-4-5-20251001` (recommended — fast and cheap). Other options:
   - `claude-sonnet-4-6` — higher quality, moderate cost
   - `claude-opus-4-6` — highest quality, highest cost
4. **Base URL:** leave blank (not used for Claude).
5. Click **Generate Statistics**.

13a. Setting up email notification (Gmail App Password)

The **Email notification** panel in **Tab 6 — Sweeps & Utilities** (Section 10h) holds the plugin's one email switch and the SMTP credentials behind it. It works with any SMTP-over-SSL provider, but Gmail is the easiest and free path — this walks through it end to end. If you'd rather use a different provider (Outlook/Office365, a work email server, etc.), skip to **Using a non-Gmail provider** at the bottom.

Step 1 — Turn on 2-Step Verification (if not already on)

App Passwords only exist for Google accounts with 2-Step Verification enabled — this is a Google requirement, not something the plugin asks for.

1. Go to https://myaccount.google.com/security.
2. Under "How you sign in to Google," click **2-Step Verification**.
3. Follow the prompts to turn it on (usually a phone number + a code sent by SMS or the Google Authenticator app).

If it's already on, skip straight to Step 2.

Step 2 — Generate an App Password

1. Go to https://myaccount.google.com/apppasswords (you may be asked to sign in again).
2. Under "App name," type something recognizable, e.g. `napari-zf-microglia-ai`.
3. Click **Create**.
4. Google shows a 16-character password (four groups of four letters, e.g. `abcd efgh ijkl mnop`). Copy it now — this is the only time it's shown. Spaces don't matter; you can paste it with or without them.

This is **not your normal Gmail password** and won't be accepted as one — Google deliberately issues a separate, revocable password for exactly this kind of use (a third-party app sending mail on your behalf). You can revoke it any time from the same App Passwords page without affecting your main account password.

Step 3 — Configure in the plugin

In **Tab 6 — Sweeps & Utilities**, General category, the **Email notification (optional)** panel:

1. **Notify email:** the address that should receive the notification — typically your own Gmail address, but it can be any address you want the report sent to.

2. **SMTP server:** leave as `smtp.gmail.com` (the default).
3. **port:** leave as `465` (the default).
4. **SMTP username:** your full Gmail address (e.g. `you@gmail.com`).
5. **SMTP password:** paste the 16-character App Password from Step 2 — *not* your normal Gmail password.
6. Leave **Email me when a job finishes or fails** ticked (the default) and set **only if it ran longer than** (default 10 min). From then on, every job that runs past the threshold emails when it finishes or fails, and every training run emails when it stops — see Section 10h.

> **All fields are saved between sessions; the password specifically is saved encrypted in your OS's credential store** (Windows Credential Manager / macOS Keychain / Linux Secret Service via `keyring`), not the plugin's own plaintext `config.json` — set this up once, no re-entering per session. An App Password is a separate, revocable credential Google issues specifically for this kind of unattended use, not your real account password. (The Statistics tab's OpenAI/Claude API keys, Section 13, use the same encrypted storage now, but those are billing-linked with no equivalent "app-scoped" variant, so they're a somewhat different risk — see the note there.) On Linux, the OS store needs an unlocked Secret Service session (GNOME Keyring/KWallet); without one, the password is saved to a local encrypted file instead — see Section 10h for that fallback in detail.

Using a non-Gmail provider

Any SMTP server that supports SSL on a fixed port works the same way — just change **SMTP server**/**port** to match your provider and use whatever credentials it issues (an app-specific password if the provider offers one, same reasoning as Gmail's). A few examples:

Provider	SMTP server	Port
Gmail	`smtp.gmail.com`	465
Outlook / Office 365 (personal)	`smtp.office365.com`	587 *(see note below)*
Yahoo Mail	`smtp.mail.yahoo.com`	465

> Note: the plugin's supervisor script always connects via `SMTP_SSL` (implicit TLS from the first byte, no STARTTLS handshake) — this matches Gmail and Yahoo's port-465 behavior. Providers that

> only offer STARTTLS on port 587 (like Office 365) are not currently supported without a small code change; Gmail is the tested, recommended path.

---

14. Full workflow: from raw stack to labelled cells

Step 1 — Open your file

1. Open the plugin (Plugins → ZF-Microglia-ToolKit (ZF-Microglia-AI)) in napari.
2. Click **Open TIF / IMS file** and select your confocal stack.
3. All channels appear as layers.
4. **Click the microglia channel** (usually ch1, green) in the Layers panel.

Step 2 — Run skin removal

Set these values in Tab 1:

Setting	Value
MONAI Threshold	**0.38** (default)
MONAI Erosion	0 (default)
Background	**Option 1** if you plan to use Cellpose-SAM in Step 3, **Option 2** if you plan to use the Pixel Classifier
BG Threshold (Option 2 only)	**1.025** (default)

Click **Run Skin-Remover** and wait.

What you should see: A `brain_only` layer. With Option 2, microglia appear as bright isolated blobs on a black background, with clear space between cells (needed for the Pixel Classifier). With Option 1, the brain interior is left intact — only the tissue outside the brain is removed (needed for Cellpose-SAM).

If blobs look hollow or have large halos (Option 2): Lower BG Threshold (e.g. 0.40).

If too much dim signal remains between cells (Option 2): Raise BG Threshold (e.g. 0.80).

Step 3 — Create labels

Switch to the **Create MG Labels** tab. It automatically shows the section matching the `brain_only` layer Tab 1 just produced (`_ExtRm` or `_NoBG`) — see Section 6a for the full logic.

Option A — Pixel Classifier (a `_NoBG` layer is open)

Select the `_NoBG` layer in the Layers panel, then check these values:

Setting	Value
Smooth σ XY	**2.6** (default)
Smooth σ Z	**1.0** (default)
Min volume	7069 (read-only; lowered only by the GT Toolkit Tuning Tool)
Min hole size	5 (default)
Final min-size fraction	0.618 (default)

Click **Create Labels**.

What you should see: A labels layer where each cell is a different colour. The status line reports how many were found.

Tuning: - Too many tiny fragments → raise Final min-size fraction (toward 1.0) or increase both σ values - Two cells merged together → try Split Label (see below) - Cells cut across slices → increase σ Z

Option B — Cellpose-SAM Segmentation (an `_ExtRm` layer is open)

1. Browse to your Cellpose-SAM checkpoint (Section 3) if not already set.
2. Check the **Signal layer** field shows your `_ExtRm` layer.
3. The parameters default to Cellprob 2.84, Flow iterations 200, Safe-merge max gap 3.52 µm, min contact 1 vox, Large-contact merge 20 vox, Min size 15 vox — see Section 6c for what each one does.
4. With **Auto-correct** on (the default), keep the raw channel layer (e.g. `NT54_ch1`) open and check the **Brain mask layer** field.

5. Click **Run Cellpose-SAM Segmentation** and wait — this can take hours for a full-size fish. Progress is shown in the status line and log box; napari stays usable while it runs.

What you should see: A `<layer>_cellpose_labels` layer where each cell is a different colour — after auto-correct, with skin as label -1.

Step 4 — Review and edit labels in napari

- Toggle the labels layer on/off to compare with the original
- Hover over cells to see their label number
- Zoom through Z slices to verify cells are correctly separated

---

Step 5 — Split merged cells (if needed)

If two cells were labelled as one because they touch:

1. Switch to **Tab 3 — Edit MG Labels**. Pick the Labels layer in the shared selector at the top (7a) if it isn't already.
2. Hover over the merged blob and note its label number (shown in the napari status bar at the bottom).
3. Click the blob, then click **Label A**'s own **Use selected** button (or type the label number directly).
4. Under **Split Label**:
   - **Split into:** 2 (or however many cells are merged).
   - **Smooth σ:** 1.0 (default).
   - **Min distance:** 5 (default).
5. Click **Split Label**.
6. The two (or more) cells are separated at their thinnest connection point.

Step 6 — Sort labels (optional)

In **Tab 3 — Edit MG Labels**, with the Labels layer picked in the shared selector, click **Resort Labels** to renumber cells by size, position or complexity. This is helpful for consistent reporting:

- By **Size** (largest = label 1) — most common
- By **Centroid Z/Y/X** — for atlas alignment
- By **Complexity** (most branched = label 1)

Step 7 — Save labels

In **Tab 3 — Edit MG Labels**, click **Save Labels**. A file dialog opens pre-filled with the output folder. Accept or change the name and click Save.

Step 8 — Generate statistics

1. Click the **Statistics** tab (Tab 4).
2. Make sure the Labels layer is selected in napari's layer list (otherwise the topmost Labels layer is used).
3. Choose your description backend.
4. *(Optional)* Select a fluorescence channel under **Intensity statistics** to add mean/integrated/CV columns.
5. *(Optional — see Step 8a below)* Draw region boundary lines in a Shapes layer, then select it under **Brain regions** and enter the region names (e.g. `Optic tectum, Hindbrain`).
6. Click **Generate Statistics**.
7. The CSV is saved automatically to the output folder.

Step 8a — Assign cells to brain regions (optional)

This lets you label each cell as belonging to the **optic tectum**, the **hindbrain**, or any other anatomical region you define by drawing a dividing line across the image.

Orientation reminder: The fish lies along the **X axis** — head at X = 0, tail at X = max. Y runs top to bottom (0 → 2048). The optic tectum / hindbrain boundary therefore appears as a roughly vertical curve when you look at the XY plane — it runs from the top of the brain (small Y) to the bottom (large Y), at some X position. Everything to the **left** of the curve (smaller X) is the optic tectum; everything to the **right** (larger X) is the hindbrain.

Step-by-step:

1. **Scroll to a representative Z slice** where the optic tectum / hindbrain boundary is most clearly visible. In zebrafish 4dpf, this boundary is typically a recognisable change in cell density roughly at the mid-point of the anterior–posterior (X) axis.

2. **Add a Shapes layer:** In napari, click the + icon in the toolbar and choose **Shapes**, or go to **Layers → Add shapes layer**.

3. **Select the path tool:** In the shapes toolbar (appears when the Shapes layer is active), click the **path** icon (a polyline with multiple vertices). Do **not** use the straight line tool — the optic tectum / hindbrain boundary is curved.

4. **Draw the boundary curve from top to bottom:** Start clicking at the top of the brain (small Y, Y ≈ 0 side) and work downward to the bottom of the brain (large Y). Follow the curved anatomical boundary as you click. Double-click on the last point to finish. You typically need 4–10 click points.

    - The optic tectum (anterior, smaller X) will be on the **left** of your drawn path.
    - The hindbrain (posterior, larger X) will be on the **right**.

    > Tip: zoom in on the XY view where the boundary is clearest. If unsure of the exact position, trace the curve slightly anterior (more to the left). You can always select the path layer, press Delete to remove it, and redraw.

5. *(For three or more regions)* Draw one additional path per boundary, each running top to bottom.

6. **Switch to Tab 4 — Statistics** in the plugin.

7. Under **Brain regions**:

    - **Boundary lines:** select your Shapes layer from the dropdown.

    - **Region names:** type the names separated by commas, anterior to posterior:

      `Optic tectum, Hindbrain`

      For three regions, e.g.: `Forebrain, Optic tectum, Hindbrain`

8. Click **Generate Statistics**.

Result: The CSV will include two extra columns:

Column	Description
`brain_region`	Name of the region each cell belongs to
`region_boundary_dist_um`	Distance in μm from the cell to the nearest boundary line

You can then filter the CSV in Excel or Python by `brain_region` to compare microglia density, morphology, or intensity between the optic tectum and the hindbrain.

---

15. Reinstalling after an update

```
pip uninstall napari-zf-microglia-ai -y
pip install git+https://github.com/CTichy/ZF-Microglia-AI.git
```

Then **fully close and reopen napari**. If napari is running when you reinstall, it uses the old version until restarted.

> **Your model path and settings are preserved** across reinstalls. The config is stored in `~/.config/napari-zf-microglia-ai/config.json`.

---

16. Troubleshooting

napari crashed / was killed — how much did I lose?

If Autosave Labels Layer (Tab 6 — Sweeps & Utilities, General) is checked — the default — the one Labels layer picked in Tab 3's own shared layer selector is auto-saved to `<output folder>/<layer_name>_recovery.tif` at the interval set there (default every 30 minutes), overwriting the previous recovery save each time — independent of Save Labels, and independent of whatever caused the crash. Look for a `*_recovery.tif` file next to your fish's other output files; at most one interval's worth of manual editing (Correct Label, Correct Adjacent Labels, Split/Join Labels, etc.) should be missing from it. If nothing is there, check that checkbox was on and a Labels layer was actually selected in Tab 3 at the time.

If napari itself was silently killed (no error dialog, the window just vanishes) rather than crashing with a visible Python traceback, that's almost always the Linux OOM-killer, not a bug in whatever you were doing at the time — confirm with `journalctl -k --since "-1 hour" | grep -i "out of memory"` in a terminal; it will name the exact process and how much memory it had grown to. Every background-worker QTimer in this plugin releases its own result once done, so a completed

operation (Correct Label, segmentation, sweeps, etc.) doesn't hold memory for the rest of the session — the recovery file above is the safety net regardless.

---

napari was killed during Auto-correct with 100+ GB of RAM in use (fixed)

Older versions could exhaust all RAM in the auto-correct's first wave: for cells classified as touching skin, 6 cell workers each started their own 6 slice threads, and every slice attempt copied the entire label volume (1.7 GB for a 101×2048×2048 fish). The OS memory guard (`systemd-oomd`) then killed napari's terminal (confirm with `journalctl --since today | grep -i oomd`). This is fixed: slice work uses a one-slice view, the thread budget is shared instead of multiplied, and RAM stays around 35 GB on a 33-cell fish regardless of cell count. If napari is still killed, keep System Monitor open, note the RAM at the moment it happens, and send the `journalctl` lines.

Auto-correct or Protect Skin seems stuck / takes very long

Typical times on one real fish (33 cells, 101×2048×2048) with a reduction of 0: **Protect Skin ≈ 7 min** (about 6.5 min is the per-slice skin trim), **Auto-correct ≈ 4 min**. **Reduce best_lo by** (Tab 3, 7d, default 2) makes both genuinely slower than these figures — a lower threshold means real extra work for skin's per-slice watershed/auto-grow, not a bug, since skin's own working area already spans nearly the whole frame on most slices; the more you reduce, the longer to expect. Crowded fish with 70–90 cells take longer too. Watch the **status line**: it names each step, and the last steps are *"removing debris…"*, *"checking for possible non-microglia blobs…"*, *"Computation finished — applying the corrected labels to the viewer…"* and *"Applying the calibrated contrast to the signal layer…"* — so you can tell whether the computation or the hand-over to napari is slow. The terminal napari was started from prints how long the hand-over took, and — if a run is still going after 30 minutes — every thread's stack, repeated every 30 minutes. If the same lines repeat, send that output.

Protect Skin says “Skin is ALREADY protected”

The selected labels layer already contains the skin label (-1), so the button did nothing on purpose. Continue with Auto-correct (it reuses that skin), or click **Remove Skin Label** first if you want to protect from scratch.

Re-run This Cell Only says no labels layer was found (fixed)

Works on the labels layer you have **selected**, whatever it is called (for example `<stem>_brain_only_ExtRm_labels` from **Load Labels layer**). If you select the volume instead it looks for `<image>_cellpose_labels`, then `<image>_labels`, then a single same-shape labels layer whose name starts with the volume’s; if several qualify it asks you to click the one you mean.

`conda env create -f environment.yml` fails on Windows with `Didn't find wheel for cucim-cu12`

Fixed in the current `environment.yml` (`cucim-cu12` is now Linux-only there — it has no Windows wheels at all, since it’s a Linux/WSL2-only RAPIDS package, and isn’t something a different pip flag or index fixes on native Windows). It only accelerates Tab 4 statistics, which fall back to CPU cleanly without it; Tab 1 and Tab 2 are unaffected.

If you still hit this error:

- You’re on an older clone — `git pull` in the repo folder, then retry.
- If the environment partially exists from the earlier failed attempt, remove it first: `conda env remove -n zf-microglia-ai`, then `conda env create -f environment.yml` again.

The plugin does not appear in Plugins menu

- Make sure napari is fully closed and reopened after installation.
- Verify installation: `pip show napari-zf-microglia-ai`

“No model selected” after reinstalling

- Click `[...]` and browse to your `.pth` file.
- Config path: `~/.config/napari-zf-microglia-ai/config.json`

Tab 5 (AI Tools) is showing a red or amber warning banner

This is expected, not an error — see Section 9. The tab is always available regardless of GPU; the banner just tells you what to expect. No CUDA GPU means CPU fallback (days-months for a full training run instead of hours); a GPU under 8GB may still work with a lower `batch_size` (try 2, or even 1) before assuming something else is wrong.

“Launch Training” errors that a script wasn’t found

`prepare_data.py`/`train.py`/`train_xzyz.py` ship with the plugin (bundled under `napari_zf_microglia_ai/training_scripts/`, installed as package data), so this shouldn’t normally happen. If you want to point at a locally modified copy instead, override the path in `~/.config/napari-zf-microglia-ai/config.json` — keys `monai_prepare_script_path`, `monai_train_script_path`, `cellpose_train_script_path`.

Processing runs on CPU (very slow)

NVIDIA GPU: Check that PyTorch sees CUDA:

```
python -c "import torch; print(torch.cuda.is_available())"
```

Should print `True`. If `False`, reinstall PyTorch with CUDA support from https://pytorch.org.

Apple Silicon: Check MPS:

```
python -c "import torch; print(torch.backends.mps.is_available())"
```

Should print `True` on M1/M2/M3.

Statistics are slow (CPU only, no GPU batch)

Linux: Install CuPy and cuCIM for GPU-accelerated regionprops:

```
pip install cupy-cuda12x cucim-cu12
```

Replace `cuda12x`/`cu12` with your actual CUDA version if different (e.g. `cuda11x` for CUDA 11). After installing, reopen napari — the console will print `regionprops: cuCIM GPU` when statistics are computed.

Windows: `cucim-cu12` has no native Windows build (RAPIDS/cuCIM is Linux/WSL2-only) — this is expected and not fixable with a different pip command on native Windows. Tab 4 statistics run on a CPU-threaded path instead; Tab 1 inference and Tab 2 GPU labelling are unaffected, since those only need `cupy-cuda12x`, which does support Windows. If GPU-accelerated statistics specifically matter to you, run the plugin inside WSL2 (Windows Subsystem for Linux) instead, where the Linux install path applies.

`brain_only` layer looks mostly empty (all black)

BG Threshold is too high — lower it (e.g. from 1.05 toward 0.50-0.60).

Create Labels finds 0 or too few objects

- Lower **Final min-size fraction** (Min volume itself is read-only).
- Increase **σ XY** and **σ Z** slightly.
- Make sure you selected the `_NoBG brain_only` layer (not the raw channel) before clicking Create Labels — the algorithm treats every non-zero voxel as signal.

Create Labels finds hundreds of tiny fragments

- Raise **Final min-size fraction** (toward 1.0).
- Ensure Option 2 with sufficient BG Threshold was used — the brain_only layer must have clean gaps between cells.

Two cells appear as one label (merged)

- Use **Split Label** (Section 7 above) to separate them at the thinnest neck.
- Or decrease σ XY and rerun Create Labels.

Split Label error: “Only N sub-volume(s) found”

The blob doesn’t have a clear separation into the requested number of parts.

- Reduce **Smooth σ** (the distance field is over-smoothed and the saddle disappears).
- Reduce **Min distance** (the two centres are being rejected as too close).
- Check the blob is genuinely two distinct cells — zoom in and inspect it slice by slice.

Ollama description shows `[Ollama error: ...]`

- Verify Ollama is running: open a terminal and run `ollama list`.
- If not running, start it: `ollama serve`
- Check the model name matches exactly: `ollama list` shows available models.
- Default endpoint `http://localhost:11434` — change only if Ollama is on a different machine.

OpenAI/Claude API returns an error

- The API key is saved encrypted (OS credential store, or a local encrypted-file fallback on Linux if that’s unavailable — see Section 10h) and prefilled from last session — double check it’s still valid (a regenerated/revoked key won’t match what’s saved).
- Check your account has billing set up and enough credit.
- The model name must match exactly (e.g. `gpt-4o-mini`, `claude-haiku-4-5-20251001`).

“Run Cellpose-SAM Segmentation” errors with `No module named 'cellpose'`

Install it in your environment: `pip install cellpose`. Already included in `environment.yml`/`environment-mac.yml` for fresh installs — if you set up your environment before this feature was added, run `conda env update --name zf-microglia-ai -f environment.yml --prune` (or `environment-mac.yml`) and reinstall.

Neither Pixel Classifier nor Cellpose-SAM section shows up in Tab 2

An Image layer whose name ends in `_ExtRm` (Cellpose-SAM) or `_NoBG` (Pixel Classifier) must be open in the viewer — run Tab 1 with Option 1 or 2, or open that output file. See Section 6a.

Quick Reference Card

Tab 1 — Skin Remover

Control	Recommended	What it does
MONAI Threshold	0.38	AI confidence cutoff (a Recommended line below the slider shows the GT-verified value once measured)
MONAI Erosion	0	Strips voxels from mask edge
Background	Option 1 (Cellpose-SAM) / Option 2 (Pixel Classifier)	Option 1 = `_ExtRm`, Option 2 = `_NoBG` (background removed globally)
BG Threshold	1.025	Background removal level, % of data range above the in-brain mode (Background mode 2 only)
Signal Erosion	0	Erodes the retained signal per slice

Control	Recommended	What it does
		(Background mode 2 only)

Tab 2 — Create MG Labels

Each section is shown when a layer with its suffix is open in the viewer — `_ExtRm` → Cellpose-SAM, `_NoBG` → Pixel Classifier, both if both (see 6a).

Common Settings (always visible, above both routes)

Control	Recommended	What it does
Min volume	7069 (default; only ever moves lower)	**Informative only, not editable** — Pixel Classifier's cutoff basis **and** Cellpose-SAM's Safe-merge "already a whole cell" floor, one shared value; only moved by a GT-verified GT Toolkit Tuning run
Max volume	not yet measured (only ever moves higher)	**Informative only, not editable** — largest cell ever confirmed real by GT (GT Toolkit Tuning); drives no pipeline stage, only flags `is_volume_outlier` (Tab 4 CSV)
Min hole size	5 (default)	Editable slider — shared by both routes — minimum voxels for an enclosed gap to survive as real background instead of being filled
Final min-size fraction	0.618 (golden ratio)	Editable slider — both routes — the real deletion cutoff is this fraction of Min volume, not Min volume itself (Cellpose-SAM: last-stage safety net; Pixel

Control	Recommended	What it does
		Classifier: the volume filter's own cutoff)
Soften label contours (sanding)	Checked	Shared by Correct Label, Correct Adjacent Labels, Auto-correct Labels, and Cellpose-SAM auto-correct — sands the touched label(s) right after each, foreign-protected, purely geometric
Fill interior cavities (vesicles) before sanding	Checked	Same tools, runs right before the blur — fills a real 3D-enclosed vesicle/vacuole cavity the intensity threshold correctly excluded as "not signal", never touches the outer contour (concave or not); cells only, never skin
Sanding sigma XY / Z (in Tab 3)	0.7 / 0.7 vox	Blur strength for sanding — deliberately much smaller than Pixel Classifier's Smooth σ XY/Z below (different job: polishing an existing label, not merging blobs into one)

Pixel Classifier (on screen: "Pixel Classifier — 3D Connected-Component Labels")

Control	Recommended	What it does
Smooth σ XY	2.6	XY strength of the 3D Gaussian blur applied before thresholding
Smooth σ Z	1.0	Z strength of the same 3D Gaussian blur —

Control	Recommended	What it does
		bridges small Z gaps before thresholding

Cellpose-SAM Segmentation

Control	Recommended	What it does
Signal layer	first `_ExtRm` layer	The one Image layer segmented; outputs are named after it
Min size	15 vox	Cellpose-SAM only, not shared with Common Settings' Min volume — tiny early noise filter
Cellprob threshold	2.84	Confidence cutoff for foreground vs. background
Flow iterations (niter)	200	Euler-integration steps for flow-following before instances are formed
Safe-merge max gap	3.52 µm	Max physical gap allowed when merging fragments
Safe-merge min contact	1 vox	Min touching surface required to merge
Large-contact merge	20 vox	Second merge pass for thick-junction splits
Auto-correct labels via contrast sweep after segmentation	Checked	Chains the full auto-correct pipeline (and sanding, if on) onto the run; needs the Brain mask layer and the raw channel layer open
Re-run This Cell Only	—	Fixes one label without redoing the whole fish — crops, re-runs do_3D + cleanup on just that label, splices the result back in. Works on the selected labels layer (any name), or finds `<image>_cellpose_labels` /

Control	Recommended	What it does
		`<image>_labels` for the selected volume

Tab 3 — Edit MG Labels

Every edit/correct/modify tool for an existing Labels layer — see Section 7.

Layers and label(s) being edited (shared selector, top of tab)

Control	What it does
Signal layer / Labels layer / Brain mask layer	Picked once here — every tool below reads from this same choice, not its own field
Label A	The only label a single-label tool (Split, Correct Label, Copy Label to Adjacent Slice) acts on
Label B	Used together with Label A by Join Labels and Correct Adjacent Labels
Use selected (x2)	Click a label in the viewer, then click Label A's or Label B's own button to copy its ID in

Correction Settings (Bbox / Auto-grow / Stability)

Control	Recommended	What it does
Bbox padding	15 px	Shared by Correct Label, Correct Adjacent Labels, Protect Skin as Label, and Auto-correct Labels (standalone or chained onto Cellpose-SAM Segmentation)
Auto-grow until signal clears the border	Checked	Retries with a bigger pad (Growth step, up to Max growth iterations) whenever the result still touches its own working-area edge
Growth step	5 px	

Control	Recommended	What it does
		How much the pad grows per retry
Max growth iterations	20	Hard cap on retries
Keep re-running until the shape stabilizes	Checked	Re-runs the correction, feeding each pass's result into the next, until it stops changing or Max stability passes is reached — independent of Auto-grow
Max stability passes	50	Hard cap on stability re-runs

Sanding

Control	Recommended	What it does
Sanding sigma XY / Z	0.7 / 0.7 vox	Blur strength for sanding (enabled via the checkbox in Create MG Labels) — deliberately much smaller than Pixel Classifier's Smooth σ XY/Z (different job: polishing an existing label, not merging blobs into one)

Contrast low (best_lo) calibration

Control	Recommended	What it does
Auto-sweep for best contrast low (best_lo)	Checked	Read by Protect Skin as Label and Auto-correct Labels — sweeps for best_lo against the active labels (same sweep as Calibrate Correct-Label Contrast, Tab 6); unchecked uses the Signal layer's own

Control	Recommended	What it does
		current contrast low instead, no sweep
Reduce best_lo by	2 (range 0-6)	Subtracted from the swept best_lo before correcting, to capture slightly more signal — ignored when Auto-sweep is off

Editing tools

Control	Recommended	What it does
Resort Labels	—	Renumbers the selected Labels layer 1…N by size, centroid Z/Y/X or complexity (Reverse order checkbox)
Remove Debris	—	Sweeps the selected Labels layer for anything below Final min-size fraction × Min volume, skin included
Split mode	3D (whole label)	2D restricts the split to the current slice only — for artifacts that only touch on one cross-section
Split σ	1.0	Smoothness for watershed split
Min distance	5	Peak separation for split detection
Join Labels	—	Merges Label B into Label A — the inverse of Split Label
Correct Label	2D mode	Regenerates Label A's shape from the shared Signal layer's live contrast window — 2D (current slice) or 3D (whole cell, per-slice

Control	Recommended	What it does
		local areas, joint correction wherever a real neighbor is nearby, auto debris cleanup, reports nearby/touching foreign labels and any slice still touching its own local area's edge) — optional auto-grow retries with a bigger pad if signal reaches the edge (per-slice in 3D), auto-folding in neighbors instead of encroaching; 3D also has an independent "keep re-running until stable" switch that re-seeds the joint watershed from each pass's own result until the shape stops changing
Copy Label to Adjacent Slice	—	Copies Label A's shape from the current slice onto the next/previous slice
Correct Adjacent Labels	2D only	Jointly corrects Label A and Label B on the current slice (rectangle scoped to Label A alone), cut placed by watershed seeded at each label's own existing footprint — same optional auto-grow as Correct Label
Protect Skin as Label	—	Fills everything outside the shared Brain mask layer with label **-1** (always, never Label A/B), trims it to real signal per-slice using the Contrast low (best_lo)

Control	Recommended	What it does
		calibration controls above (parallel, no brain-mask clamp), then sweeps stray debris it absorbed. Reports which real labels end up touching/nearby skin, worth a closer look. Refuses to run if skin is already protected
Remove Skin Label / Hide skin label in the viewer	—	Both always act on label **-1** — clear it back to background, or make it transparent in the display only (no data change)
Auto-correct Labels	—	Runs the full Cellpose-SAM auto-correct pipeline (contrast sweep per 7d, skin protection, per-cell 2D-vs-skin/3D correction, debris cleanup) against the selected Labels layer as it stands — for a labels layer segmented earlier, any route or session. Reuses skin if already protected, refuses to correct cells without skin, decides “touching skin” by geometry (≤ 3 px), and lists possible non-microglia blobs (report only). If sanding is on, chains the same fill-cavities + soften-contours stage afterward as the Cellpose-SAM route does — never saves the labels itself (only its report `.txt`)

Control	Recommended	What it does
Save Labels	—	Saves the selected Labels layer as zlib-compressed `int32` TIFF

Tab 4 — Statistics

Control	Options	What it does
Description	Rule-based / Ollama / OpenAI / Claude	Engine for the description column
Image layer	Any Image layer / None	Adds intensity statistics (mean, integrated, CV)
Boundary lines	Any Shapes layer / None	Assigns cells to named brain regions
Region names	Comma-separated text	Names for each region (N lines → N+1 names)
Output columns	All / Reset	Checklist of which columns go into the CSV
Generate Statistics	—	Computes up to 52 CSV columns per label for the selected (else topmost) Labels layer, saves CSV

Tab 5 — AI Tools

Always shown — a banner at the top warns if your GPU is missing or under the recommended 8GB (see Section 9), but doesn't block anything. Switch below it picks MONAI or Cellpose-SAM training.

Control	Default	What it does
n_val / n_test	5 / 5	(MONAI) fish held out for val/test in Prepare Training Data
epochs	1500	(MONAI) training length
n_epochs	200	(Cellpose-SAM) training length
branch_weight	0	(Cellpose-SAM) 0 = standard loss; >0 weights thin/branch pixels more heavily
branch_radius	5 px	

Control	Default	What it does
		(Cellpose-SAM) erosion-survival distance threshold for the branch-weighted loss — measurable from real GT via Calibrate branch_radius below
Calibrate branch_radius (from GT)	—	(Cellpose-SAM) measures real branch thickness from a GT labels volume (3D skeleton + distance transform) — reports only; applied to branch_radius and saved only via the GT Toolkit Tuning Tool with GT verified
pretrained	Tab 2's checkpoint	(Cellpose-SAM) starting point — "continue training" by default
Extract X/Y/Z Patches	crop_size=512, skin crops 20% / 10%	(Cellpose-SAM) generates training crops in 3 orientations, adds cell-skin boundary and skin-only crops when the GT has the skin label (-1), cleans truncated labels by default
Combine Crops into Shared Training Folder	—	(Cellpose-SAM) symlinks one fish's crop folder into a shared multi-fish pool, prefixed with its own full data-folder name
Patience (checkpoints)	5	Both — stop after N checkpoints with no improvement (Dice/ test_loss); 0 disables
Launch Training	—	Starts a detached process that survives closing napari; GUI reconnects automatically next time
(on stop, Cellpose-SAM only)	—	Writes `<model_name>_best_recommended.txt` in `models/` — a pointer, not a copy, to the best-test_loss checkpoint
Stop Training	—	Kills the training process and its children

Tab 6 — Sweeps & Utilities

Twelve tools consolidated from Tabs 1, 2, 4 and 5, each individually collapsible — see Section 10 for full detail. Every row reads from/writes back to the tab noted in parentheses. A “Show tools for…” filter at the top of the tab (4 checkboxes: Skin Removal / Pixel Classifier / Cellpose-SAM / General, all on by default) hides whichever categories you don’t need. The individual GT sweeps only report when run on their own; their results are applied only by the GT Toolkit Tuning Tool with GT verified.

Control	Scope	What it does
GT Toolkit Tuning Tool	one fish	(General) Scans one fish’s folder, fills every sweep’s inputs, measures Min/Max volume + Min hole size from GT, then runs the MONAI, BG Threshold, Smooth σ, Cellprob + Krendl and branch_radius steps its files allow — **the only tool that applies GT-sweep results, and only with its “This is verified ground truth” box ticked**
Verify MONAI Threshold / Erosion (GT Sweep)	5x5 grid, Sieve on by default	(Tab 1) Scores current values against a hand-corrected GT brain mask — MONAI runs once, rest is cheap — cross-fish average applied via the GT Toolkit
Verify BG Threshold / Signal Erosion (GT Sweep)	5x5 grid, Sieve on by default	(Tab 1/2) Scores BG Threshold/Signal Erosion against real GT IoU, and measures Min volume + Min hole size as never-rising floors from GT — CPU-OK, doesn’t survive closing napari — applied via the GT Toolkit
Verify Smooth σ XY / σ Z (GT Sweep)	grid, Sieve on by default	(Tab 2) Scores Smooth σ XY/Z against real GT IoU, BG Threshold/Signal Erosion held fixed — CPU-OK, doesn’t

Control	Scope	What it does
		survive closing napari — applied via the GT Toolkit
Calibrate Cellprob + Krendl Merge Parameters + Min size + Large-contact (GT Sweep)	5 calibration steps, Sieve on by default, full fish, ~3h total	(Tab 2) Against whole-fish GT — do_3D's network pass runs exactly once (~3h on a full-size fish, GPU-preferred), reused for Cellprob, then max_gap/ min_contact, Min size, and Large-contact are each calibrated from that same cached pass with no extra inference — averaged Cellprob + pooled recommendations applied via the GT Toolkit
Verify Best Epoch (GT Sweep)	5 cells, ±2 checkpoints	(Tab 5, Cellpose-SAM) confirms the recommendation against real GT IoU/Dice, not just test_loss — doesn't survive closing napari — **if the sweep disagrees and its own GT box was ticked, rewrites the pointer to the confirmed epoch and loads it as Tab 2's active model**
Calibrate Correct-Label Contrast	400 samples (20 cells x 20 slices)	(Tab 3, Edit MG Labels) the canonical best_lo sweep every autosweep-driven tool (Protect Skin as Label, Auto-correct Labels) correlates to — finds the lower-contrast value that best reproduces the active Labels layer's own shapes (mean IoU), not independent GT — reads its Signal/Labels layers from Tab 3's shared selector, **auto-applies the winning value to the chosen signal layer's contrast limits (low = winning value, high = an adaptive percentile of this fish's own real signal)**

Control	Scope	What it does
Score Against GT	any 2 Labels layers	Whole-fish Hungarian-matched TP/FP/FN/Score/ MeanIoU/MeanDice between any two Labels layers — synchronous, no GPU needed
Drift View in 3D	—	(Tab 6, General) Start/Stop button + Speed slider — slowly, continuously tumbles the 3D camera around whatever's in view, switching to 3D display automatically; camera-only, never touches layer data
Autosave Labels Layer	Checked, every 30 min	(Tab 6, General) Periodically saves ONLY the Labels layer selected in Tab 3's own shared selector to `<output>/ <name>_recovery.tif` — a crash loses at most one interval's worth of manual editing. Interval is minutes/hours, editable; no other layer is ever included
Build GT-Correction Package	—	(Cellpose-SAM) Zips the most-advanced correction stage available (sanded > auto-corrected > Krendl-only) + stats CSV + creation guide + brain_mask.tif and raw signal channel (both needed for Protect Skin as Label on the reviewer's end) + the auto-correct report (.txt) for external manual correction — browsing Source image (or a GT Toolkit scan) auto-fills the rest
Email notification (optional)	*(blank = off)*	Shared SMTP credentials (address, server, port, username, password — password saved **encrypted** via

Control	Scope	What it does
		the OS credential store); nothing is sent while Notify email is blank — see Section 10h
Email me when a job finishes or fails	On	The plugin's one email switch: every background job, success or failure; training runs email whenever they stop
only if it ran longer than	10 min	Shorter jobs never email; `0` = every job
Send Test Email	—	Sends one email immediately (no GPU, no waiting) to confirm SMTP settings before relying on them for a long run

---

Plugin developed at FH Technikum Wien — Artificial Intelligence & Data Science Contact: carlos.tichy@gmail.com